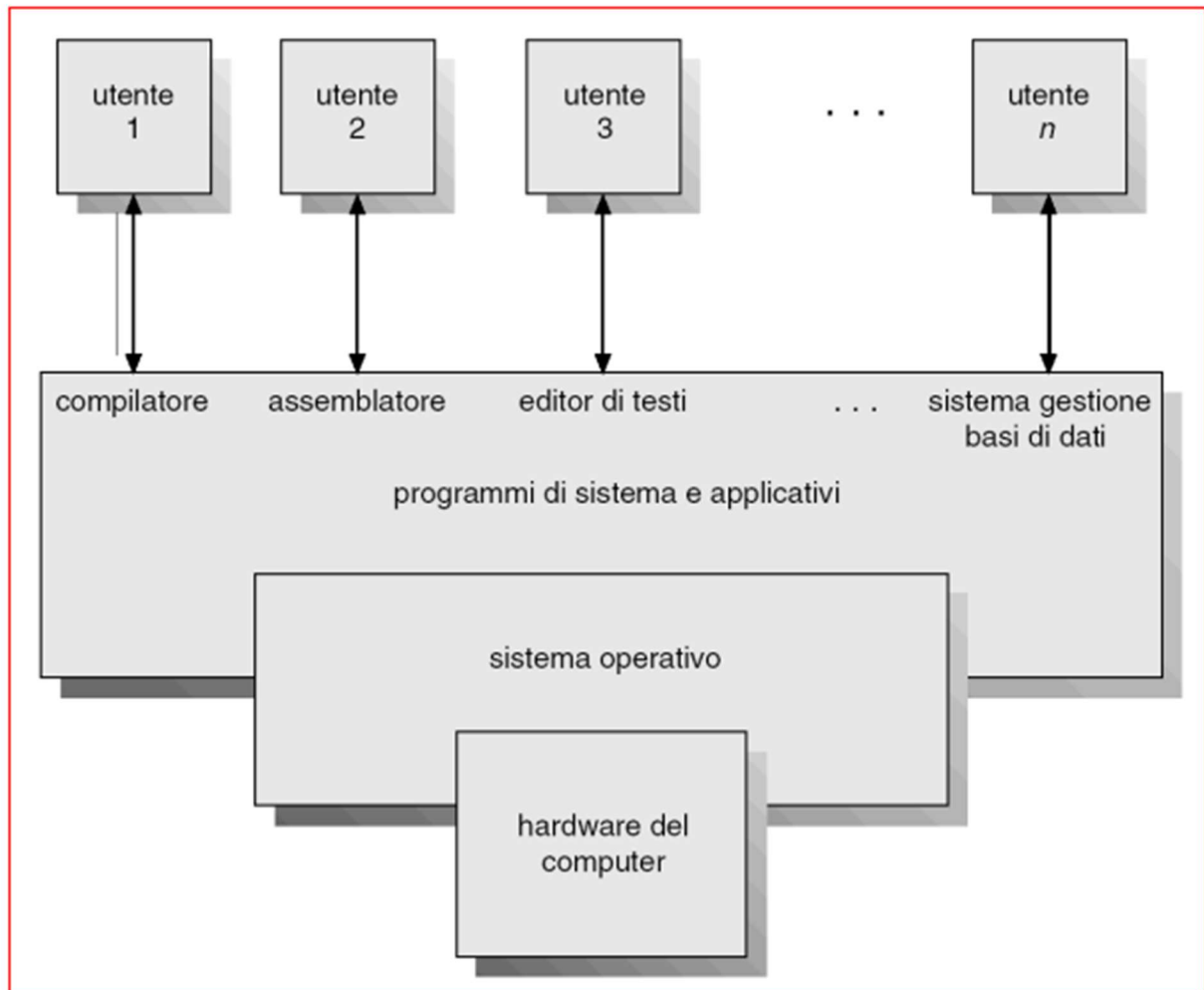


Capitolo 1

Definizione di sistema operativo: Un insieme di programmi che agisce da intermediario tra utente e hardware di un computer e hanno lo scopo di eseguire i programmi utente e risolvere più facilmente i problemi, sono formati da tre parti

1. **Hardware:** che solo la parte fisica es. cpu, ram,i/o
2. **SO:** controlla e coordina l'uso dell'hardware da parte dei programmi applicativi attivati dagli utenti.
3. **Programmi applicativi:** definiscono come le risorse dell'hardware sono utilizzate per risolvere problemi di elaborazione dei dati



Schema dell'elaborazione dell'informazione

Un sistema operativo svolge 3 principali funzioni

1. Distributore di risorse
2. Programma di controllo
3. Kernel

Difatti distribuisce le risorse e ne gestisce l'utilizzo, controlla come vanno i programmi degli utenti e se ci sono delle operazioni di tipo Input/output, e soprattutto è l'unico programma che è sempre in esecuzione, tutti i restanti programmi sono di tipo applicativo

JOB= PROCESSI

SISTEMI MAINFRAME

Si suddividono in

- **Batch**
- **Multi programmati**

I sistemi batch riducono i tempi di processo raggruppando i job in batch ossia dei lotti con delle necessità simili, ordinando in modo automatico i processi e trasferendo il controllo da un job all'altro Resident monitor:

- il controllo inizialmente è del sistema;
- il controllo viene trasferito al job;
- quando il job viene completato il controllo ritorna al sistema.

Quelli multi-programmati invece, mantiene contemporaneamente più processi in memoria e la cpu li risolve uno alla volta, per far sì che questi SO funzionino sono richieste diverse caratteristiche ossia:

- Procedure I/O fornite dal sistema.
- Job scheduling (analisi di disponibilità delle risorse).
- Gestione della memoria tra più processi
- CPU scheduling- il sistema deve scegliere tra i vari processi pronti l'ordine di esecuzione
- Gestione della cooperazione tra job
- Gestione della concorrenza tra job

Nell'esecuzione di un job, essi vengono spostati dal disco alla memoria principale e viceversa, la comunicazione tra un job e l'altro viene instaurata dall'utente tramite sistemi di i/o .

Si usano metodologie di **time sharing** per cercare di far avvenire in modo simultaneo i vari job caricati in memoria in questo modo:

- ☐ La **CPU è condivisa** tra più processi/utenti. Usata a turni da diversi job
- ☐ Ogni processo riceve una piccola **quantità di tempo CPU** (chiamato **time slice** o **quantum**), per esempio 10 millisecondi.
- ☐ Alla fine del time slice, il sistema **salva lo stato** del processo (context switch) e passa al successivo.
- ☐ Il ciclo continua, dando **l'illusione di simultaneità**.

Uno svantaggio è che se ci sono diversi utenti e diversi lotti, il time slice diventa sempre più piccolo

SISTEMI PARALLELI

Sono sistemi che possiedono più processori in comunicazione tra loro, che condividono la memoria e il clock, la comunicazione avviene attraverso la memoria condivisa.

Abbiamo diversi vantaggi:

- Maggiore quantità di elaborazione effettuata
- Economia di scala
- Aumento di affidabilità- fault tollerant, graceful degradation

Possiamo effettuare due tipi di sistemi multiprocessore

1. **Asimmetrico**: in cui ogni processore è assegnato ad uno specifico lavoro e comunicano tutti con il master che organizza il lavoro dei processori slave
2. **Simmetrico**: in cui ogni processore esegue una copia del SO e possono essere eseguiti in maniera contemporanea molti processi senza che si producano deterioramenti alle prestazioni, tuttavia non sono esenti a problematiche come la non gestione accurata delle cpu e della condivisione delle risorse e delle strutture dati
3. **DISTRIBUTI**: In cui il calcolo viene distribuito tra diversi processori fisicamente distinti che sono debolmente accoppiati, ogni processo possiede una propria memoria locale e comunicano tra loro solo tramite bus. Questi sistemi hanno numerosi vantaggi come:
 - Condivisione delle risorse
 - Rapidità di calcolo
 - Affidabilità
 - ComunicazioneQuesti però necessitano di una lan o wan per comunicare e possono essere sistemi di tipo-client server o peer-to-peer
Certo! Ecco un riassunto chiaro e ordinato, basato sulla slide, adatto per i tuoi appunti:
4. **Sistemi Cluster – Riassunto**
 - **Clustering**: tecnica che consente a due o più sistemi di condividere risorse hardware, come i dischi.
 - **Obiettivo principale**: garantire **alta disponibilità** dei servizi, grazie alla **ridondanza**.
 - **Funzionamento in caso di guasto**:
 - Se una macchina ha un malfunzionamento, il **cluster head** può "prelevare" le sue risorse.
 - Il cluster head può riavviare il servizio su un altro nodo.
 - Per l'utente l'interruzione è minima o quasi impercettibile.
 - Tipi di cluster:
 - **Asimmetrico**: solo un server è attivo, gli altri sono in attesa (**hot-standby**).
 - **Simmetrico**: tutti i server lavorano insieme e si controllano a vicenda.
 - **Evoluzione**: uso di **reti SAN** (Storage Area Network) per la memoria condivisa.
5. **Sistemi Real-time**: funzionano in un vincolo di tempo entro la quale l'uscita è accettabile e possono essere di tipo hard o soft real-time:
Caratteristiche principali Hard real-time:
 - **Vincoli rigidi sulla capacità di calcolo** → devono essere progettati con attenzione per assicurare che ogni operazione finisca entro il tempo previsto.
 - **Poca o nessuna memoria di massa** → usano memorie veloci e affidabili (es. RAM o ROM), così da rispondere subito.
 - **Non compatibili con i sistemi time-sharing** → cioè quelli che alternano tra più utenti/processi (come Windows o Linux per uso normale).
 - **Non supportati dai sistemi operativi generici**, perché questi separano l'utente dall'hardware e non garantiscono tempi precisi. Esempio d'uso: sistemi per airbag, macchinari medici, reattori nucleari.

2. Soft Real-Time (tempo reale lasco)

 In questi sistemi i **tempi sono importanti, ma non critici**. Se c'è un piccolo ritardo, il sistema può comunque funzionare, anche se con prestazioni peggiori.

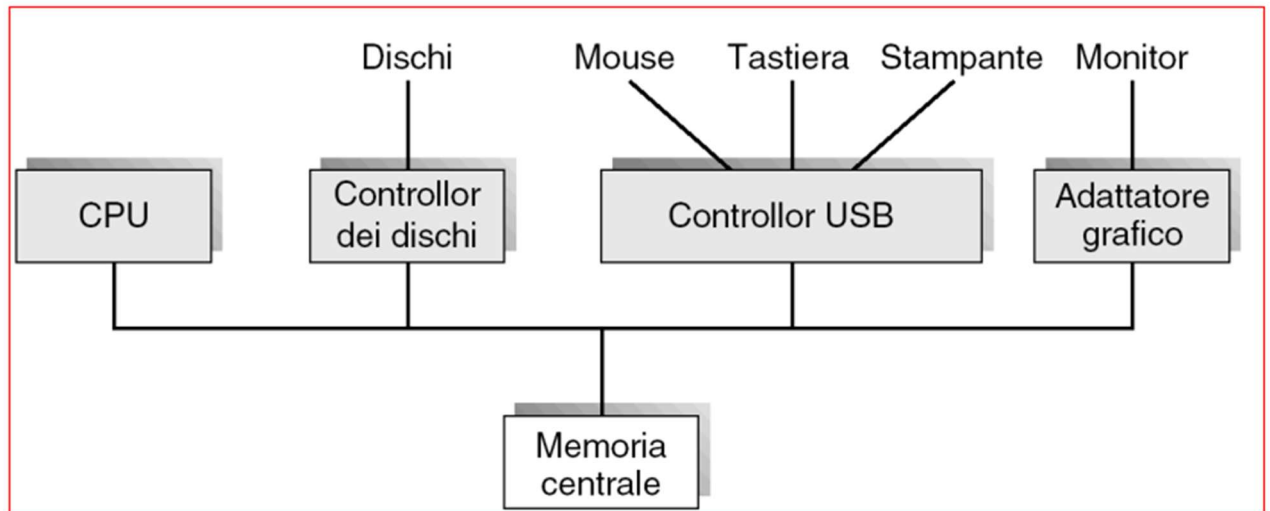
Caratteristiche:

- **Gestione a priorità dei task critici** → quelli più importanti ricevono più attenzione.
- **Possono convivere con sistemi time-sharing** → adatti ad ambienti più flessibili.

- **Non garantiscono tempi certi** → ma cercano comunque di rispettare i vincoli temporali il più possibile.
 - **Meno usati nel controllo industriale/robotica** rispetto agli hard real-time.
 - **Utili in campi avanzati** → come **multimedia** (es. video in tempo reale), **realtà virtuale**, videogiochi.
6. **Sistemi palmari- o mobili:** sono telefoni cellulari, e hanno a disposizione memoria limitata, processi lenti e display con dimensioni ridotte

CAPITOLO 2

Struttura generale di un PC



La cpu e i controller di i/o operano in modo concorrente, ogni controller è incaricato di gestire una specifica periferica o diverse simili, ogni controller ha una memoria temporanea locale detta *buffer*. La cpu sposta i dati dalla memoria principale ai buffer locali e viceversa, l'input procede dal dispositivo fino al buffer locale del controller, e dunque l'output fa il percorso opposto ossia dal buffer locale al dispositivo.

Il controller dei dispositivi informa la cpu che ha concluso la sua operazione generando una interruzione detta *interrupt*

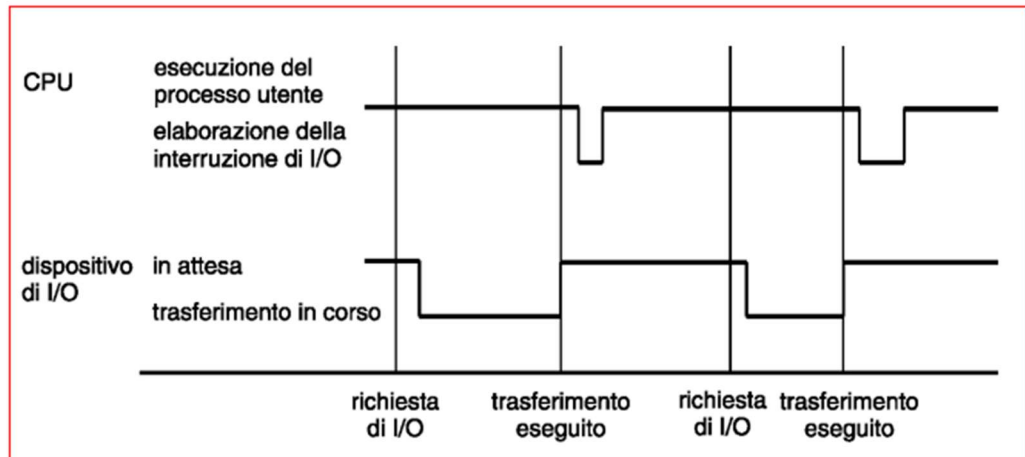
Funzionamento interrupt:

ogni interrupt trasferisce il controllo della cpu alla routine di gestione interrupt, questo viene fatto attraverso un vettore di interrupt che contiene gli indirizzi di tutte le routine di servizio, la gestione interrupt deve salvare l'indirizzo dell'istruzione interrotta e lo stato del processore.

Gli interrupt relative alle periferiche possono essere disabilitati dalla cpu durante l'esecuzione di istruzioni critiche per prevenire la perdita di segnali.

Una *trap* è un interrupt generato dal software e può essere causato da un errore o da una richiesta utente tramite operazione di i/o

Esecuzione di un interrupt per un singolo processo in uscita



- Il sistema operativo preserva lo stato della CPU memorizzando i registri ed il program counter.
- Determina il tipo di interrupt ricevuto
- polling.
- sistema a interrupt vettorizzato.
- Porzioni di codice separate determinano quali azioni devono essere eseguite per ogni tipo di interrupt.

Perfetto, andiamo **passo per passo**! Ti guiderò nella **gestione completa di un interrupt**, come se lo stessi simulando insieme.

GESTIONE DEGLI INTERRUPT — PASSO PER PASSO

◆ 1. Generazione dell'interrupt

Un **evento esterno o interno** genera un interrupt:

- Es. premi un tasto, finisce la lettura da disco, un timer scade...

 Il **dispositivo** manda un **segnale elettrico** alla CPU per dirle:

"Ehi, ho bisogno di attenzione!"

◆ 2. Interruzione del programma corrente

La CPU:

- **Sospende** l'istruzione che stava eseguendo.
- **Salva lo stato** attuale (es. Program Counter, registri...) per poter riprendere dopo.

 Questo salvataggio va nello **stack**, come un "segnalibro".

◆ 3. Consultazione del vettore degli interrupt

La CPU:

- Usa il **numero dell'interrupt** ricevuto per cercare nel **vettore degli interrupt**.
- Trova lì l'indirizzo della **ISR** (Interrupt Service Routine) corrispondente.

📌 Esempio:

Interrupt #3 → vai all'indirizzo 0x0040, dove c'è il codice per gestire la tastiera.

◆ 4. Esecuzione della ISR

La CPU "salta" a quell'indirizzo ed esegue la **ISR**, cioè:

- Un piccolo **programma specifico** che gestisce **quel tipo di evento**.
- Es. legge il tasto premuto e lo scrive in memoria.

🕒 La ISR **deve essere veloce**, per non bloccare il sistema.

◆ 5. Aggiornamento delle strutture del sistema operativo

- Viene aggiornata la **device-status table** (stato del dispositivo → "pronto").
- Se un processo era in attesa, viene **sbloccato**.

📄 È qui che il sistema operativo "riprende il controllo".

◆ 6. Ripristino del programma interrotto

Una volta terminata la ISR, la CPU:

- **Recupera lo stato salvato** nello stack (es. il Program Counter).
 - **Riprende l'esecuzione** esattamente da dove era stata interrotta.
-

🧠 Riepilogo in 6 fasi:

1. Interrupt generato da un evento.
 2. CPU sospende il programma corrente e salva lo stato.
 3. Cerca la ISR nel vettore degli interrupt.
 4. Esegue la ISR.
 5. Aggiorna lo stato del dispositivo e del processo.
 6. Riprende il programma sospeso.
-

Gli input e output possono essere di due tipi

1. Sincroni: la gestione viene iniziata e al suo completamento il controllo viene restituito al processo utente, in questo modo la cpu rimane in wait fino all'interrupt di ritorno, questa metodologia consente solo una richiesta alla volta di essere in wait
2. Asincroni: la gestione viene iniziata e il controllo viene restituito al processo utente senza attendere il completamento dell'operazione

◆ **Chiamata di sistema (System call)**

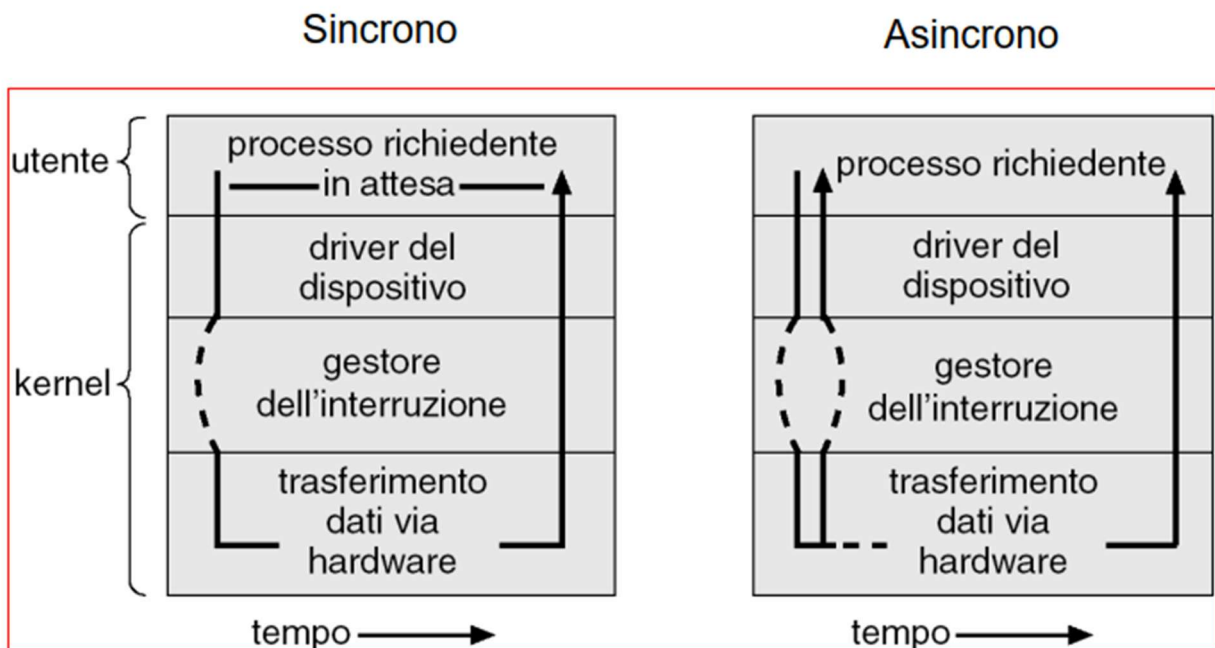
- È una richiesta fatta da un **programma utente al sistema operativo**.
- Serve, in questo caso, per dire al sistema operativo:
"Sto aspettando che l'I/O finisca, tienimi aggiornato."
- Viene usata per coordinare l'**attesa** del processo (soprattutto in I/O sincrónico).

◆ **Device-status table**

- È una **tabella tenuta dal sistema operativo**.
- Contiene **una voce per ogni dispositivo di I/O** collegato al computer.
- Per ogni dispositivo memorizza:
 - **Tipo** (che dispositivo è: disco, tastiera, stampante...)
 - **Indirizzo** (dove si trova nella memoria o nella rete)
 - **Stato** (è attivo? occupato? ha finito? ha un errore?)

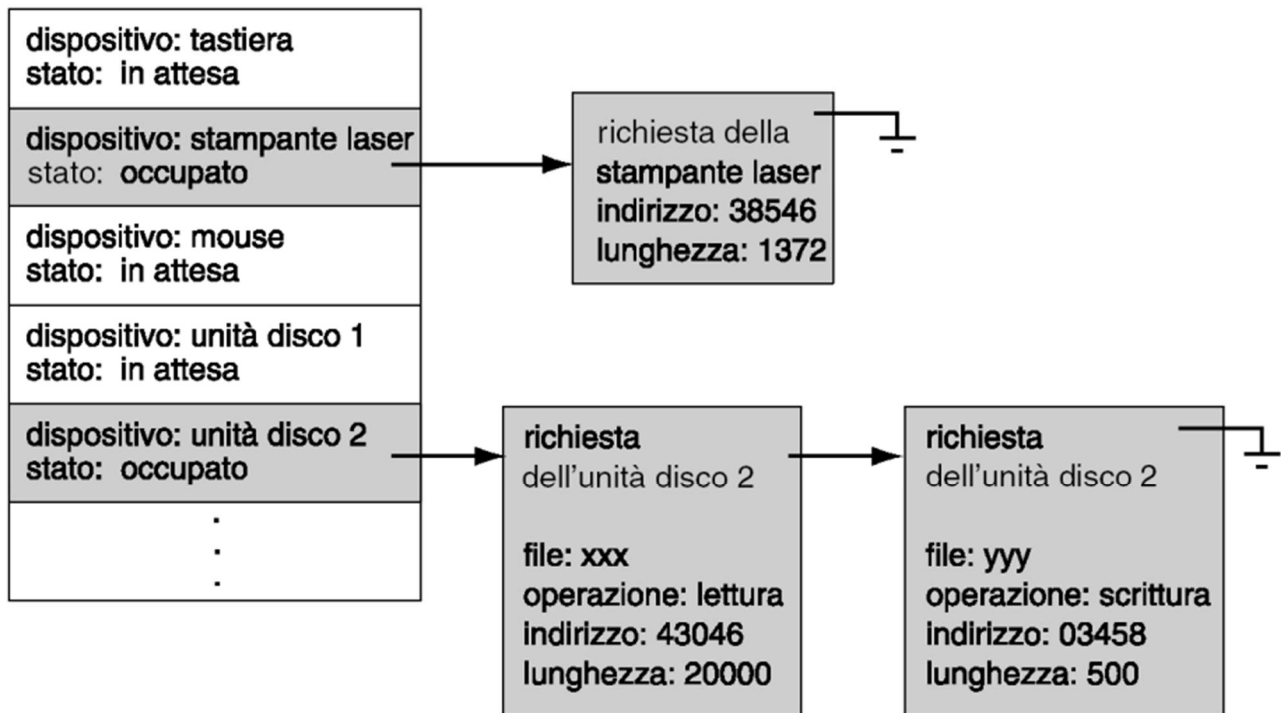
◆ **Gestione tramite tabella e interrupt**

- Quando un dispositivo **completa un'operazione**, invia un **interrupt** al sistema operativo.
- Il sistema operativo:
 1. **Cerca** il dispositivo nella device-status table.
 2. **Controlla lo stato** attuale del dispositivo.
 3. **Aggiorna** le informazioni nella tabella (es. cambia da "occupato" a "libero").
- In questo modo può **comunicare al processo utente** che l'I/O è completato.



Es. device status table

Device status Table



STRUTTURA DEL DMA

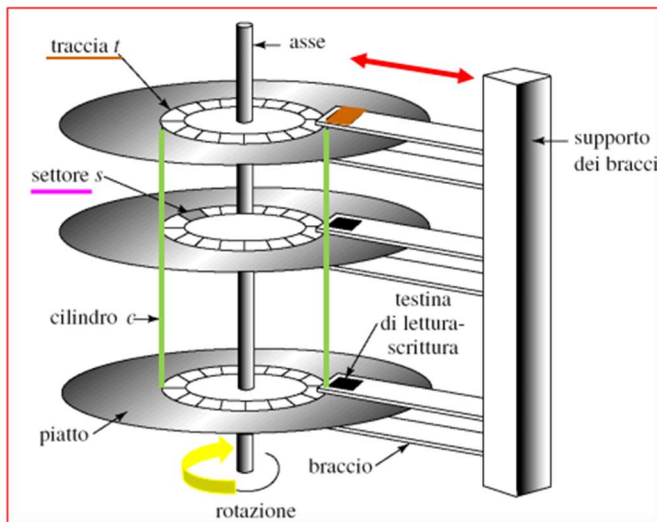
Ossia la direct memory access.

1. Un programma richiede un trasferimento di dati,
2. Il SO identifica il buffer (ossia la memoria temporanea creata al controller i/o),
3. il modulo del DEVICE driver del SO inizializza i registri del controller del DMA identificando la periferica di sorgente o destinazione, con annessi indirizzi di memoria,
4. il controller del dispositivo trasferisce un intero blocco di dati direttamente dal proprio buffer alla memoria o viceversa, senza usare la cpu e alla fine di ogni blocco avviene un interrupt

STRUTTURA DELLA MEMORIA

- Memoria Centrale- unica memoria per la cpu che vi può accederci tramite bus,
 - serve per gestire l'indirizzamento alla word in modo che la più piccola unità di memoria indirizzata sia della dimensione di una word (word = dimensione massima che può calcolare la cpu per volta es. 32 bit, 64 bit)
 - vi è lo spostamento di dati da/verso la memoria centrale ai/verso i registri della cpu
- I/O mappato in memoria (blocchi di indirizzi di memoria riservati sono mappati in registri dei controller dei dispositivi di I/O) - tempi di risposta rapidi

- I/O programmato (la CPU interroga un bit di controllo relativo ad una data PORTA di I/O per scegliere l'istante di trasferimento dei dati)
- I/O interrupt driven (la CPU riceve un interrupt quando il device è pronto)
- Memoria Secondaria- estensione della memoria centrale che è in grado di mantenere grandi quantità di dati in modo permanente es. hard disk magnetici = piatti con profilo piano circolare in metallo con diametro da 1.8" a 5,25", ogni superficie è divisa in tracce che sono suddivise in settori, il controller dei dischi determina l'interazione logica tra il dispositivo e il pc dischi determina l'interazione logica tra il dispositivo e il pc



Funzionamento del disco:

- 1) Moto traslazionale
 - 2) Moto rotazionale
- } Localizzazione singola
(c, t, s)

c → cilindro - luogo geometrico dei punti del disco equidistanti dall'asse (**SPLINDE**)

t → individua la traccia sul piatto (la testina da accendere)

s → settore - luogo geometrico dei punti che identifica una distanza angolare specifica rispetto allo zero (identificato dalla velocità di rotazione)

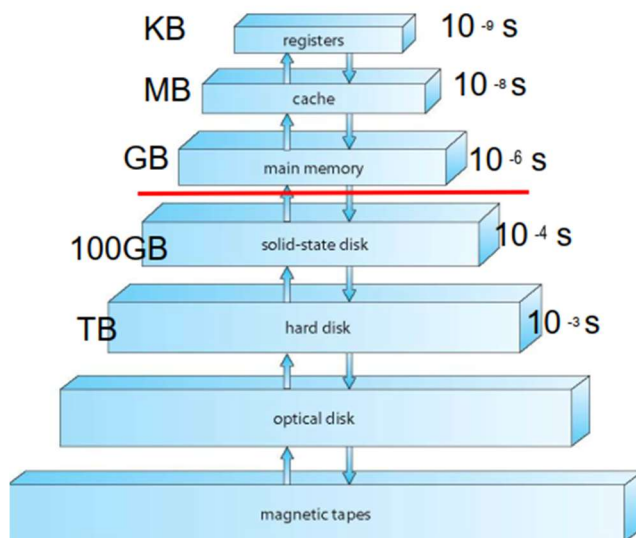
↑ Efficienza ↓ Attrito tra testina e piatto

↑ Velocità accesso ai dati ↑ Velocità di rotazione

Velocità rotazionali da 5400 a 7200 rpm.

Il tempo di accesso (prestazione) dipende da:

- Transfer rate.
- Tempo di posizionamento (accesso al Settore):
 - Search time(tempo per raggiungere il cilindro desiderato-seek)
 - Latenza rotazionale (tempo per raggiungere il settore desiderato)
- Un drawback molto comune è l'head crash - APS (Active Protection System)
- EIDE (Enhanced Integrated Drive Electronics), ATA (Advanced Technology Attachment), FC (Fiber Channel) e SCSI (Small Computers System Interface) sono tecnologie di bus standard per l'interfacciamento verso dischi magnetici.
- Tipicamente i disk controller usano meccanismi memory mapped dell'I/O

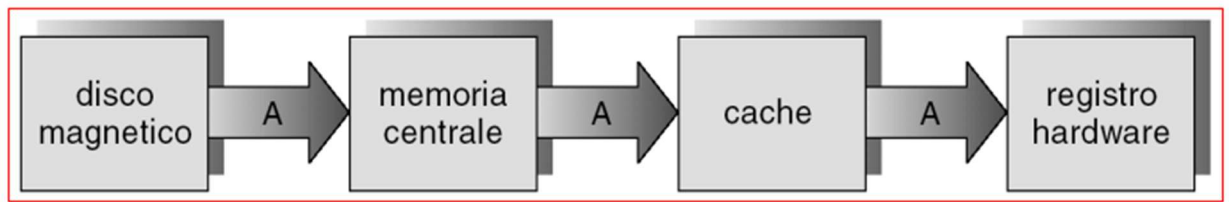


- I sistemi di memorizzazione sono organizzati a livello gerarchico in base a:
 - Velocità/tempo di accesso.
 - Costo/dimensioni.
 - Volatilità.-
- *Caching* – copia temporanea dell'informazione in un sistema di memorizzazione più veloce.
- La memoria centrale può essere vista come *cache* veloce per dispositivi di memorizzazione secondari.

Implementiamo l'uso della cache che serve per memorizzare ad alta velocità i dati ad accesso recente, ovviamente necessita di una gestione

E dunque per il passaggio di informazioni dal disco magnetico ai registri bisogna passare attraverso tutte le gerarchie, e dunque i dati devono rimanere coerenti e consistenti per tutti i passaggi di gerarchia

Migrazione di un dato A dall'HD ai registri HW



PROTEZIONI

Hardware, che servono per evitare errori prodotti dalla multiutenza e programmazione, in assenza di **protezione hw** un sistema dovrebbe eseguire solo un processo per volta altrimenti i successivi output potrebbero essere sbagliati.

Usiamo **un supporto hardware** che permetti di distinguere vari modi di esecuzione:

user mode

monitor mode- eseguita dal SO

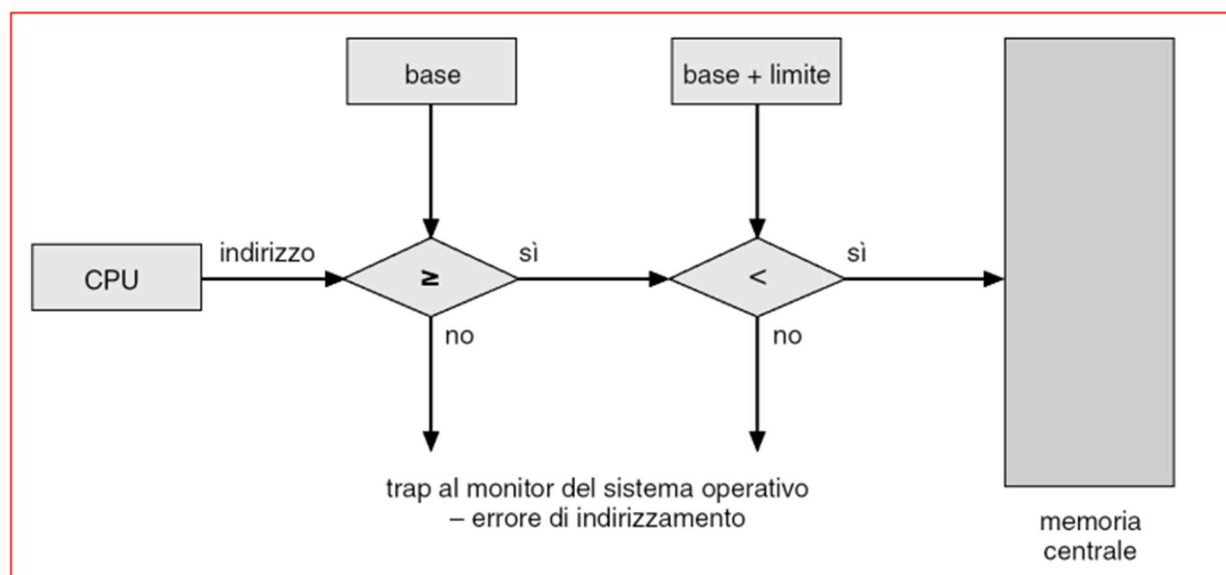
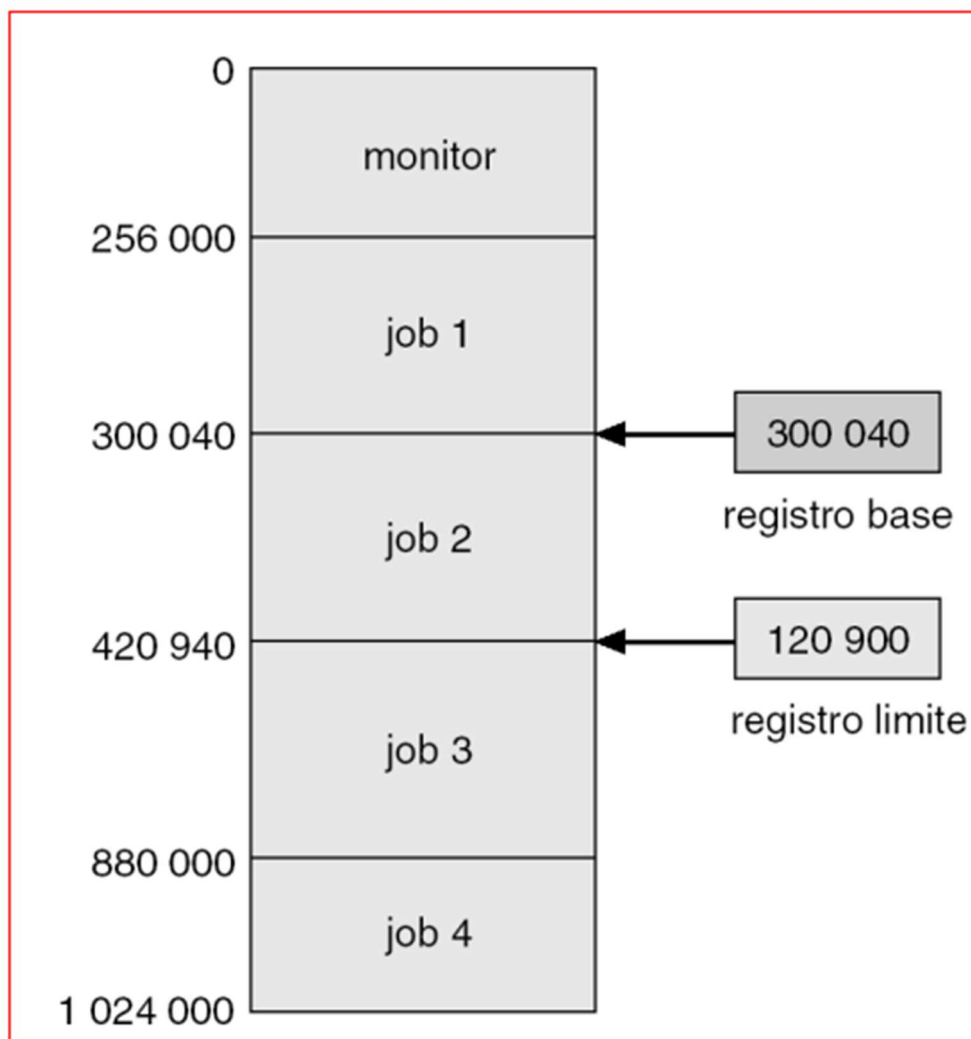
questo switch tra i due modi avviene tramite un bit 0 per monitor e 1 per user

quando vi è una trap(interrupt da programma) o interrupt commuta in monitor mode

per protezioni di tipo i/o facciamo in modo che gli utenti possano fare azioni di tipo i/o solo in system call,

per **protezione della memoria** è necessario assicurare almeno il vettore di interrupt (che serve per la comunicazione tra i controller e la cpu) e la gestione degli interrupt, per fare ciò nei registri scegliamo due range di indirizzi a cui un programma può accedere senza problemi e sono

1. Registro base -> indirizzo più piccolo
2. Registro limite -> indirizzo più grande



PROTEZIONE DELLA CPU

Si utilizza un timer per interrompere l'elaborazione della cpu per non bloccarla in un ciclo infinito, interrompendo l'elaborazione affinché il SO mantenga il controllo

un contatore viene decrementato ad ogni ciclo di clock, quando arriva a 0 genera un interrupt che manda un segnale al SO che può trattare attraverso la possibilità di generare un fatal error o estendendo un time slice riservato ad un processo utente (time slice = tempo dato ad un job dalla cpu per poter essere eseguito)

capitolo 3

STRUTTURA DEI SO

Componenti dei sistemi operativi

1. Gestione dei processi

con processo si definisce un programma in esecuzione, per compiere le proprie attività un processo ha bisogno di specifiche risorse tra cui: tempo della cpu, memoria, file e dispositivi di i/o

Ricordando che si definisce programma un'entità passiva e processo come entità attiva

In relazione alla gestione un SO deve

- .1. Generare e cancellare processi
- .2. Sospendere e riattivare processi
- .3. Gestione di situazioni di deadlock-concorrenza
- .4. Fornire meccanismi per la sincronizzazione e comunicazione dei processi

2. Gestione della memoria centrale

È essenzialmente un vettore di byte a cui accedono in modo velocemente la cpu e i dispositivi di i/o è una memoria di tipo volatile, ossia perde il suo contenuto in caso di arresto del sistema, il compito degli so è quello di tenere traccia di quali parti della memoria sono correttamente in uso e da chi, decidere quali processi devono essere caricati, allocare e deallocare lo spazio in memoria

3. Gestione dei file

Un file è un sistema di informazioni correlate definite dal loro creatore, normalmente i file rappresentano programmi e dati. L'**unità di memorizzazione logica** è il **file**, cioè un'entità gestita dal sistema operativo per salvare e organizzare i dati sulla memoria. Il file rappresenta una **vista logica** dei dati, indipendente da dove sono scritti fisicamente sul disco. Il **file system**, parte del sistema operativo, si occupa di creare, leggere, scrivere e cancellare i file, assegnando lo spazio necessario, gestendo i nomi, i permessi e i metadati. In questo modo, l'utente può accedere ai dati in modo semplice e ordinato, senza preoccuparsi di come sono salvati nel dispositivo fisico.

4. Gestione dell'i/o

Gestisce una porzione della memoria che include i vari buffer e la cache, una componente per il processo di stampa, un'interfaccia generale del driver e i driver per ciascun hardware

Uno dei meccanismi più importanti è quella del Sistema di protezione che serve per il controllo dell'accesso alle risorse da parte dei processi o dagli utenti, questo sistema deve:

1. distinguere tra uso autorizzato e non da parte dell'utente

2. specificare i controlli che devono essere effettuati
3. fornire un metodo per imporre criteri stabiliti

5. gestione della memoria di massa

Poiché la memoria centrale è volatile e poco capiente (la RAM) per contenere tutti i dati e i programmi in modo permanente, il calcolatore deve essere munito di un'unità di memorizzazione secondaria in cui salvare i dati della memoria centrale, in maggioranza è previsto l'uso di dischi come dispositivi di memorizzazione, il compito dei SO è quello di gestione lo spazio, allocare e occuparsi della schedulazione dei dischi

6. interprete dei comandi

La maggior parte degli ordini sono dati al SO attraverso istruzioni di controllo, le quali permettono di creare e gestire processi, gestire le chiamate di i/o, gestire la memorizzazione centrale e secondaria, accedere al file-system, accedere alle protezioni e alla rete

Il programma che riceve ed interpreta le istruzioni di controllo è definito come shell o interprete delle istruzioni di controllo (LOL), la sua funzione è quella di ricevere ed eseguire una sequenza di comandi, questa è la parte più esterna del SO attraverso la quale gli utenti richiedono l'esecuzione di programmi o a servizi del kernel

L'interfaccia può essere di due tipi:

- 1- a caratteri: shell come quello di linux, in cui si mandano stringhe di comandi in modo batch
- 2- GUI : a interfaccia grafica, tramite pulsanti, icone, vi è un desktop in cui risiedono icone, cartelle e i file

Una shell si basa su tre tipi di comandi

1. oggetti eseguibili es- in linux p.es
2. comandi built-in: sono comandi che la realizzano funzioni semplici eseguite direttamente sulla shell
3. script o batch: sono programma in linguaggio shell

Se si tratta di un comando built-in la shell lo esegue, altrimenti crea una fork ossia un nuovo processo che esegue il comando, oltre a questi programmi la shell esegue anche: ridirezione di i/o, pipeline di comandi, filtering

Per la *ridirezione dell'input o dell'output* ogni processo di unix ha tre canali standard:

1. input: associato alla tastiera, possono essere ridirezionati da comando <- file
2. output: associato allo schermo, possono essere ridirezionati da comando -> file
3. error: associato allo schermo

Per la *pipeline di comandi* è possibile combinare i programmi tra loro collegando lo standard output di uno di essi con lo standard input di un altro:

comando1 | comando2

per il *filtering* sono una particolare classe di comandi che possiedono i seguenti requisiti:

- leggono l'input dallo standard input,
- effettuano delle operazioni sull'input ricevuto,
- inviano il risultato delle operazioni allo standard output.

Sono molto utili per eseguire pipeline molto complesse come quelle di sorting, finding, grep, cat

Servizi del sistema operativo

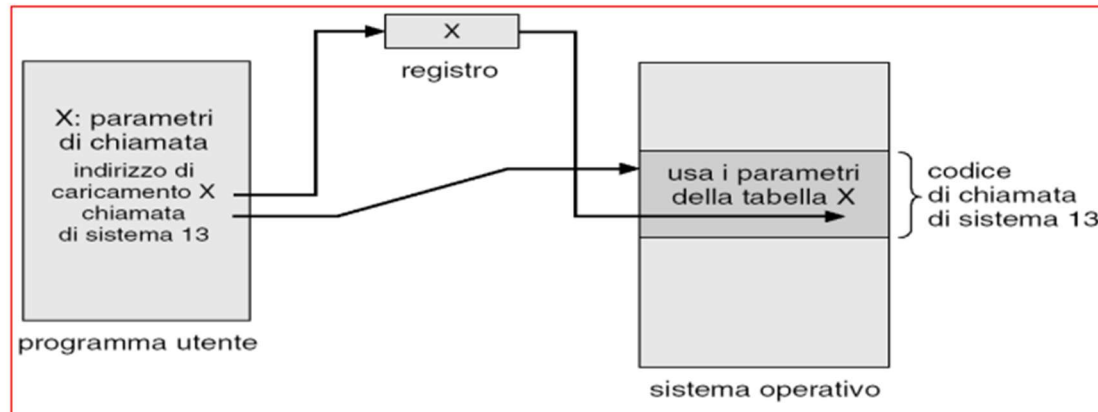
- Esecuzione di un programma – il sistema deve potere caricare un programma in memoria e far funzionare tale programma.
- Operazioni di I/O – in generale gli utenti non possono controllare direttamente i dispositivi di I/O; è il sistema operativo che deve fornire i mezzi per compiere le operazioni di I/O.
- Manipolazione del file system – i programmi devono poter leggere, scrivere, creare e cancellare i file.
- Comunicazioni – la comunicazione può avvenire fra processi in esecuzione sullo stesso calcolatore o in esecuzione su calcolatori differenti collegati fra loro in rete. Le comunicazioni possono essere implementate attraverso una memoria condivisa o attraverso l'invio di messaggi.
- Rilevamento degli errori – il sistema deve assicurare un metodo efficiente per individuare errori a carico di CPU, hardware della memoria, dispositivi di I/O e programmi utente.

Esiste un altro gruppo di funzioni del sistema operativo che non servono a coadiuvare l'utente quanto piuttosto ad assicurare una gestione efficiente delle risorse della macchina.

- Allocazione delle risorse – quando ci sono più utenti o più processi contemporaneamente in funzione le risorse devono essere assegnate a ciascuno di loro in maniera opportuna.
- Contabilità – mantiene traccia di quali utenti usano quali risorse (specificazione di tipo e genere) per stilare statistiche d'uso e mantenerne aggiornata la contabilità.
- Protezione e sicurezza – implica la garanzia che tutti gli accessi alle risorse del sistema siano controllati.

Vi sono **le chiamate di sistema** che forniscono l'interfaccia tra un processo in esecuzione il sistema operativo, i parametri passati tra questi due, sono scambiati tramite: registri, messi all'interno di una tabella in memoria e l'indirizzo della tabella viene passato come parametro in un registro, viene creata una pila temporanea dal programma e prelevati automaticamente dal SO

Passaggio dei parametri come blocco



Tipologie di chiamate di sistema

- Controllo del processo.
- Gestione dei file.
- Gestione del dispositivo.
- Gestione delle informazioni.
- Comunicazioni.

Programmi di sistema

- I programmi di sistema forniscono un ambiente opportuno per lo sviluppo e l'esecuzione dei programmi. Possono essere distinti in:
 - Gestione dei file (gestione di file e directory).
 - Informazioni di stato (dati relativi all'uso della macchina formattate ed inviate in output).
 - Modifica dei file (manipolazione di file di testo).
 - Supporto ai linguaggi di programmazione (compilatori, assembleri, interpreti per i linguaggi di programmazione più comuni).
 - Caricamento ed esecuzione dei programmi (linker, loader, debugger per linguaggi di alto livello e/o linguaggio macchina).
 - Comunicazioni (process networking, mail editor, messenger, remote desktop, file transfer).
 - Programmi applicativi (web browser, utility di sistema).
- La visione del sistema operativo offerta alla maggior parte degli utenti è definita dai programmi di sistema piuttosto che dalle effettive chiamate di sistema.

ARCHITETTURA DI UN S.O

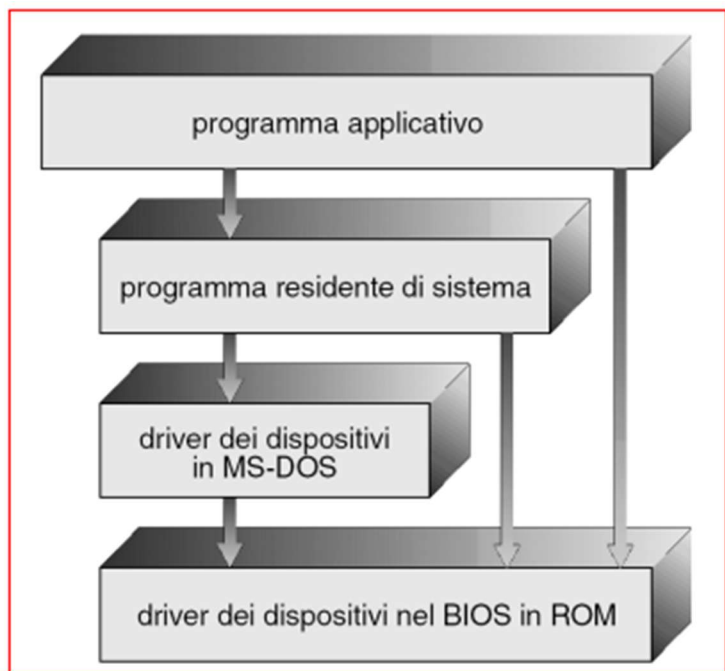
Possiamo fare una suddivisione delle architetture dei S.O:

- **Semplici** = minima organizzazione gerarchica MS DOS, UNIX. Massima funzionalità, minimo spazio
- **Monolitiche** = quando esse sono composte da un unico modulo che serve le richieste dei programmi- utente singolarmente
- **Microkernel** = se hanno un nucleo minimo di funzioni comuni e moduli addizionali per fornire servizi
- **A livelli** = se sono articolate in diversi moduli, ciascuno dei quali svolge funzioni specifiche, ogni modulo può servire le richieste di più programmi-utente
- **Macchina virtuale** = se offrono a ciascun programma-utente visibilità di un particolare hardware

MS-DOS

Massima funzionalità nel minor spazio

Non è diviso in moduli, le sue interfacce e i livelli di funzionalità non sono ben separati, ed è vulnerabile a programmi errati o pericolosi

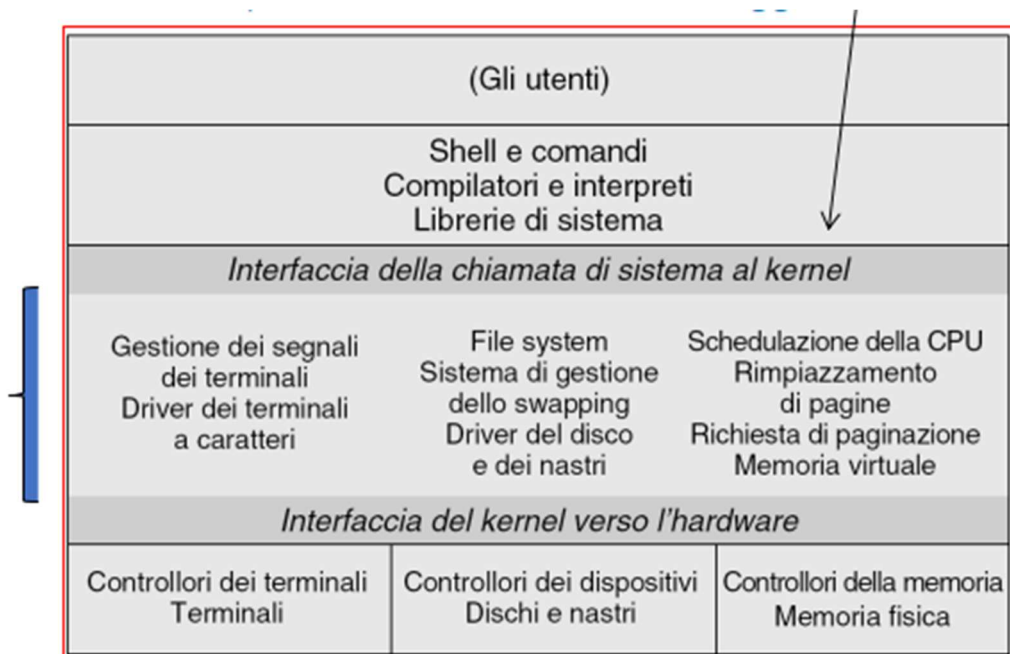


STRUTTURA DI UNIX

Limitato dall'hardware, originariamente nasce come monolitica

Ed era formato solo da un kernel e dai programmi di sistema

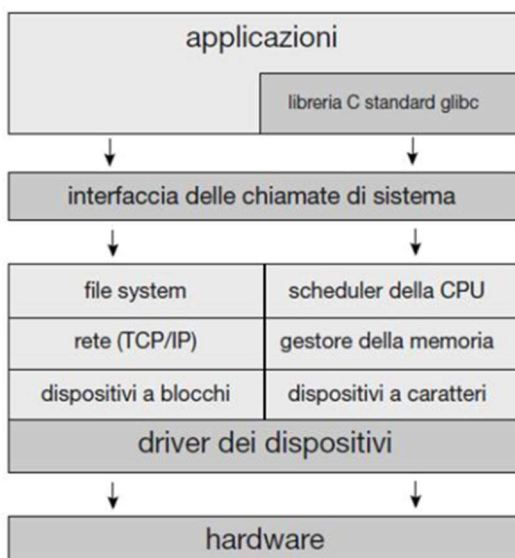
Il kernel comprende qualsiasi parte tra chiamate di sistema e hardware fisico



Le chiamate di sistema definiscono le interfacce per i programmi applicativi

STRUTTURA LINUX

Il kernel di linux è monolitico, in quanto tutte le funzioni vengono eseguite in modalità kernel in un unico spazio di indirizzi. Però è dotato anche di struttura modulare in cui i moduli possono essere caricati un runtime



STRUTTURA STRATIFICATA

Il sistema operativo è diviso in un certo numero di strati ognuno costruito sulla sommità dello strato inferiore, si parte dallo strato più interno ovvero l'hardware, e quello al livello n ovvero l'interfaccia utente, ogni strato non è fisico, ma è un insieme di strutture dati e funzionalità atte ad intervenire su di esse, comunicanti tra loro, attraverso il principio di modularità che sostiene che ogni strato comunica con lo strato strettamente inferiore, in questo modo si agevola l'individuazione degli errori, tuttavia necessita di una attenta progettazione, gli strati superiori conoscono solo le funzioni degli strati inferiori. Uno svantaggio è il sovraccarico alle system call, in quanto per arrivare dall i/o faccio 3 o 4 chiamate prima di arrivare all hw.

Si preferisce usare pochi strati con più funzionalità

STRUTTURA MICROKERNEL

Sposta la maggior parte delle funzionalità del kernel allo spazio utente in cui vivono i programmi di sistema e utente. In questo modo i kernel diventano più piccoli ma con meno funzionalità e solo limitati a fare:

- Gestione dei processi
- Gestione della memoria
- Funzioni di comunicazione: tramite scambio di messaggi

I vantaggi sono :

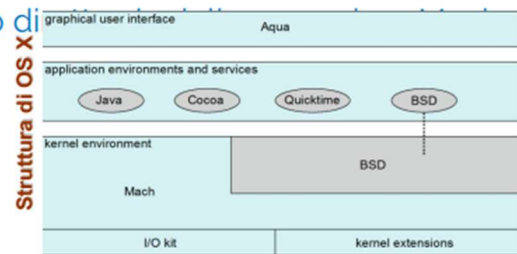
- Facilità di estendere il sistema operativo, aggiungendo spazio utente
- Migliore portabilità su macchine vecchie
- Più affidabile
- Maggiore sicurezza: pochi comandi in kernel mode

Difetti:

- Calo di prestazioni dovuto ad un sovraccarico di funzioni di sistema

Struttura ibrida

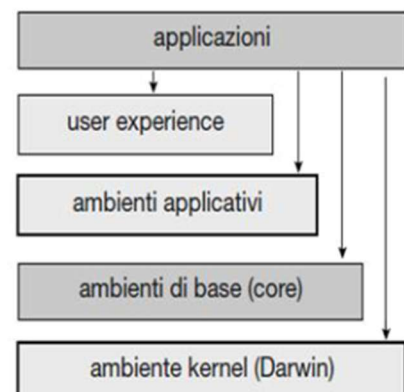
- Combinano approcci diversi allo scopo di migliorare performance, sicurezza e affidabilità
- Apple Macintosh OSX ne è un esempio. La struttura prevede la presenza di:
 - Kernel environment (M. Mach + BSD + supporto driver I/O + estensioni del kernel)
 - Microkernel Mach - gestione memoria, supporto RPC (Remote Procedure Call) e IPC (Inter Process Communication), schedulazione dei thread
 - BSD (Berkley Software Distribution) - interfaccia a riga di comando, supporto per la gestione dei file e la rete, implementazione delle API POSIX (librerie multithreading)
 - Application environment e System services
 - Applicazioni e servizi comuni possono far uso di quelle di quelle BSD



macOS e iOS

Il sistema operativo macOS di Apple è progettato per funzionare principalmente su computer desktop e laptop, mentre iOS è un sistema operativo mobile progettato per iPhone e iPad.

- Strato dell'interfaccia utente Aqua /Springboard.
- Strato degli ambienti applicativi Cocoa/Cocoa Touch (API per Objective-C e Swift)
- Ambienti di base (core) definisce gli ambienti di supporto alla grafica e ai contenuti multimediali
- Ambiente kernel (noto anche come Darwin) Mach e BSD



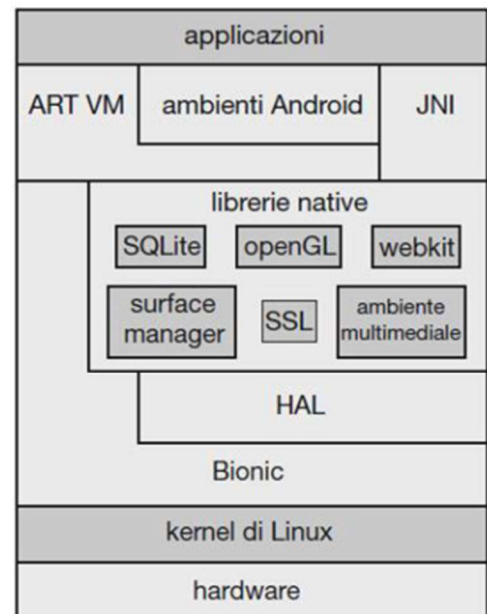
Struttura Android

Sviluppato dalla *Open Handset Alliance* – guidata da **Google**
gira su una grande varietà di piattaforme mobile ed è open-source

Costituito da una "pila" di strati software (come iOS)

Basato su *un kernel Linux modificato* (al di fuori delle distribuzioni standard)

- Gestione dei processi, della memoria, delle periferiche
- Ampliato per includere l'ottimizzazione dei consumi energetici

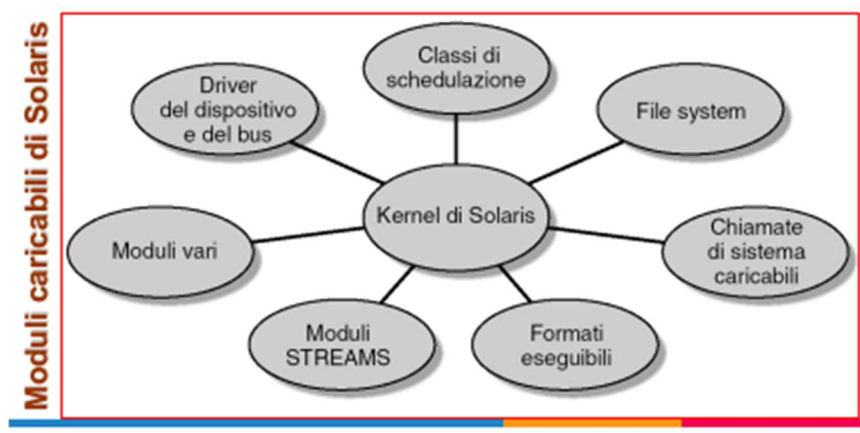


Struttura Android

- L'ambiente di run-time include un insieme di librerie di base e la macchina virtuale ART (Android Run Time)
 - App sviluppate in Java con il supporto dell'Android API
 - Class file di Java compilati in bytecode e quindi tradotti in eseguibili per la ART virtual machine
 - Compilazione anticipata, per migliorare l'efficienza delle applicazioni
- Disponibile anche l'interfaccia Java nativa, JNI, per bypassare ART, e scrivere programmi Java con accesso più diretto all'hardware
- Le librerie includono ambienti per lo sviluppo di *browser* (webkit), di supporto ai *database* (SQLite) e all'*accesso alla rete* (SSL), *ambienti multimediali* e una *libreria C standard minimale* (Bionic, libera da GPL)

STRUTTURA MODULARE

Usa orientend objective program per creare kernel modulari, il kernel possiede una serie di componenti di base che si collegano dinamicamente ai moduli addizionali in fase di boot o in fasi particolari in run-time



MACCHINE VIRTUALI

La macchina fisica ripartisce le sue risorse alla o alle macchine virtuali.

Il processore attraverso la schedulazione crea dei processori dedicati per ciascuna vm, utilizzando una gestione asincrona delle operazioni di i/o e dei processi

È necessario molto lavoro per creare un esatto duplicato della macchina fisica

Inoltre, la macchina fisica ha sia la modalità utente che quella di controllo ma, nonostante il so caricato sulla vm supporti la modalità di controllo, può solo usare quella utente

La **modalità utente** e la **modalità di controllo** (spesso detta anche **modalità kernel** o **supervisore**) in un **sistema operativo (SO)** riguardano i **livelli di accesso alle risorse del computer** e **quali operazioni sono permesse a un programma**.

Sono fondamentali per garantire **sicurezza**, **stabilità** e **isolamento** tra i programmi e l'hardware.



Le due modalità principali:



1. Modalità utente (User Mode)

- È la modalità in cui **girano le applicazioni normali** (es. browser, editor di testo, giochi).
- Ha **accesso limitato** alle risorse hardware.
- **Non può accedere direttamente** a memoria protetta, dispositivi o istruzioni privilegiate.
- Se un'applicazione ha bisogno di fare operazioni speciali (es. leggere da disco), deve **chiedere al SO** tramite una **system call**.



Serve a evitare che un'applicazione possa danneggiare il sistema.

2. 🛠️ Modalità di controllo (Kernel Mode / Supervisor Mode)

- È la modalità in cui gira il **nucleo (kernel)** del sistema operativo.
- Ha **pieno accesso** all'hardware e a tutte le istruzioni della CPU.
- Può:
 - Gestire la memoria
 - Eseguire processi
 - Controllare dispositivi I/O
 - Gestire file system, permessi, ecc.

⚠️ È una modalità **potente ma delicata**: un errore qui può **bloccare l'intero sistema**.

🔄 Come funzionano insieme?

- Quando un'app in modalità utente **richiede un servizio del sistema operativo** (es. scrivere su disco), avviene:
 1. Una **system call**.
 2. Il sistema passa dalla modalità utente alla **modalità kernel**.
 3. Il SO esegue l'operazione.
 4. Poi torna in modalità utente.
-

📌 In sintesi:

Modalità	Chi la usa	Accesso all'hardware	Sicurezza
Utente	Programmi dell'utente	Limitato	Alta
Controllo/Kernel	Sistema operativo	Totale	Bassa (serve cautela)

Questa distinzione è **essenziale nei moderni sistemi operativi** (Windows, Linux, macOS, ecc.) per evitare che un programma **malfunzionante o malevolo** danneggi il sistema.

Se vuoi ti posso spiegare anche come questa distinzione viene implementata a livello di **CPU (privileged instructions)** o con un esempio pratico!

- Deve essere implementata una virtualizzazione della modalità utente e una della modalità controllo.
- Entrambe le modalità dovranno funzionare in modalità utente sulla macchina fisica.
- Le azioni che causano un trasferimento di modalità sulla macchina fisica (reale) devono ripercuotersi in un trasferimento dalla modalità utente virtuale alla modalità monitor virtuale sulla VM.

Certo! Ti faccio una spiegazione **completa e integrata** combinando quanto c'è nella **tua slide** con la spiegazione precedente, in modo **più chiaro e coerente**.

Passaggio di modalità in una VM (Macchina Virtuale)

Quando una **VM** (Virtual Machine) deve eseguire un'**istruzione privilegiata** (tipicamente in seguito a una system call), non può farlo direttamente come avverrebbe in un sistema reale. Entra in gioco il **monitor virtuale** (cioè l'hypervisor o VMM – Virtual Machine Monitor).

Ecco come avviene il processo:

1. Modalità utente virtuale → modalità monitor virtuale

- All'interno della VM, un'applicazione in **modalità utente virtuale** fa una system call.
- Questo provoca un **passaggio di modalità all'interno della VM**, come se si stesse passando da utente a kernel.

♦ Tuttavia, la VM **non può realmente eseguire istruzioni privilegiate**, perché gira in modalità **utente sulla macchina fisica**.

2. Interviene il monitor virtuale (VMM)

- L'hypervisor intercetta questa operazione.
 - Cambia lo stato del **Program Counter (PC)** e del **Instruction Register (IR)** della VM, per simulare il comportamento previsto dal sistema operativo guest.
-

3. Esecuzione dell'istruzione privilegiata

- L'**istruzione privilegiata viene eseguita indirettamente dal monitor virtuale**, che può:
 - emulare il comportamento previsto,
 - oppure usare dispositivi e meccanismi **virtualizzati**.
-

4. La VM "crede" di essere in modalità kernel

- Anche se gira ancora in modalità utente sulla macchina fisica, la VM viene **modificata nello stato** come se avesse davvero eseguito l'istruzione in modalità kernel.
 - Questo mantiene la **trasparenza**: il guest OS non si accorge della virtualizzazione.
-

5. Ritorno alla VM e possibili differenze di tempo

- Completata l'operazione, il controllo torna alla VM.

- L'operazione può essere **più lenta rispetto a un'esecuzione su macchina reale**, a causa del passaggio di controllo al monitor virtuale.
-

🔴 Riepilogo Visuale (semplificato):

[App in VM]

↓ (System Call)

[Guest OS - vuole passare a kernel mode]

↓ (ma è in user mode fisica)

[Intercettato da Hypervisor]

↓

[Hypervisor simula/virtualizza istruzione privilegiata]

↓

[Stato della VM aggiornato]

↓

[Controllo restituito alla VM]

🧠 Program Counter (PC)

🔴 Definizione:

Il **Program Counter** è un **registro della CPU** che **contiene l'indirizzo della prossima istruzione da eseguire** in memoria.

🔄 Dopo che un'istruzione viene eseguita, il PC viene **aggiornato** per puntare all'istruzione successiva.

✅ In breve:

- Dice **“dove siamo arrivati”** nell'esecuzione del programma.
 - Guida il flusso sequenziale delle istruzioni.
 - Viene modificato da **salti, chiamate di funzione, ecc.**
-

⚙️ Instruction Register (IR)

🔴 Definizione:

L'**Instruction Register** è il registro della CPU che **contiene l'istruzione attualmente in fase di esecuzione**.

🔄 Dopo che l'istruzione è caricata dalla memoria (tramite il PC), viene salvata nell'IR per essere **decodificata ed eseguita**.

✅ In breve:

- Contiene “**cosa stiamo facendo ora**”.
 - Viene aggiornato a ogni ciclo di fetch.
 - Serve alla CPU per **sapere quale operazione compiere**.
-

✓ Progettazione di un Sistema Operativo (O.S.)

◆ Influenze iniziali

- La progettazione di un O.S. a livello alto è influenzata da:
 - Tipologia di hardware
 - Tipo di sistema:
 - **Batch, Time-sharing,**
 - **Mono/multi-user,**
 - **Realtime, Distribuito,** ecc.
-

◆ Obiettivi di progettazione

🎯 Obiettivi lato utente

- Comodità d'uso
- Facilità di apprendimento e utilizzo
- Affidabilità
- Sicurezza
- Velocità

⚠ Questi obiettivi **non possono influenzare direttamente** le scelte di sistema perché **non esiste un accordo univoco** su come realizzarli.

🔧 Obiettivi lato sistema

- Facilità di progettazione, implementazione e manutenzione
- Flessibilità
- Affidabilità ed efficienza
- Assenza di errori

🔍 Questi requisiti sono spesso **interpretati in modo soggettivo** e non univoco.

📌 Nota

La progettazione di un O.S. è un'attività **altamente creativa**.

Tuttavia, nell'ambito dell'ingegneria del software esistono **linee guida generali** per affrontarla.

Meccanismi e Politiche

◆ Principio chiave: separazione tra politica e meccanismo

- **Meccanismi:** *come* fare qualcosa
- **Politiche:** *cosa* fare

✓ Vantaggi della separazione:

- Maggiore **flessibilità**
- Le **politiche cambiano**, i **meccanismi restano stabili**

Esempi:

- **Microkernel OS:** implementano un set minimo di **primitive indipendenti**, lasciando politiche e funzioni avanzate a moduli esterni.
- **Windows:** integra meccanismi e politiche per offrire un'interfaccia utente uniforme (GUI parte del kernel).

Implementazione dei Sistemi Operativi

◆ Linguaggi utilizzati:

- **Prima:** linguaggio Assembly
- **Oggi:** linguaggi **ad alto livello** (es. C)

✓ Vantaggi:

- Scrittura più veloce e compatta
- Codice leggibile e facilmente debug-abile
- Portabilità migliorata
- Benefici indiretti grazie ai miglioramenti nei compilatori

 Esempio: **Linux** e **Windows XP** sono scritti **prevalentemente in C**, con Assembly solo per:

- **Context switching**
- **Driver dei dispositivi**

▼ Svantaggi dei linguaggi ad alto livello

- Maggiore consumo di memoria
- Leggero calo di performance

✓ Ma: i compilatori moderni offrono

- Ottimizzazioni efficaci

- Gestione avanzata delle dipendenze dati
 - Supporto per unità funzionali multiple
-



Considerazioni sulle performance

- I veri miglioramenti derivano da **algoritmi migliori**, non solo da codice nativo ottimizzato
- Le parti più critiche:
 - **Gestione della memoria**
 - **CPU Scheduler**
- In caso di **bottleneck**, si può sostituire il codice critico con versioni in Assembly



Valutazione delle performance:

- Tramite:
 - **Log esterni** processati offline
 - **Visualizzazione in tempo reale** (può però alterare i risultati)
-



Generazione del Sistema Operativo

◆ Adattamento alla macchina

- Ogni O.S. deve essere **configurato per la macchina target**
- Il programma **SYSGEN** raccoglie informazioni sull'hardware:
 - CPU e opzioni
 - Memoria
 - Periferiche (porte, IRQ, modello, ecc.)
 - Opzioni dell'O.S. (buffer, algoritmi di scheduling, multiprocessing)

I dati raccolti permettono di selezionare **moduli precompilati** da assemblare nel sistema operativo.



Obiettivo:

- Includere **solo i moduli necessari**, per un O.S. leggero ed efficiente
-



Moduli caricabili anche a runtime

- Alternativa al caricamento statico (compilazione/linking)
 - Comporta **maggiore dimensione**, ma anche **maggiore adattabilità**
-

◆ Processo di Boot

- **Boot:** fase di avvio del computer
 - **Bootstrap program:**
 - Contenuto nella **ROM**
 - Carica il **kernel** in memoria e ne avvia l'esecuzione
 - **Boot loader:**
 - Installato nel **MBR (Master Boot Record)**
 - Carica il bootstrap program, che a sua volta carica il kernel
-

I PROCESSI

Job = processo,

un processo è un programma in esecuzione, questa esecuzione deve progredire in modo sequenziale

il processo comprende diverse parti:

- Una sezione di codice
- Attività corrente in cui vi sono
 - Program counter
 - Registri della cpu
 - Stack (contiene dati temporanei)
 - Sezione dati (variabili globali)
 - Heap (memoria dinamica in fase di esecuzione)

Due o più processi possono essere associati allo stesso programma e vengono considerati come due istanze separate

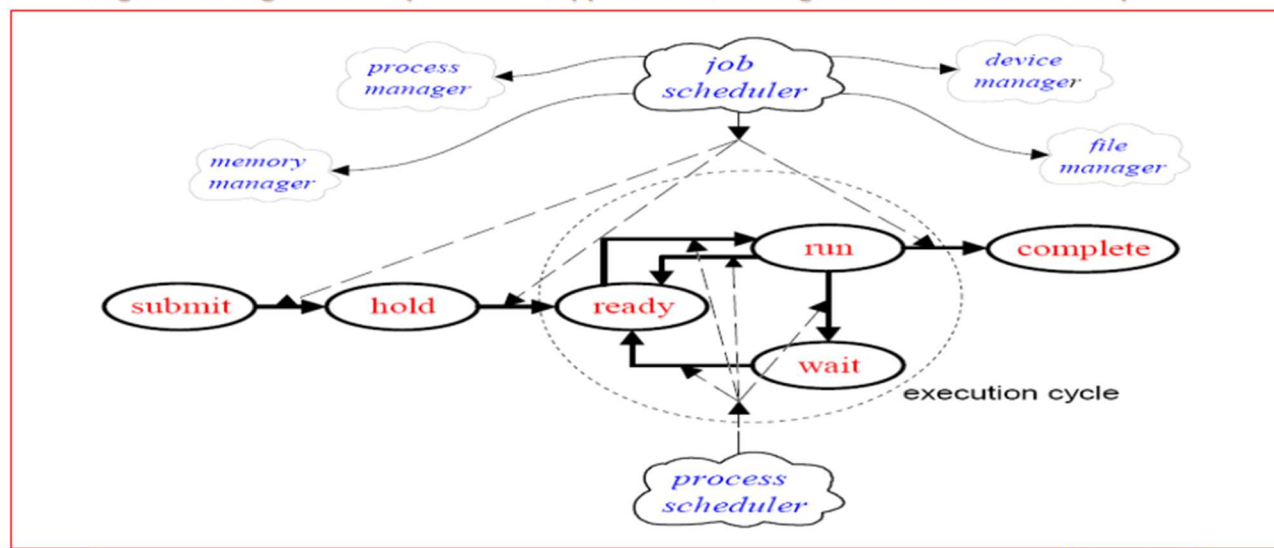
STATO DEI PROCESSI

Un processo può trovarsi in uno dei seguenti stati

- Nuovo: il processo è stato appena creato
- Running: le istruzioni vengono eseguite
- Waiting: aspetta qualche evento come i/o, attesa di un segnale, assegnazioni di dispositivi
- Ready: è in attesa di essere assegnato ad un processore
- Terminato

Le richieste dei job vengono raggruppate in una coda relativa allo stato di submit, e vengono ordinate attraverso il job scheduler, costruendo la coda relativa allo stato di hold

Diagramma degli stati dei processi – rappresentazione logica della transizione dei processi



A partire dalla prima richiesta di processo nella coda di hold, il job scheduler verifica la disponibilità delle risorse necessarie e dopo attiva il process scheduler, la richiesta viene trasformata in processo

Quando un task (o processo) è pronto per essere eseguito, viene inserito nella coda dello stato "ready" (pronto).

Questa coda è gestita dal **Process Scheduler** (lo schedulatore dei processi), il quale decide **quale task eseguire per primo** in base a determinati **criteri di priorità**.

Questa fase, in cui si stabilisce l'ordine di esecuzione tra i task pronti, prende il nome di **microscheduling**. In particolare, questi task sono in attesa solo della **CPU** per poter essere eseguiti: non devono aspettare input/output o altre risorse.

Non appena la CPU si libera, oppure se un processo con priorità più alta lo richiede, il sistema operativo effettua un **context switching** (cambio del contesto computazionale), facendo passare **un processo dalla coda "ready" allo stato di esecuzione ("run")**, mentre eventualmente quello in esecuzione torna in "ready" o in un altro stato.

Se durante l'esecuzione un processo deve attendere un evento esterno (ad esempio l'esito di un'operazione di input/output), il sistema operativo lo sposta nella **coda dello stato "wait"**. Una volta soddisfatta la condizione di attesa, il processo può tornare nello stato "ready".

Infine, dopo aver attraversato i vari stati del ciclo di vita (ready, run, wait), il processo giunge alla **conclusione della sua esecuzione** e viene spostato nella **coda dello stato "complete"**, uscendo così dal ciclo attivo della CPU.

PROCESS CONTROL BLOCK (PCB)

È una struttura del kernel che si occupa di mantenere le informazioni relative ad ogni processo come

1. **Stato del processo**
 - a. Indica in quale fase del ciclo di vita si trova il processo: ad esempio, *ready* (pronto), *running* (in esecuzione), *waiting* (in attesa), *terminated* (concluso),
2. **Program counter**
 - a. Contiene l'indirizzo dell'**istruzione successiva** da eseguire per quel processo. Serve per **riprendere l'esecuzione correttamente** dopo un'interruzione o uno switch.
3. **Registri della cpu**

- a. Includono i valori dei **registri interni** del processore (come accumulatore, registri generali, registri indice, ecc.) che il processo stava usando. Sono salvati e ripristinati durante un **context switch**.

4. Informazioni per la schedulazione della cpu

- a. Comprendono dati come **priorità del processo**, tempo di esecuzione accumulato, e altri parametri utilizzati dallo scheduler per decidere **quando e per quanto tempo** assegnare la CPU al processo.

5. Info per la gestione della memoria centrale

- a. Riguardano i dati necessari per **mappare lo spazio di memoria** del processo, come le **tabelle delle pagine** o i **registri base e limite**, usati per accedere correttamente alla memoria.

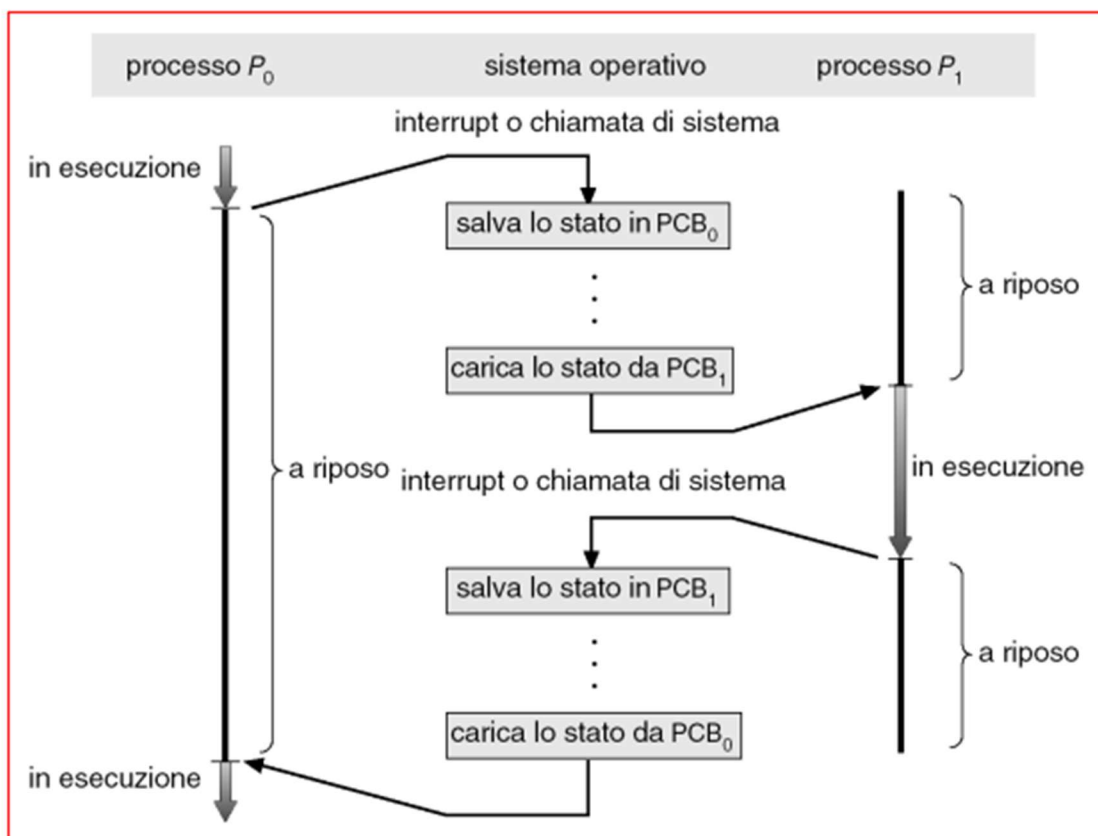
6. Info per l'accounting

- a. Contengono statistiche e dati di monitoraggio come il **tempo totale di CPU usato**, **tempo di esecuzione**, **numero di I/O effettuati**, utili per **analisi delle prestazioni** o **fatturazione** in ambienti multiutente.

7. Info sullo stato dell' i/o

- a. Elenca i dispositivi di I/O associati al processo, le **operazioni in corso**, e lo stato delle **richieste pendenti**. Serve per riprendere correttamente l'I/O o gestire le attese su dispositivi esterni.

CONTEXT SWITCHING



È l'azione della cpu che fa passare da un processo ad un altro, questo richiede il salvataggio dello stato di esecuzione della computazione del vecchio processo ed il caricamento dello stato di esecuzione di quello nuovo, questa azione è effettuata dal Dispatcher, queste operazioni hanno bisogno di tempo, che viene definito come dispatch latency che rappresenta il tempo per il cambio di contesto, più basso è il dispatch latency più è performante la macchina

CODE DI SCHEDULAZIONE

In un sistema operativo multitasking, i **processi non vengono eseguiti tutti contemporaneamente**, ma si **muovono attraverso diverse code** in base al loro stato e alle risorse necessarie. Queste **code di schedulazione** permettono al sistema operativo di **organizzare e gestire l'esecuzione dei processi** in modo efficiente e ordinato.

Ecco le principali **tipologie di code** utilizzate:

✅ Coda di submit (submit queue)

Contiene **tutti i processi presenti nel sistema**, sia quelli in attesa di essere analizzati che quelli che attendono l'assegnazione di risorse. È il **primo punto di ingresso** per ogni processo.

🕒 Coda di hold (hold queue)

Qui sono conservati i processi per cui è già stata effettuata **l'analisi delle risorse richieste**, ma che **non possono ancora essere trasferiti in memoria centrale**, ad esempio per mancanza di spazio o risorse disponibili.

● Coda dei processi pronti (ready queue)

Contiene tutti i processi **caricati in memoria centrale, pronti a essere eseguiti** dalla CPU, ma attualmente **in attesa di ricevere il controllo** della CPU. Lo scheduler sceglie da questa coda quale processo eseguire.

🗂 Code delle periferiche di I/O (device queues)

Ogni periferica di input/output (come disco, stampante, tastiera) ha una propria coda. Qui vengono inseriti i processi **che sono in attesa di completare un'operazione di I/O** su quella periferica.

🔄 Movimenti tra le code

Durante il ciclo di vita, un processo **si muove dinamicamente tra queste code**, a seconda dello stato in cui si trova:

- Entra nella **submit queue** al momento della creazione.
- Passa alla **hold queue** dopo l'analisi delle risorse richieste.
- Quando le risorse sono disponibili, va nella **ready queue**.
- Se richiede un dispositivo I/O, viene spostato nella relativa **device queue**.
- Una volta completata l'operazione I/O o ricevuta la CPU, può tornare nella **ready queue**.
- Al termine dell'esecuzione, il processo esce definitivamente dal sistema.

SCHEDULATORI

Uno schedulatore a lungo termine seleziona quale processo deve essere inserito nella coda dei processi pronti.

A differenza dello scheduler a breve termine (che agisce frequentemente per decidere quale processo assegnare alla CPU, utilizzando lo swapping tra memoria centrale e di massa, ogni 100msec), questo lavora **con una frequenza molto più bassa**, tipicamente **nell'ordine dei minuti**.

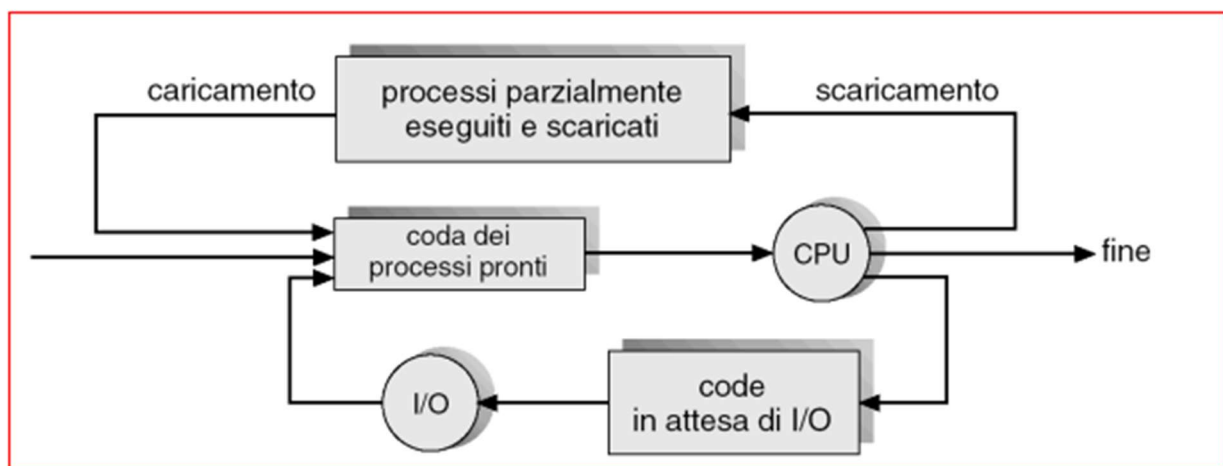
Il suo compito principale è **controllare il livello di multiprogrammazione**, ovvero **il numero di processi presenti contemporaneamente in memoria centrale**.

Deve garantire una certa **stabilità** di questo numero, per evitare sovraccarichi o sprechi di risorse.

Inoltre, lo scheduler a lungo termine si occupa anche del **bilanciamento tra processi I/O bound e CPU bound**:

- I **processi I/O bound** usano la CPU per brevi periodi e passano molto tempo in attesa di dispositivi di input/output.
- I **processi CPU bound** usano la CPU per periodi più lunghi e interagiscono meno con le periferiche.

Un buon bilanciamento tra questi due tipi di processi permette di **massimizzare l'utilizzo della CPU** e **mantenere il sistema efficiente e reattivo**.



CREAZIONE PROCESSI

Creazione ed esecuzione dei processi: relazioni padre-figlio

Molti sistemi operativi moderni offrono la possibilità di **creare e terminare dinamicamente i processi**. Durante la propria esecuzione, un **processo padre** può **generare uno o più processi figli**, i quali a loro volta possono generare altri processi, dando origine a una **struttura ad albero** di discendenza tra processi.

Questa relazione padre-figlio implica alcune **scelte fondamentali** da parte del sistema operativo, che riguardano principalmente due aspetti: la **condivisione delle risorse** e la **gestione dell'esecuzione**.

1) Condivisione delle risorse

Quando un processo padre crea un figlio, è possibile definire **come verranno gestite le risorse** tra di loro. Le opzioni più comuni sono:

- **Condivisione totale:** padre e figli **condividono tutte le risorse** (memoria, file aperti, dispositivi, ecc.).
 - **Condivisione parziale:** i figli **condividono solo un sottoinsieme** delle risorse del padre. In questo caso è il **padre a decidere** cosa condividere.
 - **Nessuna condivisione:** padre e figli **non condividono risorse**. Il sistema operativo **alloca nuove risorse** per ogni processo figlio in modo indipendente.
-

2) Gestione dell'esecuzione

Dopo la creazione del processo figlio, il sistema operativo può gestire l'esecuzione in due modi principali:

- **Esecuzione concorrente:** il processo padre **continua a eseguire in parallelo** ai suoi figli. Entrambi proseguono l'elaborazione indipendentemente.
- **Attesa dei figli:** il processo padre **sospende la propria esecuzione** fino a quando **uno o più figli terminano**. Questo comportamento è utile quando il padre **dipende dai risultati** dei figli.

3) Spazio di indirizzamento

Infine, anche lo **spazio di memoria** (cioè, il contenuto del programma in esecuzione) può essere gestito in due modalità:

- **Duplicazione del padre:** il figlio è **una copia esatta** del padre (stesso codice, dati, stato iniziale).
- **Nuovo programma:** il figlio **carica un programma diverso** nel proprio spazio di indirizzamento, diventando quindi un processo con un comportamento differente.

TERMINAZIONE DEI PROCESSI

La **terminazione di un processo** rappresenta la fase finale del suo ciclo di vita. Può avvenire in modo **normale (volontario)** oppure per **cause esterne (forzata)**. I sistemi operativi gestiscono attentamente questa fase per **liberare risorse** e **mantenere la stabilità del sistema**.

Terminazione volontaria (normale)

Un processo termina **in modo controllato** quando:

- Esegue la **sua ultima istruzione** e invoca la **chiamata di sistema exit()**, notificando al sistema operativo che ha concluso il suo compito.
 - Se è un **processo figlio**, può restituire un **valore di stato** al processo padre.
 - Il **padre riceve questo valore** tramite la chiamata di sistema **wait()**, che gli consente di attendere la terminazione dei figli.
 - Una volta terminato, il **sistema operativo dealloca tutte le risorse** associate al processo (memoria, file, dispositivi, ecc.).
-

Terminazione forzata (anomala)

Un **processo padre può terminare un figlio** in modo forzato per diversi motivi, ad esempio:

- Il figlio ha **superato i limiti** nell'uso di una risorsa (memoria, tempo CPU, file aperti).
- Il **compito del figlio non è più necessario** o è diventato obsoleto.
- Il padre stesso **sta per terminare**, e in alcuni sistemi ciò comporta la fine automatica dei suoi figli.

Comportamento del sistema operativo alla terminazione del padre

Le politiche adottate dai sistemi operativi quando un **processo padre termina** possono variare:

- In **alcuni sistemi**, i figli **non possono continuare ad eseguire** se il padre si è terminato: vengono anch'essi terminati. Questo meccanismo è noto come **terminazione a cascata** (*cascading termination*).
- In **altri sistemi**, i figli **possono continuare ad esistere**, e vengono "**adottati**" da un **processo speciale**, chiamato **init** (tipico nei sistemi Unix/Linux), che ne assume la responsabilità.

I processi possono essere indipendenti e dunque non influenzare o essere influenzato da altri processi

Ma possono anche cooperare, questo fa sì che vi è una condivisione di informazioni, più velocità di computazione dovuta ad una suddivisione dei job, modularità e convenienza

Nel caso di processi cooperanti, vi è un paradigma tra un processo produttore e consumatore, ossia colui che genera un'informazione che serve in input ad un secondo processo per eseguirsi. La concorrenza implica una presenza di buffer che servono da tampone per il passaggio dal produttore al consumatore e può essere di due tipi

- Illimitato
 - Può dover attendere nuovi progetti
- Limitato
 - Il consumer deve aspettare se il buffer è vuoto, e il producer aspettare se è pieno

Si necessita quindi di strategie per la sincronizzazione di questi due passaggi

Arriviamo all' **INTER PROCESS COMMUNICATION**

Comunicazione tra processi (IPC – Inter Process Communication)

L'**Inter Process Communication (IPC)** è un insieme di **meccanismi che permettono ai processi di comunicare e sincronizzare le loro azioni**, fondamentale nei sistemi operativi multitasking, dove processi distinti devono cooperare o scambiarsi dati.

Scambio di messaggi

Uno dei metodi principali per realizzare l'IPC è lo **scambio di messaggi**, in cui **i processi comunicano tra loro senza condividere lo spazio di indirizzamento** (quindi senza accedere alla stessa area di memoria).

L'IPC tramite messaggi si basa su due operazioni fondamentali:

- **send(messaggio)**: invia un messaggio a un altro processo o a una mailbox. La dimensione del messaggio può essere **fissa o variabile**.
- **receive(messaggio)**: riceve un messaggio.

Affinché due processi (es. P e Q) possano comunicare:

1. Devono **stabilire un canale di comunicazione**.
2. Devono **scambiarsi messaggi** tramite le primitive send e receive.

Il canale di comunicazione può essere implementato a livello:

- **Fisico** (memoria condivisa, bus hardware, rete),
 - **Logico** (connessioni software gestite dal sistema operativo).
-

Comunicazione diretta

Nella comunicazione diretta, i **processi devono conoscere esplicitamente il nome del mittente o del destinatario**. Esistono due modalità:

- **Indirizzamento simmetrico:**
 - send(P, messaggio): manda un messaggio al processo **P**.
 - receive(Q, messaggio): riceve un messaggio dal processo **Q**.
- **Indirizzamento asimmetrico:**
 - send(P, messaggio): manda un messaggio al processo **P**.
 - receive(id, messaggio): riceve un messaggio da **qualunque processo**; il sistema salva l'identificativo del mittente nella variabile id.

Proprietà della comunicazione diretta:

- Le connessioni sono **stabilite automaticamente**.
 - Ogni connessione è **associata a due processi esatti**.
 - Per ogni coppia di processi esiste **una sola connessione**.
 - Le connessioni sono normalmente **bidirezionali** (ma possono essere anche unidirezionali).
 - Ha **modularità limitata** a causa dell'esplicitazione degli identificatori dei processi.
-

Comunicazione indiretta: le mailbox (caselle di posta)

Nella comunicazione indiretta, i messaggi sono inviati e ricevuti **attraverso mailbox (o porte)**, oggetti software con **un identificatore univoco**.

- I processi comunicano solo se **condividono una mailbox**.
- Il canale di comunicazione viene stabilito solo quando **entrambi i processi accedono alla stessa mailbox**.

Caratteristiche della comunicazione con mailbox:

- Una mailbox può essere associata a **più di due processi**.
- Possono esistere **più connessioni** tra due processi, ognuna tramite una diversa mailbox.
- Le connessioni possono essere **unidirezionali o bidirezionali**.

Operazioni principali:

- `create_mailbox(A)`: crea la mailbox A.
 - `send(A, messaggio)`: invia un messaggio alla mailbox A.
 - `receive(A, messaggio)`: riceve un messaggio dalla mailbox A.
 - `delete_mailbox(A)`: elimina la mailbox A.
-

Condivisione di una mailbox

Quando **più processi condividono una stessa mailbox** (es. P1, P2, P3 condividono A), possono verificarsi ambiguità:

Se P1 invia un messaggio alla mailbox A, **chi lo riceverà** tra P2 e P3?

La risposta dipende dallo **schema di gestione scelto**:

- Solo **due processi** possono essere associati alla stessa connessione.
- Solo **un processo alla volta** può fare `receive()`.
- Il **sistema operativo decide arbitrariamente** chi riceverà il messaggio, e può anche **notificare il mittente** sull'avvenuta ricezione.

Tipi di mailbox:

- **Sistema operativo**: esistono indipendentemente dai processi.
 - **Processo utente**: la mailbox è parte dello spazio di memoria del processo.
 - Il **proprietario** può solo ricevere.
 - Gli **utenti** (altri processi) possono solo inviare.
 - Alla **terminazione del processo proprietario**, la mailbox viene distrutta e i mittenti ricevono una **notifica di errore**.
-

Sincronizzazione nella comunicazione

Il comportamento della comunicazione può essere **bloccante (sincrono)** o **non bloccante (asincrono)**:

- **Invio/Ricezione bloccanti (sincroni)**:
 - Il **mittente si blocca** finché il messaggio non viene ricevuto.
 - Il **ricevente si blocca** finché non è disponibile un messaggio.
 - Questo tipo di comunicazione è detto **rendezvous** (incontro tra i due processi).
 - **Invio/Ricezione non bloccanti (asincroni)**:
 - Il mittente **invia il messaggio e continua l'esecuzione**.
 - Il ricevente **riceve immediatamente** un messaggio, se presente, oppure un **valore nullo**.
-

Bufferizzazione dei messaggi

I messaggi tra processi sono temporaneamente conservati in **code di comunicazione** (buffer). Esistono tre modelli di buffer:

1. **Capacità zero** (rendezvous):
 - La coda **non può contenere messaggi**.
 - Il mittente **deve bloccarsi finché il ricevente non è pronto** a ricevere.
2. **Capacità limitata**:
 - La coda ha una **dimensione massima finita**.
 - Il mittente **si blocca solo se la coda è piena**; altrimenti invia e continua.
3. **Capacità illimitata**:
 - La coda può contenere **un numero indefinito di messaggi**.
 - Il mittente **non si blocca mai**.

Questi meccanismi di comunicazione e sincronizzazione tra processi sono **fondamentali per la cooperazione** tra programmi indipendenti, garantendo **correttezza, efficienza e affidabilità** nei sistemi operativi.

Sistemi Client-Server e Comunicazione Remota

Nei sistemi distribuiti, la comunicazione tra processi può avvenire tra **nodi diversi di una rete**, secondo l'architettura **client-server**. In questo modello, **il client richiede un servizio e il server lo fornisce**, spesso attraverso una rete IP.

Socket: il canale di comunicazione

Un **socket** rappresenta un **estremo di un canale di comunicazione** tra due processi su rete. È identificato in modo univoco tramite una **coppia (indirizzo IP, numero di porta)**, detta **semi-associazione**.

- I **server** restano in ascolto su una **porta specifica** per ricevere richieste dai client.
 - Alcuni server noti (come quelli per HTTP, FTP o Telnet) utilizzano **well-known ports**, ovvero numeri di porta inferiori a 1024.
 - La comunicazione client-server può essere implementata in diversi modi, tra cui:
 - **Socket**
 - **Remote Procedure Call (RPC)**
 - **Remote Method Invocation (RMI)**
-

Remote Procedure Call (RPC)

La **chiamata di procedura remota (RPC)** permette a un processo di invocare una procedura che risiede su un sistema remoto, astratta come una normale chiamata di funzione.

- La **RPC** è basata su uno scambio strutturato di messaggi, non semplici dati grezzi.
- Ogni messaggio contiene:
 - L'identificativo della funzione da eseguire
 - I **parametri**, tradotti tramite un meccanismo detto **marshalling**
- I messaggi sono indirizzati a un **demone RPC** che ascolta su una porta specifica sul sistema remoto.

Componenti della RPC

- **Stub client**: prepara il messaggio, effettua il marshalling e stabilisce la connessione con il server.
- **Stub server**: riceve il messaggio, effettua il **demarshalling**, invoca la funzione e restituisce il risultato.
- Il **formato di scambio dei dati** è standardizzato (es. **XDR – eXternal Data Representation**).

Esecuzione sicura

- Ogni messaggio è **etichettato con una marca temporale** per evitare **duplicazioni o perdite**.
- Il server mantiene uno **storico dei messaggi elaborati**.
- Il client riceve un **acknowledgment (ack)** di conferma.
- In caso di timeout senza ack, il client **ripete la chiamata RPC**.
- Le informazioni di connessione possono essere:
 - **Predeterminate**
 - Ottenute dinamicamente tramite un **demone matchmaker** (accoppiatore).

Remote Method Invocation (RMI)

La **Remote Method Invocation (RMI)** è l'equivalente in **Java** della RPC, ma estende il concetto alla **programmazione orientata agli oggetti**.

- Consente a un programma Java di **invocare metodi di oggetti remoti**, situati su un altro sistema della rete.
- RMI permette di passare **oggetti completi come parametri**, non solo dati primitivi.

Componenti della RMI

- **Stub (client)**: crea un pacchetto (**parcel**) contenente il nome del metodo e i parametri (con marshalling).
- **Skeleton (server)**: riceve il pacchetto, traduce i parametri e invoca il metodo corretto.

Flusso di esecuzione

1. Il client chiama un metodo remoto tramite lo **stub**.
2. Lo stub esegue il **marshalling dei parametri** e invia il pacchetto.
3. Lo **skeleton sul server** riceve il pacchetto, esegue il metodo e restituisce il **risultato** (o un'eccezione).
4. Il risultato viene **ritrasformato** dallo stub nel formato interno e passato al client.

Passaggio dei parametri in RMI

- Se gli oggetti sono **locali**, vengono **serializzati** e passati **per valore**.
- In altri casi, il passaggio avviene **per riferimento**.

THREAD

È un flusso di controllo relativo ad un dato processo. Molti SO permettono ad un processo di contenere più flussi di controllo

- **Creare un nuovo processo** è un'operazione **costosa**: richiede tempo e molte risorse (memoria, spazio indirizzi, tabelle, ecc.).
- Quando più attività devono eseguire **funzioni simili o ripetitive**, **non ha senso duplicare l'intero processo**.
- In questi casi, è **più efficiente usare thread**, che sono:
 - **più leggeri** dei processi,
 - **condividono risorse** (memoria, file, ecc.) con il processo principale.
- Esempi tipici:
 - **Server HTTP** che gestisce migliaia di richieste contemporaneamente.
 - **Server RPC e RMI**, che lavorano **naturalmente in modalità multithread**.
- I **thread** permettono quindi un'esecuzione **più veloce e scalabile** in ambienti ad alta concorrenza.

La definizione di **thread** è un'unità atomica di utilizzo della cpu, che comprende

- Identificatore
- Program counter
- Register set
- Stack

Due o più thread relativi al medesimo processo condividono sezioni di codice e dati e risorse

I vantaggi sono numerosi come:

- Prontezza di risposta
 - Migliore interazione con l'utente nel caso di app interattive
- Condivisione di risorse
 - I thread condividono memoria e risorse del processo al quale appartengono e questo rende possibile avere un elevato numero di thread all'interno dello stesso spazio di indirizzi
- Economico
 - È più economico cambiare il contesto dei thread piuttosto che fra processi dato che essi condividono memoria e risorse

Possiamo avere due tipi di thread

- **Thread a livello utente**
 - Sono gestiti apparte sopra il kernel del SO
 - Sono formati tramite librerie di funzioni per la creazione, lo scheduling e la gestione
- **Thread a livello kernel**
 - Sono gestiti direttamente dal SO
 - Il kernel si occupa di crearli, di fare lo scheduling, sincronizzazione e la cancellazione

Abbiamo diversi tipi di modelli per i thread

MOLTI A UNO

Mette insieme molti thread di livello utente in un unico thread a livello kernel, la gestione avviene a livello utente, l'intero processo si blocca se effettua una chiamata bloccante, non si possono seguire thread multipli in parallelo in quanto solo uno può accedere al kernel

A UNO A UNO

Ogni thread a livello utente corrisponde ad un thread a livello kernel, utile perché aumenta la concorrenza, in quanto possono lavorare in parallelo sullo stesso processore

MOLTI A MOLTI

Permetto di aggregare molti thread a livello utente verso un numero più piccolo o equivalente di kernel thread, questo permette al SO di creare un numero sufficiente di kernel thread in base alle caratteristiche dell'architettura

In generale nei processi

- fork() primitiva (chiamata di sistema) che si occupa di generare un nuovo processo – riporta il valore zero id) nel nuovo processo e diverso da zero id del figlio nel processo padre
- exec() chiamata di sistema usata dopo fork() per rimpiazzare lo spazio di memoria del processo che ha chiamato la exec() con quella del nuovo programma

nei thread invece

- La chiamata exec() ha un funzionamento pressoché analogo:
 - si procede a rimpiazzare l'intero processo inclusi tutti i suoi thread. E puoi duplicare tutti i thread o duplicare solo i thread che invoca la fork
- Quale delle due versioni di fork() è da usare?
- Se la chiamata è seguita da una exec(), non è necessario duplicare l'intero thread set dal momento che il programma specificato come parametro della exec() rimpiazzerà l'intero processo.
- In caso di fork() singola è opportuno che si duplichi l'intero thread set.

CANCELLAZIONE DI THREAD

È l'atto di terminare un thread prima che abbia completato la propria esecuzione

Per eliminare un thread target si può intervenire in due modi

- cancellazione asincrona: un thread termina il thread target
- cancellazione differita: il thread target può periodicamente effettuare un controllo di terminazione così si può eliminare in modo ordinario

la cancellazione asincrona può recare problemi quando viene eliminato un thread che sta modificando dati condivisi con altri thread oppure le risorse del thread cancellato non vengono integralmente restituite

🔴 Segnali nei Sistemi UNIX

- I **segnali** sono un meccanismo per **notificare a un processo** che si è verificato un **evento** (es. errore, interruzione, richiesta).
- Possono essere ricevuti in modo:
 - **Sincrono** → generati da eventi causati dal processo stesso (es. divisione per zero).
 - **Asincrono** → generati da eventi esterni (es. kill, timer, interruzioni hardware).

⚙️ Gestione dei Segnali

1. **Gestore di default** → fornito dal **kernel**.
2. **Gestore definito dall'utente** → il processo definisce una propria funzione per gestire il segnale.

👉 La gestione può:

- Ignorare il segnale.
- Eseguire una specifica azione tramite un **signal handler**.
- Terminare il processo.

📧 Segnali e Multithreading

La gestione è più complessa nei processi con più thread. Esistono varie strategie per il **signal delivery**:

1. Inviarlo **solo al thread coinvolto** (tipico per segnali sincroni).
 2. Inviarlo a **tutti i thread**.
 3. Inviarlo a un **sottoinsieme di thread**.
 4. Inviarlo a un **thread designato** per ricevere i segnali.
- **Segnali sincroni**: vanno al **thread che ha causato l'evento**.
 - **Segnali asincroni**: UNIX può decidere quale thread li riceve (es. primo thread che non li blocca).

🖥️ Segnali in Windows

- Windows **non ha segnali** nel senso UNIX.
- Usa **Asynchronous Procedure Calls (APC)** per **emulare un comportamento simile**.

Gestione dei Thread nei Sistemi Operativi

Thread Pool (gruppi di thread)

- Nei **server multithread**, creare un nuovo thread per ogni richiesta può:
 - Esaurire risorse di sistema (CPU, memoria).
 - Introdurre attese imprevedibili per la creazione di nuovi thread.

Soluzione: uso di un pool di thread

- I thread vengono creati **all'avvio del processo** e messi in **attesa (sleeping)**.
- Quando arriva una richiesta, un thread del pool viene **risvegliato** e assegnato al lavoro.
- Completato il lavoro, il thread torna **nel pool**.

♦ Vantaggi:

- **Maggiore efficienza**: evitare overhead di creazione/distruzione thread.
 - **Controllo** sul numero massimo di thread attivi.
 - Le dimensioni del pool possono essere **statiche** o **adattive** in base al carico.
-

LWP – LightWeight Process

- Nei modelli **multi-a-molti** o **a due livelli**, serve una mediazione tra **user thread** e **kernel thread**.
- Un **LWP** è un'entità intermedia che:
 - Funziona come **processore virtuale** per i thread utente.
 - È visibile al kernel.
 - Permette ai thread utente di essere **schedulati** sul sistema.

♦ Utilizzo tipico:

- In **applicazioni CPU-bound**: può bastare 1 LWP.
 - In **applicazioni I/O-bound**: servono più LWP per evitare blocchi.
-

Attivazione dello Scheduler (Upcall)

- Il **kernel** comunica con la **libreria dei thread utente** attraverso **upcall** (richiami).
- Quando un thread si blocca, il kernel:
 1. **Genera una upcall.**
 2. **Alloca un nuovo LWP.**
 3. La libreria dei thread salva lo stato del thread bloccato e **programma un altro thread** sul nuovo LWP.
- Quando il blocco termina, una nuova **upcall** permette di **riattivare** il thread fermo.

✅ Obiettivo: mantenere il **giusto numero di thread attivi** → massimizzare la reattività ed efficienza.

Pthreads (POSIX Threads)

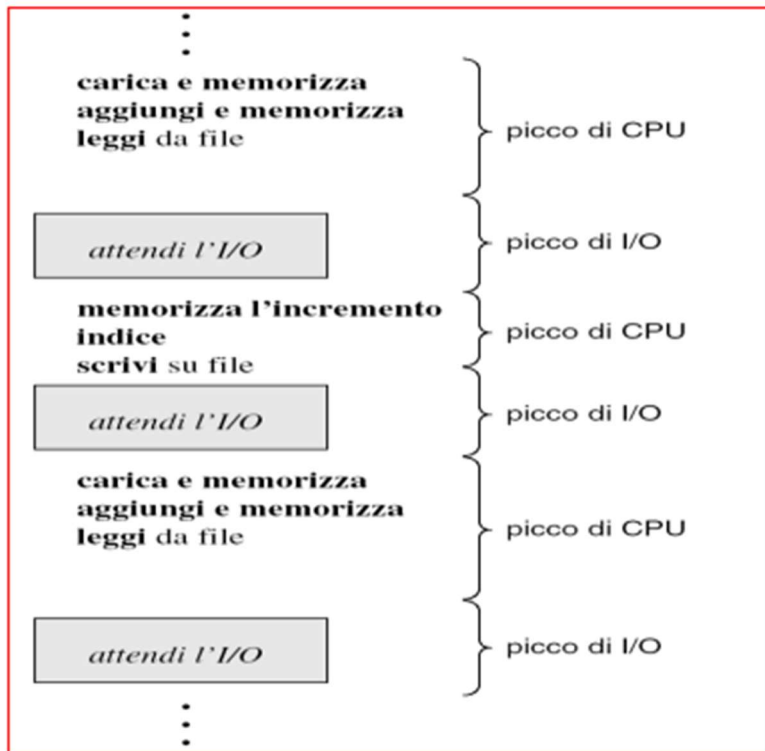
- Standard POSIX (IEEE 1003.1c) per la gestione dei thread.
- Fornisce **API per creare, sincronizzare, gestire** thread a livello utente.
- Ampiamente usato nei sistemi UNIX (Linux, Solaris, macOS).
- Non vi è obbligo di **mappatura diretta** tra un **pthread** e un thread del **kernel**.

CAPITOLO 6 IMPORTANTE

CPU SCHEDULING

La multiprogrammazione cerca di massimizzare la cpu, mantenendo in memoria molti processi in contemporanea, difatti se un processo va in wait il SO assegna alla cpu un nuovo processo per far si che non si fermi mai. Questa schedulazione va fatta bene

L'esecuzione di un processo consiste in un ciclo di esecuzione della cpu ed in un attesa di i/o , i processi si alternano tra picchi di cpu e picchi di i/o

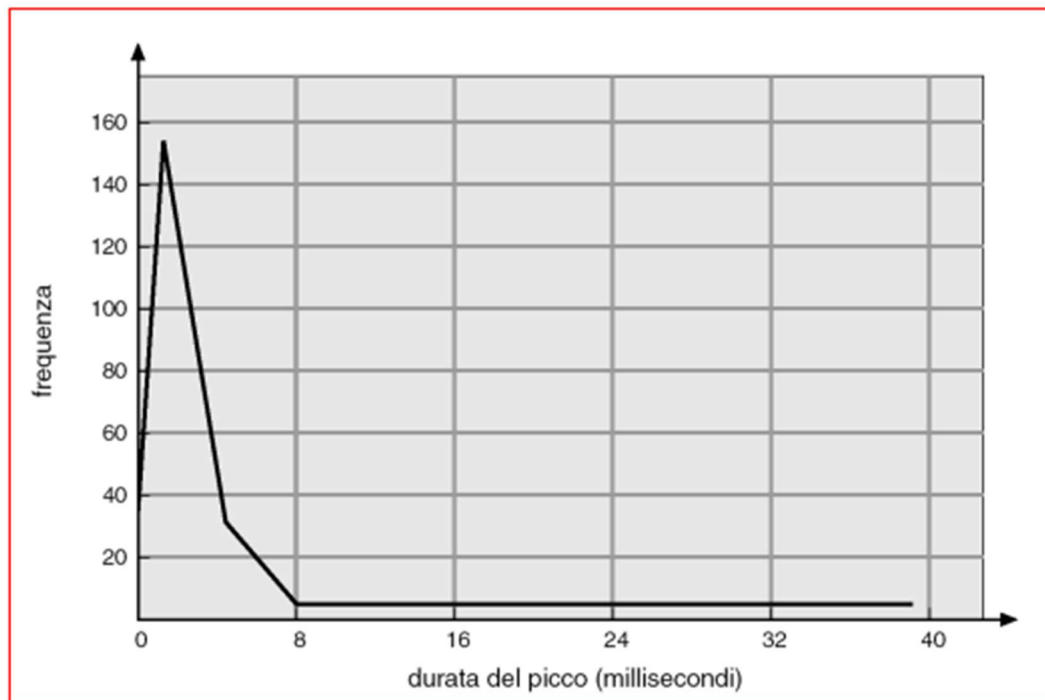


PICCO DI CPU

Le durate dei picchi di cpu variano in base all'architettura ed al processo, ma grosso modo seguono tutti un andamento iperesponenziale con un elevato numero di picchi brevi e pochi picchi lunghi

- Un programma I/O-bound ha molti picchi brevi di CPU.
- Un programma CPU-bound ha pochi picchi di lunga durata.

Tempi di picco della CPU



FUNZIONAMENTO DELLO SCHEDALUTORE DELLA CPU

Il sistema operativo deve scegliere quale processo mandare alla cpu tra la coda dei ready, le decisioni possono avvenire nelle seguenti 4 circostanze

1. Quando un processo passa da esecuzione alla coda di wait
2. Quando un processo passa da esecuzione alla coda di ready
3. Quando un processo passa dalla coda di wait alla coda di ready
4. Quando termina un processo

Il punto 1. e 4. Vengono definiti nonpreemptive ovvero senza sospensione dell'esecuzione, perché semplicemente deve essere scelto un nuovo processo dalla coda di ready

Tutto il resto ovvero 2 e 3 sono preemptive ossia con la sospensione dell'esecuzione, questa è più sofisticata e comporta un costo legato all'accesso di dati condivisi. A differenza della schedulazione non preemptive, in cui un processo mantiene il controllo del processore fino al completamento o al rilascio volontario, con la preemptive il sistema operativo può **interrompere un processo in esecuzione** per assegnare la CPU a un altro processo più prioritario. Questo permette una **maggiore reattività**, soprattutto nei sistemi multitasking.

Tuttavia, questa flessibilità comporta **una serie di complessità** tecniche, soprattutto quando si ha a che fare con **dati condivisi tra processi**. Se un processo viene interrotto mentre sta modificando dei dati che saranno utilizzati da un altro processo, si rischia che quei dati risultino **incompleti o incoerenti**. Per evitare situazioni di inconsistenza, è necessario introdurre dei **meccanismi di sincronizzazione**, come:

- **Mutex (mutual exclusion)**: per garantire che solo un thread o processo possa accedere a una risorsa alla volta;
- **Semafori**: per gestire il coordinamento tra più processi;
- **Sezioni critiche protette**, che bloccano temporaneamente l'accesso concorrente a porzioni sensibili di codice.

Questa esigenza di protezione influisce anche sul **progetto del kernel**. Durante l'esecuzione di una chiamata di sistema, il kernel può essere impegnato nella modifica di strutture dati interne. Se il processo associato venisse interrotto in quel momento, potrebbero verificarsi gravi **errori di consistenza**. Alcuni sistemi operativi risolvono il problema **disabilitando temporaneamente la preemption durante le operazioni critiche**, ma questa scelta:

- riduce la capacità del sistema di **rispondere tempestivamente** agli eventi;
- peggiora il supporto per il **tempo reale** e il **multiprocessing**.

Un ulteriore elemento di complessità è dato dagli **interrupt**, ovvero segnali hardware o software che informano la CPU di eventi esterni. Essendo asincroni per natura, **non possono essere ignorati** o ritardati troppo. Per questo, le sezioni di codice soggette a interrupt devono essere **progettate per gestire l'accesso concorrente**, ad esempio utilizzando tecniche di disabilitazione temporanea degli interrupt o meccanismi di sincronizzazione.

Il **process loading** rappresenta quella fase cruciale in cui il sistema operativo passa il controllo della CPU da un processo a un altro. Questa operazione è gestita da un componente specifico del sistema operativo chiamato **dispatcher**.

Il **dispatcher** entra in azione dopo che il **micro-schedulatore** (o short-term scheduler) ha scelto quale processo deve essere eseguito tra quelli pronti. Il suo compito è quello di **attivare concretamente il processo selezionato**, svolgendo una serie di operazioni fondamentali:

- **Cambio di contesto**: il dispatcher salva lo stato del processo attualmente in esecuzione (registri, program counter, ecc.) e carica lo stato del nuovo processo selezionato;
- **Passaggio alla modalità utente**: poiché il sistema operativo lavora in modalità kernel per motivi di sicurezza, è necessario passare alla modalità utente affinché il processo possa eseguire codice non privilegiato;
- **Salto alla locazione di memoria corretta**: il dispatcher individua il punto in cui il nuovo processo aveva interrotto la sua esecuzione e riprende da lì.

Un aspetto importante legato a questo meccanismo è la **latenza del dispatcher**, ovvero il **tempo impiegato per completare il cambio da un processo all'altro**. Idealmente, questa latenza dovrebbe essere la più bassa possibile, poiché rappresenta un overhead (cioè un costo) che non contribuisce direttamente all'esecuzione di alcun processo, ma è necessario per gestire il multitasking.

CRITERI DI SCHEDULAZIONE

Gli algoritmi di schedulazione della cpu hanno diverse proprietà e possono favorire una categoria di processi piuttosto che un'altra, di base i criteri fondamentali sono:

- **Massimizzare l'Utilizzo della cpu** – in modo da mantenere la cpu il più impegnata possibile
- **Massimizzare Frequenza di completamento throughput** - numero di processi completati per unità di tempo
- **Minimizzare Tempo di completamento turnaround time** – intervallo che comprende va dall'immissione del processo al momento di completamento = $T_{loading} + T_{ready} + T_{cpu} + T_{i/o}$
- **Minimizzare Tempo di attesa** – somma dei tempi spesi in attesa della coda dei processi pronti
- **Minimizzare Tempo di risposta** – tempo che intercorre dalla formulazione della richiesta fino alla produzione della prima risposta

ESEMPI DI SCHEDULAZIONE FCFS: FIRST CAME- FIRST SERVED

1. processi non preentiv: una volta entrati non può essere interrotto
2. i processi vengono eseguiti in ordine di arrivo
3. utilizzo una coda FIFO(first in first out) per gestire l'ordine dei processi

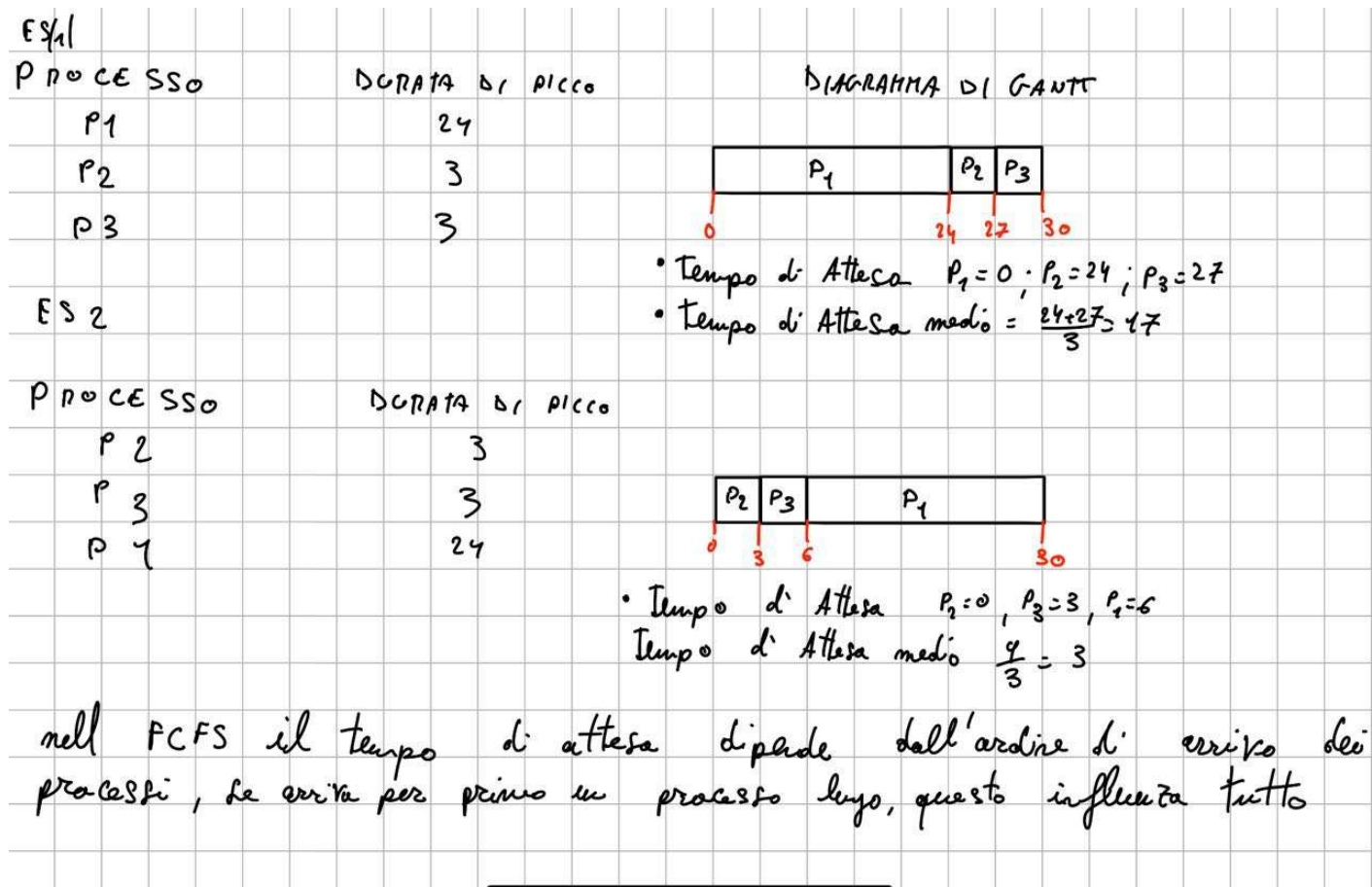
Abbiamo dei contro:

1. poco reattivo – i processi più corti devono aspettare troppo a causa di processi più lunghi
2. tempi di attesa elevati

PRO:

1. deterministico: facile da utilizzare e implementare

Non può essere usato per sistemi real time o interattivi



SCHEDULAZIONE SJF

Viene associata a ciascun processo la lunghezza del successivo picco di cpu del processo medesimo da qui (SHORTEST NEXT CPU BURST FIRST)

Serve per schedulare il processo con il minor tempo di cpu richiesto.

Si calcola il tempo di attesa per ogni processo che sarebbe il tempo che passa dall'arrivo a quando viene eseguito, per intero nel caso di preemptive

Questo algoritmo può essere usato sia per i nonpreemptive che per quelli preemptive:

- nonpreemptive – quando un processo arriva nella coda dei processi pronti mentre il processo precedente è ancora in esecuzione l'algoritmo permette al processo di finire il suo uso della CPU. Poi mette il successivo con il tempo di picco più corto

ES 2 **NON PREEMPTIVE**

PROCESSO	TEMPO DI ARRIVO	TEMPO DI PICCO	DI AGGIUNTA DI C-OUT
P1	0,0	7	
P2	2,0	4	
P3	4,0	1	
P4	5,0	4	

Diagramma di Gantt:

TEMPO $P_1 = 0$ / $P_2 = (7 - 2) = 5$ / $P_3 = (7 - 4) = 3$ / $P_4 = (12 - 5) = 7$

TEMPO DI ATTESA MEDIA = $\frac{0 + 5 + 3 + 7}{4} = 4$

- preemptive – quando un processo arriva nella coda dei processi pronti mentre il processo precedente è ancora in esecuzione l'algoritmo ferma il processo correntemente in esecuzione. Questa schedulazione è anche detta shortest-remaining-time-first. Al nuovo arrivo, confronta il tempo rimasto a quello in esecuzione rispetto a quello di picco del nuovo arrivato e sceglie quello più breve

ES 3 **PREEMPTIVE (CON CONTEXT SWITCH)**

PROCESSO	TEMPO DI ARRIVO	TEMPO DI PICCO	DI AGGIUNTA DI C-OUT
P1	0,0	7	
P2	2,0	4	
P3	4,0	1	
P4	5,0	4	

Diagramma di Gantt:

TEMPO $P_1 = (0 + (7 - 2)) = 5$ / $P_2 = 1$ / $P_3 = 0$ / $P_4 = 2$

TEMPO MEDIA = $\frac{5 + 1 + 0 + 2}{4} = 2$

La **schedulazione a priorità** è una strategia usata dai sistemi operativi per decidere quale processo debba ricevere la CPU, assegnando a ciascun processo una **priorità numerica**. In questo schema, la CPU viene sempre assegnata al processo con la **priorità più alta**, dove – per convenzione – **un valore numerico più basso indica una priorità maggiore**.

Esistono due varianti fondamentali di questo tipo di schedulazione:

- **Preemptive:** se un nuovo processo arriva con una priorità più alta rispetto a quello attualmente in esecuzione, il sistema interrompe quest'ultimo (lo "preemptie") e assegna immediatamente la CPU al processo prioritario.
- **Non preemptive:** il processo in esecuzione continua fino al termine o fino a quando non rilascia volontariamente la CPU, ma il nuovo processo con priorità più alta viene comunque posizionato in cima alla coda dei processi pronti.

Un esempio interessante è l'algoritmo **SJF (Shortest Job First)**, che può essere visto come una schedulazione a priorità, dove la priorità è data dall'**inverso della durata prevista del prossimo uso della CPU**: in altre parole, i processi con picchi brevi di CPU ricevono una priorità più alta.

Le priorità dei processi possono essere stabilite secondo due modalità:

- **Internamente** al sistema operativo, usando parametri misurabili come:
 - uso della memoria;
 - numero di file aperti;
 - rapporto tra tempo medio di I/O e tempo medio di CPU.
- **Esternamente**, cioè in base a criteri che non riguardano direttamente l'esecuzione del processo, come:
 - l'importanza dell'applicazione;
 - la criticità del processo (per esempio, un processo di controllo industriale può avere priorità più alta di un gioco).

Tuttavia, questo tipo di schedulazione presenta un problema rilevante: il **blocco indefinito**, o **starvation**. Se continuamente arrivano processi con priorità elevata, quelli con priorità bassa rischiano di **non essere mai eseguiti**.

Per evitare questo problema, si può adottare una tecnica detta **invecchiamento (aging)**. Essa consiste nell'**aumentare gradualmente la priorità dei processi più vecchi**, ovvero quelli che attendono da più tempo, garantendo che prima o poi anche essi avranno la possibilità di essere eseguiti.

SCHEDULAZIONE ROUND ROBIN

Si tratta di algoritmo di tipo FCFS con l'aggiunta della preemption per migliorare l'alternanza dei processi, ad ogni processo viene assegnato un quanto del tempo di cpu (time slice), tra i 10-100 msec. Se in questo tempo il processo non lascia la cpu viene interrotto tramite un interrupt e rimesso in coda nei processi pronti.

Se ci sono **n processi** pronti nella coda dei processi pronti e il **quanto di tempo è q** allora **ciascun processo ottiene 1/n del tempo di cpu in parti lunghe al più q unità di tempo**. Ogni **processo non deve attendere più di (n-1)*q unità di tempo**

12 12	TIME SLICE = 20		
PROCESSO	TEMPO DI PICCO	SI AGRUPPA DI CANT	
P1	53		
P2	17		
P3	68 24		
P4			

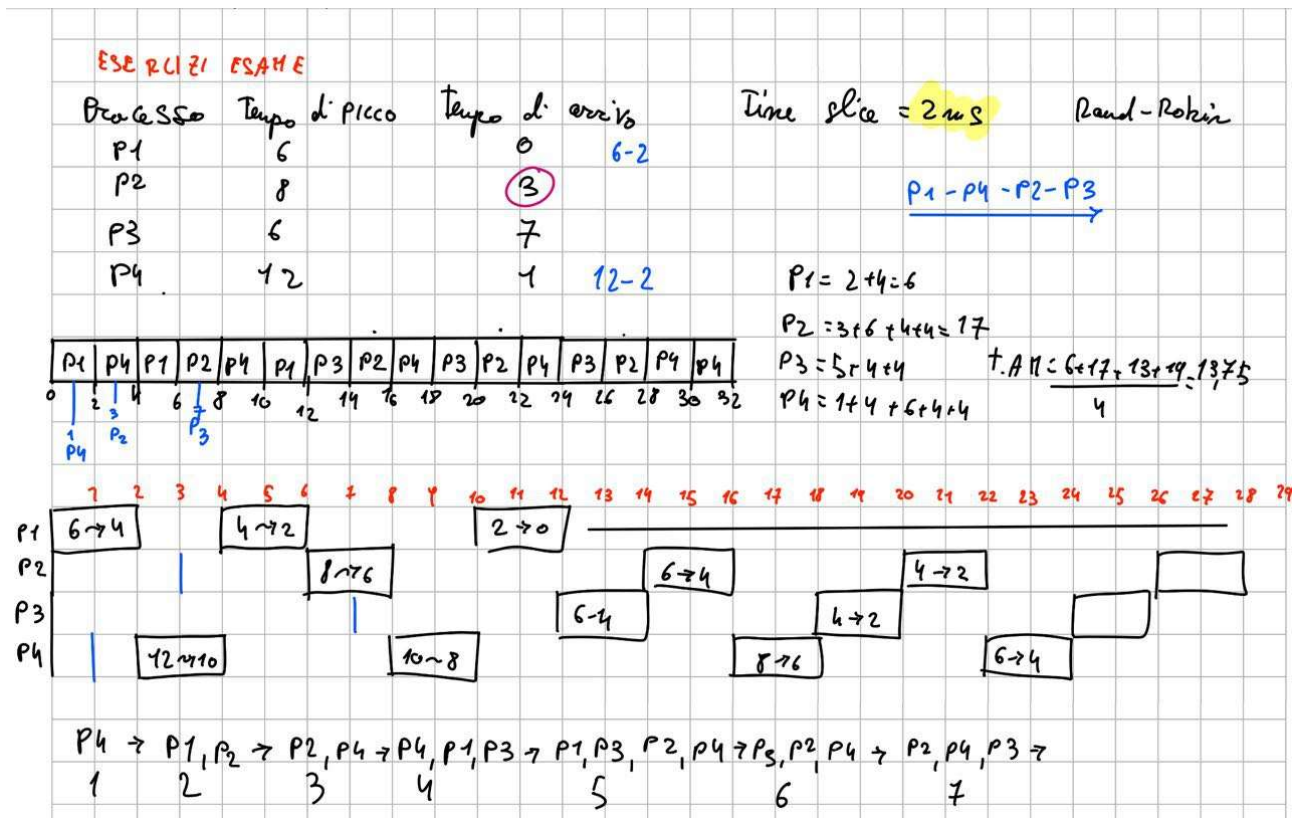
P1	P2	P3	P4	P1	P3	P4	P1	P3	P3
----	----	----	----	----	----	----	----	----	----

20 37 57 77 97 117 121 134 154 162

$P_1 = (77 - 20) + (121 - 97) = 81$
 $P_2 = 20$
 $P_3 = 37 + (97 - 57) + 134 - 117 = 94$
 $P_4 = 57 + (117 - 77) = 97$

TEMPO ATTESA MEDIO = 73

xk



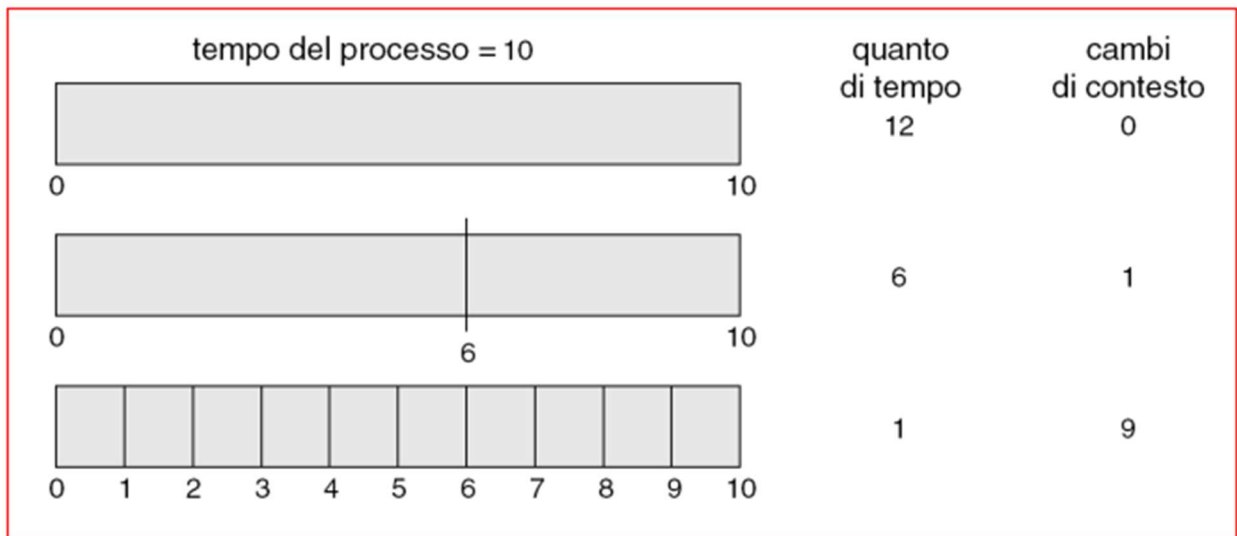
⌚ Time Slice e Context Switching

Per un'efficiente esecuzione dei processi, il **time slice (quanto di tempo)** assegnato a un processo deve essere sufficientemente più grande rispetto al **tempo richiesto per il cambio di contesto (context switching)**.

Ad esempio:

- Se il context switching rappresenta circa il 10% del time slice, allora solo il 90% del tempo viene speso effettivamente per eseguire codice utile.

- Nei sistemi moderni, i time slice sono tipicamente tra **10 e 100 ms**, mentre il cambio di contesto impiega circa **10 µs**.



Round Robin e Turnaround Time

Nel Round Robin, l'**efficienza** del sistema non migliora sempre aumentando la durata del time slice:

- Se il time slice è troppo piccolo, il sistema perde molto tempo in cambi di contesto → peggioramento del **turnaround time medio**.
- Se è abbastanza lungo da completare il picco di CPU della maggior parte dei processi, allora migliora il turnaround.

Coda a più livelli (Multilevel Queue Scheduling)

Questa tecnica suddivide la coda di ready in più sotto-code, ciascuna con **algoritmi di schedulazione diversi**. I processi vengono assegnati **in modo fisso** a una coda in base alle loro caratteristiche.

- **Esempio classico di classificazione:**
 - Foreground (interattivi): algoritmo Round Robin
 - Background (batch): algoritmo FCFS
- Due strategie di schedulazione tra code:
 1. **Priorità fissa:** ad es. foreground ha priorità assoluta su background → può causare starvation.
 2. **Time slicing tra code:** si assegna una percentuale di CPU a ciascuna coda (es. 80% foreground, 20% background).

Coda a più livelli con feedback (MLFQ)

In questo caso, i processi **possono spostarsi** tra code in base al loro comportamento, favorendo quelli interattivi e penalizzando quelli CPU-bound.

- I parametri chiave della MLFQ:
 - Numero di code
 - Algoritmo per ciascuna coda
 - Quando far **salire** un processo (es. dopo un certo tempo di attesa)
 - Quando farlo **scendere** (es. se usa troppa CPU)
 - Dove inserire un nuovo processo
- Esempio:
 - **Q0**: quanto = 8 ms
 - **Q1**: quanto = 16 ms
 - **Q2**: FCFS
 - I nuovi processi partono da Q0 → se non finiscono, scendono a Q1 → poi a Q2.

Schedulazione su Sistemi Multiprocessore

In presenza di più CPU, la schedulazione diventa più complessa:

- **Asymmetric multiprocessing**: solo una CPU gestisce la schedulazione.
- **Symmetric multiprocessing**: tutte le CPU eseguono la schedulazione in autonomia, attingendo da una **coda di ready condivisa**.
- Ciò richiede **sincronizzazione per evitare accessi concorrenti** alle strutture dati comuni.

Schedulazione Real-Time

Sistemi Hard Real-Time

- Richiedono che **ogni processo** venga completato **entro una deadline garantita**.
- Usano **admission control** e **resource reservation**.
- Impongono limiti: non si usano memoria virtuale o dispositivi lenti.
- Adatti a contesti critici (es. controllo industriale, aerospaziale).

Algoritmi:

- **Rate Monotonic**: priorità fisse basate sulla frequenza (più frequente = più alta).
- **Earliest Deadline First (EDF)**: priorità dinamiche basate sulla scadenza più vicina. EDF è teoricamente ottimale, ma difficile da applicare in pratica per via dei costi di gestione.

Sistemi Soft Real-Time

- Meno rigidi, ma privilegiano comunque i processi real-time.
- La priorità non deve cambiare (niente aging).
- Si cerca di **ridurre la dispatch latency**, ad esempio:
 - Interrompendo il kernel in punti sicuri (preemption points)
 - Sospendendo temporaneamente le system call per eseguire processi urgenti

Schedulazione dei Thread

Nei sistemi con supporto ai thread kernel-level, lo **scheduler lavora direttamente sui thread**, non sui processi.

- **Process Contention Scope (PCS)**: competizione tra i thread **dello stesso processo**.
- **System Contention Scope (SCS)**: competizione tra **tutti i thread del sistema**.
 - È quello usato nei moderni OS come Linux, Windows, Solaris, ecc.

CAP 7 SINCRONIZZAZIONE

IL PROBLEMA DELLA SINCRONIZZAZIONE

L'accesso concorrente a dato condivisi può portare alla loro inconsistenza, per mantenere la continenza dei dati sono necessari meccanismi che assicurino l'esecuzione ordinaria di processi cooperanti che condividono uno spazio di indirizzi. Un aspetto fondamentale per garantire la regolamentazione degli accessi è dichiarare come critica la sezione di codice ad accesso condiviso, per esempio se abbiamo n thread, dobbiamo individuare di questi n thread la sezione di codice che consideriamo critica.

Vi sono diversi problemi per la sezione critica:

1. **mutua esclusione** = se un thread sta eseguendo la sua sezione critica, nessun altro può eseguire la propria sezione critica
2. **progresso** – solo i thread che non stanno eseguendo la proprio sc possono scegliere chi sarà il prossimo a entrare nella sezione critica
3. **attesa limitata (bounded waiting)** – esiste un limite al numero di volte in cui gli altri thread possono entrare nelle loro sezioni critiche dopo che un thread ha fatto richiesta di entrare nel proprio e prima che tale richiesta sia esaudita

Questa soluzione diventa complessa e poco efficiente soprattutto nel caso di sistemi con pochi thread; perciò, tutte le soluzioni software sono basate sull'algoritmo di bakery

ALGORITMO DI BAKERY – SOLUZIONE SOFTWARE

L'accesso alle sezioni critiche dei processi è impostato al tipo FCFS

1. si ipotizza l'accesso alla sezione critica per n generici thread
2. prima di entrare in sc, ogni thread riceve un token numerico, il valore più basso è quello che entra prima
3. se due thread T_i e T_j ricevono il medesimo token, accede prima il thread con l'indice più basso

4. i token sono sempre generati in sequenza crescente

HARDWARE PER LA SINCRONIZZAZIONE

Per risolvere il problema della sezione critica si possono usare soluzioni hardware come;

- Monoprocessori – si possono disabilitare gli interrupt durante l'accesso alle sezioni critiche

Questa funzione è troppo inefficiente per multiprocessori, in quanto i comandi per la disabilitazione degli interrupt devono essere passati a tutti i processori e vi possono essere ritardi nell'accesso alle sezioni critiche

I calcolatori moderni forniscono istruzioni hardware speciali che consentono di :

- Controllare e modificare il contenuto di una word di memoria
- Scambiare il contenuto di due parole in modo atomico

SEMAFORI

Le soluzioni hardware se pure molto efficaci sono complesse da implementare per i programmatori

Strumenti poco complessi di sincronizzazione che impongono iterativamente un blocco durante un'attesa in modo attivo (spin lock)

Semaforo S – variabile intera

Si può accedere ad un semaforo S solo tramite due operazioni:

- Acquire()
 - Significa: **chiedo di entrare / di usare la risorsa.**
 - ```
acquire(S) {
```
  - ```
    while (S <= 0); // attesa attiva (busy waiting)
```
 - ```
 S--; // prendo la risorsa
```
  - ```
}
```
 - Se S è 0 o meno → **devo aspettare**, finché qualcuno non la libera (con `release()`).
- Release()
 - Significa: **rilascio la risorsa.**
 - ```
release(S) {
```
  - ```
    S++;
```
 - ```
}
```

Abbiamo due tipi di semafori

### ◆ Semaforo binario

- S = 0 o 1
- Serve a garantire che **un solo thread alla volta entri in una sezione critica.**
- Simile a un **lucchetto** (mutex).

Esempio: proteggere una variabile condivisa

```
Semaphore S = 1; // inizializzato a 1

acquire(S);
 // sezione critica (uso della variabile condivisa)
release(S);
```

---

♦ **Semaforo generalizzato**

- S può essere qualsiasi numero intero  $\geq 0$
- Serve a gestire **più istanze di una risorsa** (es. 3 stampanti disponibili  $\rightarrow S = 3$ )

Esempio: 3 stanze disponibili, 5 thread vogliono entrare

```
Semaphore S = 3;

acquire(S); // se S > 0 entro (uso una stanza), altrimenti aspetto
 // uso stanza
release(S); // libero stanza
```

Questi semafori possono essere usati per controllare l'accesso a sezioni critiche

Semafori come quelli sopra (spinlock) hanno lo svantaggio del busy waiting ossia: Il **busy waiting** (o **attesa attiva**) si verifica quando un **processo o thread continua a controllare ripetutamente** una condizione per poter proseguire, **senza mai sospendersi**.

In pratica, il thread **rimane attivo**, consumando CPU, **anche se non può fare nulla di utile** finché una certa condizione non si sblocca.

PRO: gli spinlock non obbligano a context switch, pertanto tornano utili quando si prevede che i blocchi vengano tenuti per brevi periodi e/o nei sistemi multiprocessore

CONTRO: mentre un processo è nella propria sezione critica, ogni altro processo che prova ad accedere al proprio deve ciclare continuamente sprecando cicli di cpu che potrebbero essere sfruttati da altri processi

Per evitare questo problema cambiamo la definizione di acquire e release

Per fare questo, il semaforo viene rappresentato non solo con un contatore (come prima), ma anche con una **coda di attesa** dei processi bloccati. La struttura può essere così rappresentata:

```
typedef struct {
 int value; // Numero di risorse disponibili
 struct process *list; // Coda dei processi in attesa
} semaforo;
```

### ✂ Cosa succede quando un processo esegue acquire()?

La funzione acquire() serve per **richiedere l'accesso** alla sezione critica:

- Il contatore value viene decrementato.
- Se il risultato è negativo, significa che non ci sono risorse disponibili:
  - Il processo non può procedere.
  - Viene **inserito in una coda di attesa associata al semaforo**.
  - Viene **bloccato**: non viene più eseguito finché qualcuno non lo risveglia.

In questo modo, il processo **non resta in esecuzione a vuoto**, ma viene messo in pausa dal sistema operativo, risparmiando risorse.

### ✅ E quando un processo esegue release()?

La funzione release() viene eseguita quando un processo **termina l'uso della risorsa** e vuole liberarla:

- Il contatore value viene incrementato.
- Se ci sono processi in attesa (cioè se value era  $\leq 0$ ), uno di questi viene:
  - **estratto dalla coda** del semaforo.
  - **risvegliato** e spostato nella **coda dei processi pronti (ready queue)**.

Appena il processore sarà libero, il sistema potrà scegliere di far ripartire il processo risvegliato.

Il SO deve fornire due nuove chiamate di sistema di base

- block: sospende il processo che la invoca
- wakeup(p): riprende l'esecuzione del processo bloccato P

### 📄 La lista d'attesa

Ogni semaforo gestisce una **lista d'attesa** per i processi bloccati. Questa lista viene implementata come una **coda FIFO** (first-in-first-out) usando due puntatori:

- **Testa**: indica il primo processo in attesa.
- **Coda**: indica l'ultimo processo che ha fatto richiesta.

Questa struttura è essenziale per garantire l'ordine corretto di risveglio dei processi.

---

### ⚠ Problema: la sezione critica della gestione del semaforo

L'esecuzione delle operazioni `acquire()` e `release()` su un semaforo deve essere **mutuamente esclusiva**. Se due processi eseguono queste operazioni *sullo stesso semaforo nello stesso momento*, possono verificarsi **inconsistenze**, perché entrambi potrebbero modificare contemporaneamente il contatore del semaforo o la lista d'attesa.

Questa situazione è un classico esempio di **sezione critica**.

---

## 🔧 Soluzioni per proteggere la sezione critica

### In ambienti monoprocesso:

Il sistema può semplicemente **disattivare gli interrupt** mentre un processo esegue `acquire()` o `release()`. In questo modo si assicura che nessun altro processo possa interrompere l'esecuzione in quella sezione critica.

### In ambienti multiprocessore:

Disabilitare gli interrupt **non basta**, perché altri processori potrebbero comunque eseguire codice concorrente. In questo caso servono:

- **Istruzioni hardware specializzate** (come Test-and-Set, Compare-and-Swap...)
- **Soluzioni software** per l'implementazione sicura delle sezioni critiche (ad es. algoritmi di Peterson o Dekker, mutex, etc.)

## 🚦 Semafori e valori negativi

Un'altra caratteristica importante è che i **semafori possono assumere anche valori negativi**. In particolare:

- Se il valore del semaforo è negativo, il **suo valore assoluto** indica **quanti processi sono in attesa** nella coda.
  - Esempio: `value = -3` significa che ci sono 3 processi in coda.

---

## 🕒 Attesa attiva (busy waiting) e semafori

Sebbene i semafori con coda evitino il busy waiting nei casi generali, in alcuni casi il problema **non è del tutto eliminato**, ma solo **spostato**:

- Le funzioni `acquire()` e `release()` includono **sezioni critiche molto brevi**, dove è ancora possibile che si verifichi **una breve attesa attiva**.
- Tuttavia, poiché queste sezioni sono **molto piccole**, il tempo di attesa è **trascurabile nella pratica**.
- Il problema diventa serio solo se le **sezioni critiche diventano troppo lunghe**: in tal caso, la **busy waiting risulta molto penalizzante**, perché blocca la CPU inutilmente per troppo tempo.

## 🔒 Stallo (Deadlock) e Blocco Indefinito (Starvation)

Quando si lavora con più processi che condividono risorse, possono verificarsi problemi come **stallo** e **starvation**, entrambi legati alla sincronizzazione.

### ✓ Cos'è lo stallo (Deadlock)?

Lo stallo si verifica quando **due o più processi restano bloccati, ciascuno in attesa di una risorsa detenuta dall'altro**, in un circolo vizioso dal quale nessuno può uscire.

Nel contesto dei semafori, ciò avviene quando un processo aspetta che venga eseguita una `release()` da un altro processo che, però, è anch'esso bloccato.

#### 💡 Esempio classico di stallo:

```
// Processo P0 // Processo P1
acquire(S); acquire(Q);
acquire(Q); acquire(S);
release(S); release(Q);
release(Q); release(S);
```

In questo scenario:

- P0 ottiene **S** e aspetta **Q**
  - P1 ottiene **Q** e aspetta **S**
- Nessuno può procedere: **deadlock**.

### ⚠ Cos'è il blocco indefinito (Starvation)?

Si parla di **starvation** quando un processo rimane **in attesa troppo a lungo o per sempre** perché non viene mai scelto per accedere alla risorsa, anche se è pronto.

Può accadere se la **coda del semaforo** è organizzata in modo non equo, ad esempio **LIFO (ultimo arrivato, primo servito)**, dove i processi nuovi scavalcano sempre quelli già in attesa.

---

### 🛠 I monitor: alternativa ai semafori

Anche se i semafori sono strumenti potenti per sincronizzare processi, il loro **uso scorretto** può portare a errori difficili da individuare, come quelli di temporizzazione o sincronizzazione.

Per questo, esistono costrutti ad alto livello chiamati **monitor**, che permettono di gestire la sincronizzazione in modo più sicuro e strutturato.

### ✓ Cos'è un monitor?

Un **monitor** è una struttura definita in alcuni linguaggi di programmazione che:

- **Incapsula** variabili condivise e le funzioni che possono manipolarle;
- Garantisce **accesso esclusivo**: solo un thread per volta può eseguire una procedura all'interno del monitor;
- Gestisce la **sincronizzazione automaticamente**, evitando la necessità per il programmatore di scrivere codice `acquire` e `release`.

### Struttura tipica di un monitor:

```
cpp Copia Modifica

monitor NomeMonitor {
 // Variabili condivise (stato)

 public entry funzione1(...) {
 ...
 }

 public entry funzione2(...) {
 ...
 }

 // Codice di inizializzazione opzionale
}
```

Le **entry** (funzioni pubbliche) sono **mutuamente esclusive**, quindi non servono semafori espliciti all'interno.

---

### Sincronizzazione nei monitor: le variabili condition

Per gestire la sincronizzazione tra thread all'interno di un monitor, dato che non vi possono accedere direttamente, si usano le **variabili di tipo condition** (ad esempio x o y) e due operazioni fondamentali:

- `x.wait()` → sospende il thread fino a che **qualcun altro** non invoca...
- `x.signal()` → risveglia **un thread in attesa su x**

Questo meccanismo consente **sincronizzazione cooperativa**, evitando busy waiting.

### DEADLOCK

I processi che sono in attesa possono permanere indefinitamente in tale stato se le risorse che hanno richiesto, sono in possesso di altri processi a loro volta in attesa

Le  $R_m$  risorse possono essere: cicli di cpu, spazio di memoria, periferiche di i/o

Ogni risorsa  $R_i$  può avere più istanze identiche.

Se un processo chiede una risorsa, **gli viene assegnata una copia di quella risorsa (se disponibile)**, quindi la richiesta viene soddisfatta.

Tuttavia, questo **presuppone che la risorsa sia stata definita correttamente nel sistema**.

Se invece ci sono **errori nella classificazione o nella gestione del tipo di risorsa**, allora anche se la risorsa esistesse, la richiesta **potrebbe non essere soddisfatta correttamente**.

I processi usano le risorse in questo modo:

- Richiesta - se essa non può essere soddisfatta immediatamente
- Uso



- Rilascio

Richiesta e Rilascio sono definite chiamate di sistema se la risorsa è controllata dal SO, altri casi possono essere implementate con una coppia di acquire e release di un semaforo

I processi indirizzano al SO le richieste per le risorse gestite dal SO stesso, difatti il SO genera una tabella in cui registra lo stato (free/occupied) di ogni risorsa e per ogni risorsa allocata traccia il processo che la impegna, ovviamente se un processo richiede una risorsa già occupata viene messo in coda.

Un gruppo di processi è in deadlock quando ciascun processo del gruppo è in attesa di un evento che può essere generato solo da un altro processo del raggruppamento, vi sono diversi eventi che generano deadlock:

- Acquisizione/ rilascio di risorse
- IPC ossia comunicazione tra processi

Le successive quattro devono verificarsi contemporaneamente

- **Mutua esclusione** – soltanto un processo alla volta può usare la risorsa
- **Possesso e attesa** – un processo che detiene almeno una risorsa deve attendere per acquisire ulteriori risorse utili alla sua computazione ed allocate ad altri processi
- **Assenza di preemption** – una risorsa non può essere liberata volontariamente ma solo quando ha completato le operazioni su di essa
- **Attesa circolare** – un processo P0 attende una risorsa da P1 e così via

### Rappresentazione grafica di allocazione delle risorse

Vi sono un insieme di nodi V e un insieme di archi E

Gli insiemi V sono divisi in

- Pn – processi del sistema
- Rm – insieme di tutti i tipi di risorse del sistema

Gli archi E possono essere

- Arco di richiesta =  $P \rightarrow R$
- Arco di assegnazione =  $R \rightarrow P$

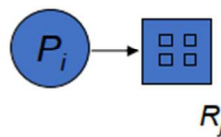
- Processo:



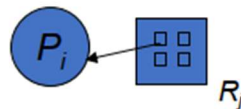
- Tipo di risorsa con 4 istanze (esemplari):



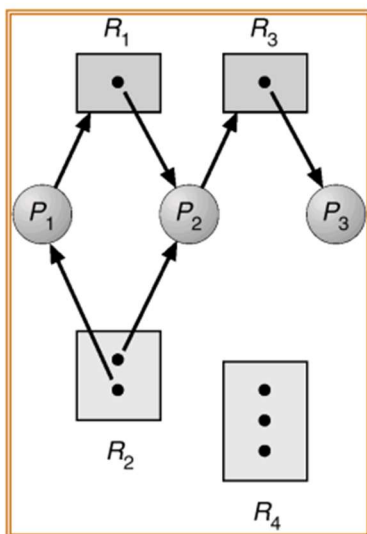
- $P_i$  richiede un'istanza di risorsa di tipo  $R_j$ :



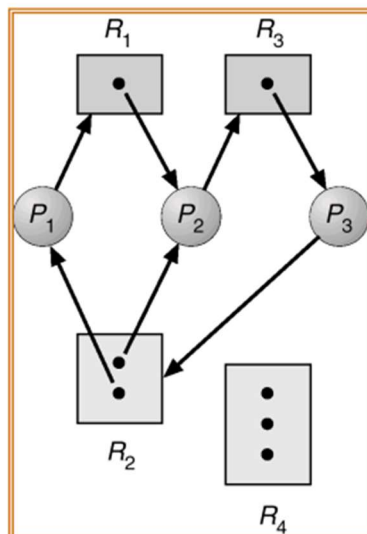
- $P_i$  detiene un'istanza di risorsa di tipo  $R_j$ :



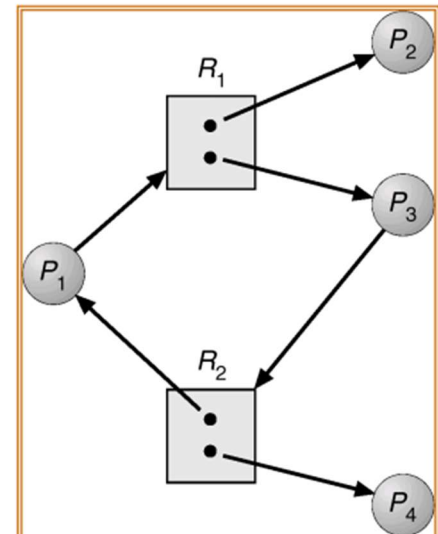
Grafo di allocazione delle risorse



Presenza di cicli con deadlock



Presenza di cicli senza deadlock



- Se il grafo non contiene cicli  $\Rightarrow$  nessun deadlock.
- Se il grafo contiene cicli  $\Rightarrow$ 
  - se ogni tipo di risorsa ha un'unica istanza, allora si verifica un deadlock.
  - se ogni tipo di risorsa ha parecchie istanze, possibilità di deadlock.

## Deadlock Prevention (Prevenzione)

Consiste nell'agire **in anticipo**, strutturando il sistema in modo che **non possano mai verificarsi tutte le condizioni che causano un deadlock**.

In pratica, si **evita alla radice** che i processi entrino in una situazione pericolosa.

Ad esempio:

- Si può impedire che un processo mantenga una risorsa mentre ne chiede un'altra.
- Si può imporre un ordine rigido di richiesta risorse.

✓ **Vantaggio:** Deadlock impossibile.

✗ **Svantaggio:** Poco flessibile e può limitare l'efficienza del sistema.

---

### ⚖️ Deadlock Avoidance (Evitamento)

Qui si **permette ai processi di richiedere risorse**, ma si verifica ogni volta se quella richiesta **porterebbe a uno stato pericoloso**.

Il sistema decide se **accettare o rifiutare** la richiesta in base a una valutazione preventiva.

Un classico esempio è l'**algoritmo del banchiere** di Dijkstra.

✓ **Vantaggio:** Più flessibile della prevenzione.

✗ **Svantaggio:** Richiede conoscenza anticipata del comportamento dei processi (es. quante risorse richiederanno in totale).

---

### 🔍 Deadlock Detection and Recovery (Individuazione e recupero)

In questo approccio si **lascia che i deadlock possano accadere**, ma il sistema li **rileva periodicamente** e poi **interviene** per rimuoverli.

I metodi di recupero possono includere:

- Terminare uno o più processi coinvolti.
- Forzare il rilascio delle risorse.
- Ripristinare uno stato precedente.

✓ **Vantaggio:** Nessuna restrizione preventiva.

✗ **Svantaggio:** Rischio di perdita dati, necessità di rollback, gestione complicata.

---

### 🙄 Ignore the Problem (Ignora il problema)

Approccio più semplice (e anche più comune in molti sistemi, come **UNIX**): si **assume** che i deadlock **non succedano quasi mai** e quindi **non si implementa nulla** per gestirli.

✓ **Vantaggio:** Nessuna complessità aggiuntiva.

✗ **Svantaggio:** Se un deadlock si verifica davvero, il sistema può rimanere bloccato.

La prevenzione del deadlock si basa sull'idea di **evitare che si verifichino tutte insieme le quattro condizioni necessarie per causare uno stallo**. Se anche **una sola di queste condizioni viene negata**, il deadlock non può avvenire.

#### 1. 🗝️ Mutua esclusione

- Alcune risorse, come file o stampanti, **non possono essere condivise**: devono essere usate da **un solo processo per volta**.

- In questi casi, la **mutua esclusione è inevitabile**.
  - Tuttavia, per risorse che possono essere condivise (es. variabili in sola lettura), è **possibile evitare questa condizione**.
  - **Conclusione**: non sempre è possibile evitare la mutua esclusione, ma si può fare dove le risorse sono condivisibili.
- 

## 2. Possesso e attesa (Hold and Wait)

- Per prevenire questa condizione, si impone che **un processo richieda tutte le risorse di cui ha bisogno in una sola volta, prima di iniziare l'esecuzione**.
  - Oppure, si consente a un processo di **richiedere risorse solo se non ne sta già trattenendo altre**.
  - **Problema**: questo metodo riduce l'efficienza del sistema, perché:
    - Le risorse potrebbero restare inutilizzate mentre il processo aspetta altre.
    - Si può verificare **starvation** (alcuni processi non ricevono mai le risorse).
- 

## 3. Assenza di preemption (non sottraibilità)

- Alcune risorse possono essere **interrotte e salvate**, come la CPU o la memoria virtuale.
  - In questi casi, si può applicare **preemption** (cioè forzare un processo a rilasciare risorse).
  - Strategie:
    - Se un processo **richiede una risorsa non disponibile**, viene **forzato a rilasciare tutte le risorse già possedute**.
    - Il processo **attenderà** finché potrà ricevere **tutte le risorse in una volta**, incluse quelle rilasciate.
  - **Vantaggio**: si spezza la catena di attesa.
  - **Nota**: non tutte le risorse supportano questa strategia.
- 

## 4. Attesa circolare (Circular Wait)

- Si impone un **ordine globale** sulle risorse (es.  $R_1 < R_2 < R_3 \dots$ ).
- I processi devono **richiedere le risorse seguendo questo ordine**.
- In pratica:
  - Se un processo possiede  $R_2$ , può solo richiedere  $R_3$ ,  $R_4$ , ecc. (cioè risorse con **indice maggiore**).
  - Se invece vuole una risorsa con indice più basso, **deve rilasciare prima tutte le risorse possedute**.
- Questo metodo **impedisce la formazione di cicli di attesa**, e quindi elimina il rischio di deadlock.

---

### Strumenti di supporto: Witness

- Alcuni sistemi usano un software chiamato "**witness**" che verifica in tempo reale se i processi stanno **richiedendo risorse secondo l'ordine corretto**.
- Questo aiuta a **rilevare eventuali violazioni** della strategia di prevenzione.

### Evitare il Deadlock

L'evitamento del deadlock si differenzia dalla prevenzione perché **non impone restrizioni fisse a priori**, ma valuta **caso per caso**, durante l'esecuzione del sistema, se l'assegnazione di una risorsa può portare o meno a uno stallo.

### Principio di base

Per evitare che si verifichi un deadlock, il sistema:

- Deve conoscere **in anticipo** la **sequenza massima di richieste** che ogni processo potrebbe fare.
- Deve **simulare** se, dopo ogni richiesta, il sistema rimarrebbe in uno **stato sicuro**, cioè uno stato in cui è garantita almeno l'esecuzione di un processo (che poi rilascerà le risorse agli altri).

---

### Cosa valuta il sistema prima di assegnare una risorsa

Al momento in cui un processo richiede una risorsa (tempo  $t_0$ ), il sistema considera:

1. **Quante risorse sono attualmente disponibili.**
2. **Quali risorse sono già assegnate agli altri processi.**
3. **Qual è il massimo numero di risorse che ogni processo potrebbe richiedere in futuro.**

Queste informazioni permettono al sistema di **prevedere se la richiesta può essere soddisfatta senza entrare in uno stato pericoloso**.

---

### Come si ottengono queste informazioni

- Ogni processo deve **dichiarare in anticipo** il **numero massimo di risorse** di ciascun tipo di cui potrebbe aver bisogno.
- Questi dati vengono usati dagli algoritmi per **eseguire controlli dinamici** ogni volta che viene fatta una richiesta.

---

### Algoritmi per l'evitamento

Un esempio famoso è l'**algoritmo del banchiere** (Banker's Algorithm), che:

- Esamina lo stato corrente delle risorse.
- Simula la concessione della richiesta.
- Verifica se, dopo l'assegnazione, esiste una sequenza sicura di esecuzione per i processi.

- Se la risposta è sì → la risorsa viene assegnata.
- Se no → il processo viene fatto **attendere**.

### ✓ Stato sicuro vs. stato pericoloso

- Uno **stato sicuro** è uno stato in cui **esiste almeno una sequenza** in cui tutti i processi possono terminare senza deadlock.
- Uno **stato pericoloso non è ancora un deadlock**, ma **può diventarlo** se si soddisfano altre richieste.

La sequenza  $\langle P_1, P_2, \dots, P_n \rangle$  è sicura se per ogni  $P_i$ , le richieste di risorse che  $P_i$  può attualmente fare possono essere soddisfatte da:

- risorse attualmente disponibili;
- risorse detenute da tutti i processi  $P_j$ , con  $j < i$ .

### ALGORITMO DEL BACHIERE (HABERMANN)

IPOTESI:

- Istanze multiple delle risorse
- Ogni processo  $P$  deve dichiarare il numero massimo di istanze per ogni tipo di risorsa di cui può avere bisogno
- La richiesta massima di risorse non può eccedere la disponibilità totale del sistema
- Un processo deve restituire le risorse in un periodo di tempo finito

Ogni volta che un processo richiede delle risorse, si determina se l'assegnazione genera uno stato sicuro

### ✚ Strutture dati principali

Supponiamo che vi siano:

- $n$  processi:  $P_0, P_1, \dots, P_{n-1}$
- $m$  tipi di risorse:  $R_0, R_1, \dots, R_{m-1}$

Le principali strutture dati utilizzate sono:

#### 1. Available[m]

Un **vettore** che rappresenta il numero di istanze disponibili per ciascun tipo di risorsa.

Esempio:  $\text{Available}[j] = k \rightarrow$  ci sono  $k$  istanze libere del tipo di risorsa  $R_j$ .

#### 2. Max[n][m]

Una **matrice** che specifica il **massimo numero** di istanze di ogni risorsa che ciascun processo può richiedere.

$\text{Max}[i][j] = k \rightarrow$  il processo  $P_i$  può richiedere al massimo  $k$  istanze di  $R_j$ .

#### 3. Allocation[n][m]

Indica quante risorse sono **attualmente assegnate** a ciascun processo.

$\text{Allocation}[i][j] = k \rightarrow P_i$  sta usando  $k$  istanze di  $R_j$ .

#### 4. Need[n][m]

Derivata dalla differenza tra Max e Allocation, rappresenta quante **altre istanze** un processo potrebbe ancora richiedere.

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$

#### ✓ Algoritmo di sicurezza (Safety Algorithm)

L'obiettivo è determinare se lo stato attuale delle risorse sia **sicuro**, cioè se esiste una sequenza di esecuzione in cui ogni processo può completare le proprie operazioni senza causare deadlock.

**Passaggi:**

##### 1. Inizializzazione

- Work = Available
- Finish[i] = false per ogni processo  $i \in [0, n-1]$

##### 2. Ricerca del processo idoneo

Trova un processo  $i$  tale che:

- $\text{Finish}[i] == \text{false}$
- $\text{Need}[i] \leq \text{Work} \rightarrow$  il processo può terminare con le risorse disponibili

##### 3. Simulazione del completamento

Se tale processo esiste:

- $\text{Work} = \text{Work} + \text{Allocation}[i] \rightarrow$  restituisce le risorse al termine
- $\text{Finish}[i] = \text{true}$
- Torna al passo 2

##### 4. Verifica dello stato sicuro

Se tutti i  $\text{Finish}[i]$  sono true, il sistema è **in uno stato sicuro**.

Altrimenti, è **potenzialmente pericoloso** e potrebbe verificarsi un deadlock.

⚠ Il numero massimo di iterazioni è  $m \times n^2$ , quindi può essere costoso in ambienti con molti processi e risorse.

| S/COREZZA | allocation     |   |   | MAX |   |   | AVAILABLE |   |   |   |
|-----------|----------------|---|---|-----|---|---|-----------|---|---|---|
|           | A              | B | C | A   | B | C | A         | B | C |   |
| A - 10    | P <sub>1</sub> | 0 | 1 | 0   | 7 | 5 | 3         | 3 | 3 | 2 |
| B - 5     | P <sub>2</sub> | 2 | 0 | 0   | 3 | 2 | 2         |   |   |   |
| C - 7     | P <sub>3</sub> | 3 | 0 | 2   | 9 | 0 | 2         |   |   |   |
|           | P <sub>4</sub> | 2 | 1 | 1   | 2 | 2 | 2         |   |   |   |
|           | P <sub>5</sub> | 0 | 0 | 2   | 4 | 3 | 3         |   |   |   |

1) initialization  
 Work = available, FINISH = [FALSE, ..., FALSE]

NEED = MAX - AVAILABLE

| NEED           |   |   |   |
|----------------|---|---|---|
| A              | B | C |   |
| P <sub>1</sub> | 7 | 4 | 3 |
| P <sub>2</sub> | 1 | 2 | 2 |
| P <sub>3</sub> | 6 | 0 | 0 |
| P <sub>4</sub> | 0 | 1 | 1 |
| P <sub>5</sub> | 4 | 3 | 1 |

1) finish(i) = false; need(i) ≤ Work  
 P<sub>1</sub> → 7 ≤ 3 ✗; P<sub>2</sub> → 1 ≤ 3 ✓ → Work = Work + allocation[P<sub>2</sub>] = (3, 3, 2) + (2, 0, 0) = (5, 3, 2)  
 FINISH(2) = true

2) P<sub>1</sub> → 7 ≤ 5 ✗; P<sub>3</sub> → 6 ≤ 5 ✗; P<sub>4</sub> → 0 ≤ 5 ✓ Work = (5, 3, 2) + (2, 1, 1) = (7, 4, 3) finish[P<sub>4</sub>] = true

3) P<sub>1</sub> → 7 ≤ 7 ✓ Work = (7, 4, 3) + (0, 1, 0) = (7, 5, 3) finish[P<sub>1</sub>] = true

4) P<sub>3</sub> → 6 ≤ 7 ✓ Work = (7, 5, 3) + (3, 0, 0) = (10, 5, 3) /// (P<sub>3</sub>) = true <P<sub>2</sub>, P<sub>4</sub>, P<sub>1</sub>, P<sub>3</sub>, P<sub>5</sub>>

5) P<sub>5</sub> → 4 ≤ 7 ✓ Work = (10, 5, 3) + (0, 0, 2) = (10, 5, 5) /// (P<sub>5</sub>) = true

### Algoritmo di richiesta di risorse (Resource-Request Algorithm)

Utilizzato quando un processo P<sub>i</sub> fa una nuova richiesta. L'obiettivo è decidere **dinamicamente** se concedere o meno la risorsa, verificando che il sistema resti in uno stato sicuro.

Passaggi:

#### 1. Controllo del limite massimo

- Se Requesti > Need[i] → Errore: il processo ha superato il proprio limite massimo

#### 2. Verifica della disponibilità

- Se Requesti > Available → Le risorse non sono disponibili → P<sub>i</sub> deve **attendere**

#### 3. Allocazione simulata (tentativa)

- Available = Available - Requesti
- Allocation[i] = Allocation[i] + Requesti
- Need[i] = Need[i] - Requesti

#### 4. Esecuzione del safety algorithm

- Se lo stato simulato è **sicuro**, la richiesta viene **approvata** e l'allocazione è definitiva.
- Se lo stato simulato è **insicuro**, la richiesta viene **negata**:



- Le risorse vengono **ripristinate**
- Il processo  $P_i$  viene  **messo in attesa**

**RICHIESTA**

$Request_2 = [1, 0, 2]$

1.  $Need_2 \geq Request_2$ ?  $(1, 2, 2) \geq (1, 0, 2)$  ✓

2.  $Available \geq Request_2$ ?  $(3, 3, 2) \geq (1, 0, 2)$  ✓

•  $AVAILABLE_2 = AVAILABLE - REQUEST = (3, 3, 2) - (1, 0, 2) = (2, 3, 0)$

•  $ALLOCATION_2 = ALLOCATION_1 + REQUEST = (2, 0, 0) + (1, 0, 2) = (3, 0, 2)$

•  $NEED_2 - Request = (1, 2, 2) - (1, 0, 2) = (0, 2, 0)$

e rifaccio l'esercizio

### ALGORITMO DI RILEVAZIONE DEL DEADLOCK

Scopo: rilevare se il sistema è in **deadlock** usando le strutture dati simili all'algoritmo del banchiere.

Ingredienti principali:

- m tipi di risorsa.
- n processi.
- **Available**: vettore lunghezza m — quante istanze libere di ogni tipo ci sono.
- **Allocation**: matrice  $n \times m$  — quante istanze di ogni risorsa sono attualmente assegnate a ciascun processo.
- **Request**: matrice  $n \times m$  — quante istanze in più (eventuali) ogni processo richiede in questo momento.

Idea base: simuliamo *ipoteticamente* l'ordine di completamento dei processi. Se esiste un ordine in cui tutti i processi possono finire (dato lo stato corrente), allora non c'è deadlock; altrimenti c'è deadlock e i processi che non possono essere completati sono in stallo.

1. Inizializza:

- $Work = Available$  (vettore).
- Per ogni processo i:  
se  $Allocation[i, \cdot] == 0$  allora  $Finish[i] = true$  (processo non ha risorse assegnate)  
altrimenti  $Finish[i] = false$ .

2. Cerca un indice i tale che:

- $Finish[i] == false$  e

- $\text{Request}[i, :] \leq \text{Work}$  (confronto elemento per elemento).  
Se non esiste, vai al punto 4.

3. Se trovi tale i:

- $\text{Work} = \text{Work} + \text{Allocation}[i, :]$  (simulo che i finisca e rilasci le sue risorse).
- $\text{Finish}[i] = \text{true}$ .
- Torna al punto 2.

4. Se esiste qualche i con  $\text{Finish}[i] == \text{false}$ , allora quei processi sono in **deadlock**. Altrimenti il sistema non è in deadlock.

RI LEVAZIONE

|                | allocation |   |   | Request |   |   | Available |   |   |
|----------------|------------|---|---|---------|---|---|-----------|---|---|
|                | A          | B | C | A       | B | C | A         | B | C |
| A = 7 istanze  |            |   |   |         |   |   |           |   |   |
| B = 2 istanze  |            |   |   |         |   |   |           |   |   |
| C = 6 istanze  |            |   |   |         |   |   |           |   |   |
| P <sub>1</sub> | 0          | 1 | 0 | 0       | 0 | 0 | 0         | 0 | 0 |
| P <sub>2</sub> | 2          | 0 | 0 | 2       | 0 | 2 |           |   |   |
| P <sub>3</sub> | 3          | 0 | 3 | 0       | 0 | 0 |           |   |   |
| P <sub>4</sub> | 2          | 1 | 1 | 1       | 0 | 0 |           |   |   |
| P <sub>5</sub> | 0          | 0 | 2 | 0       | 0 | 2 |           |   |   |

①  $\text{Work} = \text{Available} = (0, 0, 0)$   
 $\text{finish}[i] = \text{false}$

② if  $\text{Request} \leq \text{Work}$

P<sub>1</sub>  $\text{Request} = (0, 0, 0) \leq (0, 0, 0) \checkmark \rightarrow \text{finish}(P_1) = \text{true} \rightarrow \text{Work} = (0, 0, 0) + (0, 1, 0) = (0, 1, 0)$

P<sub>3</sub>  $\text{Request} = (0, 0, 0) \leq (0, 1, 0) \checkmark \rightarrow \text{finish}(P_3) = \text{true} \rightarrow \text{Work} = (0, 1, 0) + (3, 0, 3) = (3, 1, 3)$

P<sub>2</sub>  $\text{Request} (2, 0, 0) \leq (3, 1, 3) \checkmark // (P_2) = \text{true} \rightarrow \text{Work} = (3, 1, 3) + (2, 0, 0) = (5, 1, 3)$

P<sub>4</sub>  $(1, 0, 0) \leq \text{Work} \checkmark \text{Work} = (5, 1, 3) + (2, 1, 1) = (7, 2, 4)$

P<sub>5</sub>  $\leq \text{Work} \checkmark$

sequenza sicura  $\langle P_1, P_3, P_2, P_4, P_5 \rangle$

## CAPITOLO 9

### MEMORIA CENTRALE

Un programma per essere eseguito deve essere portato dalla memoria di massa a quella centrale (RAM) ed essere attivato come processo

La memoria può essere considerata come un vettore di word ciascuna delle quali è accessibile singolarmente tramite un indirizzo specifico

Ciclo istruzione-esecuzione:

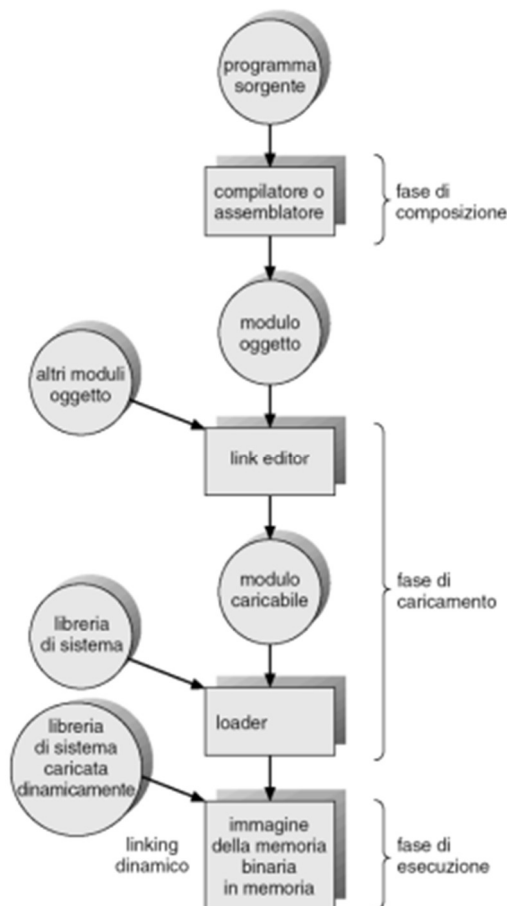
- Prelievo di una istruzione dalla memoria centrale in base al valore del program counter;
- Decodifica dell'istruzione
- Prelievo di operandi dalla memoria ed esecuzione

- Memorizzazione dei risultati in memoria

Dal punto di vista della memoria i passaggi di ingresso e uscita non sono altro che un flusso di indirizzi.

I processi che sono in attesa di essere caricati in memoria centrale finiscono nella coda di entrate. Mentre i programmi utente attraversano più stadi prima di essere eseguiti

Il collegamento delle istruzioni e dei dati agli indirizzi di memoria viene effettuato mediante la seguente successione di passi:



- **Fase di composizione:** se in questa fase si conoscono la locazione del processo in memoria, allora si può generare un codice assoluto
- **Fase di caricamento:** se nella fase precedente non si conosce la locazione del processo in memoria, il compilatore deve generare un codice rilocabile, per ovviare questo problema si usano:
  - **Linking finale ritardato:**  
Normalmente il linking (cioè, collegare il tuo codice con librerie e moduli esterni) può essere fatto subito. Ma qui lo si rimanda **al momento in cui il programma viene caricato in RAM**, così gli indirizzi possono essere risolti in base alla posizione reale
  - **Aggiornamento degli indirizzi nel codice utente:**  
Se il programma viene caricato a un indirizzo di partenza diverso da quello previsto, si devono aggiornare **solo** le istruzioni e i riferimenti nel codice utente che contengono indirizzi assoluti, sostituendoli con quelli corretti.
- **Fase di esecuzione:** se il processo può essere spostato durante l'esecuzione, il collegamento deve essere ritardato fino al momento dell'esecuzione

I vari indirizzamenti possono essere di tipo:

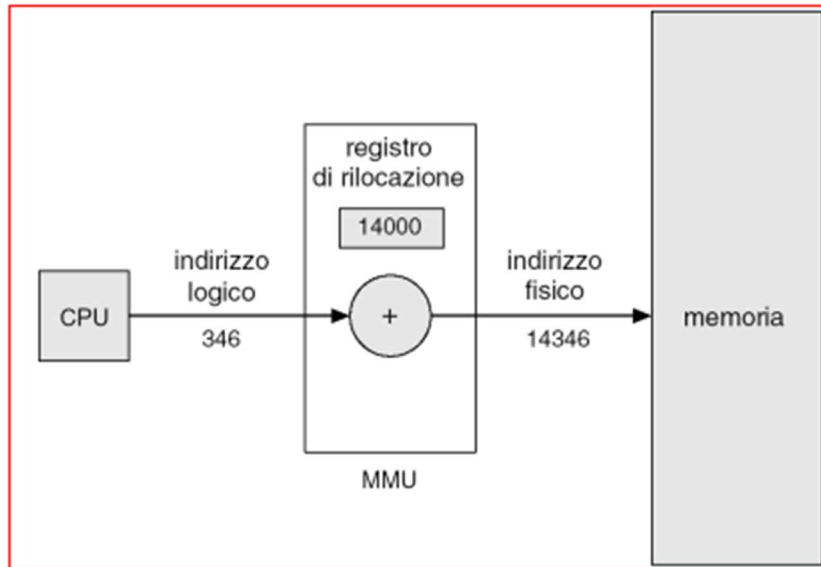
- LOGICO: indirizzo generato dalla CPU, che viene definito come indirizzo virtuale
- FISICO: visto dalla memoria che è caricato nel registro degli indirizzi in memoria

Nella fase di compilazione e di caricamento gli indirizzi fisici corrispondono a quelli logici, nella fase di esecuzione sono diversi

### **MEMORY MANAGEMENT UNIT**

Dispositivo hardware che effettua la trasformazione dagli indirizzi virtuali a quelli fisici in fase di esecuzione, nel suo schema, il valore nel registro di rilocazione è aggiunto ad ogni indirizzo generati da un processo nel momento in cui è trasmesso alla memoria

Il programma utente lavora con gli indirizzi logici



Ecco il riassunto strutturato e fluente del testo che hai fornito:

---

### Caricamento dinamico (Dynamic Loading)

- **Problema di base:** normalmente un processo deve avere tutto il codice e i dati in memoria per poter essere eseguito → la sua dimensione è limitata dalla memoria fisica disponibile.
- **Soluzione:** caricare in memoria solo ciò che serve, quando serve.
  - Una procedura viene caricata **solo** quando viene effettivamente richiamata.
  - Le procedure non utilizzate **non** vengono mai caricate.
  - Tutte le procedure sono salvate su disco in formato **rilocabile** (possono essere caricate in qualsiasi punto della memoria).
- **Funzionamento:**
  - Il programma principale viene caricato e avviato.
  - Se chiama una procedura non presente in memoria → interviene il **loader** per caricarla.
- **Vantaggi:**
  - Miglior utilizzo della memoria, specialmente per codice che serve solo in situazioni rare.
  - Non richiede modifiche particolari al sistema operativo.

---

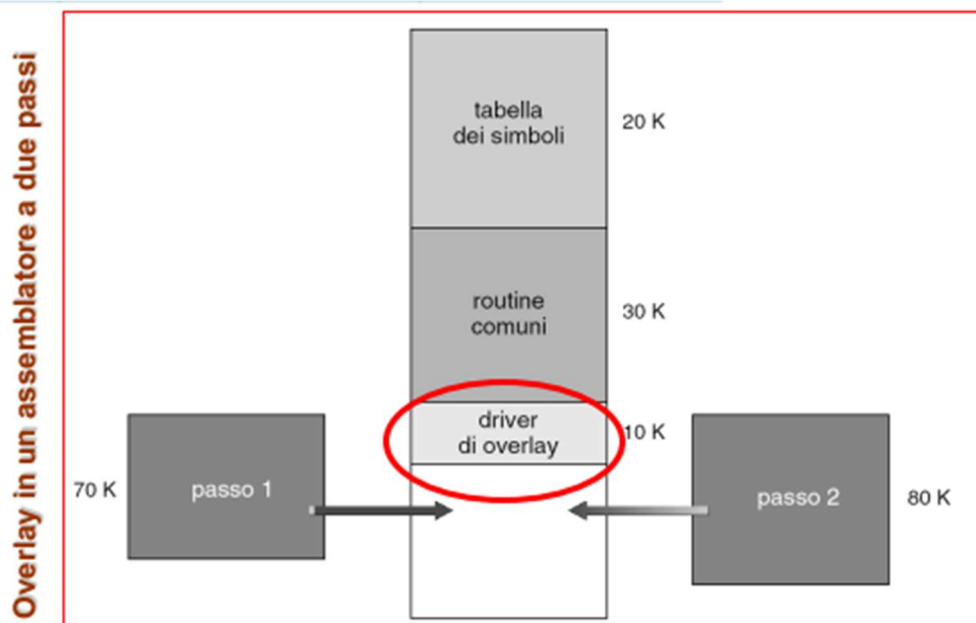
### Collegamento dinamico (Dynamic Linking)

- **Problema di base:** senza collegamento dinamico, ogni programma dovrebbe includere una copia delle librerie di funzioni → spreco di memoria e spazio su disco.
- 
-

- **Soluzione:** il collegamento avviene **durante l'esecuzione**, non in fase di compilazione.
    - Una piccola porzione di codice detta **stub**:
      - Controlla se la procedura di libreria è già in memoria.
      - Se sì → ne ottiene l'indirizzo e la esegue.
      - Se no → la carica e poi la esegue.
    - Dopo il primo uso, lo stub viene sostituito dall'indirizzo effettivo della procedura.
  - **Vantaggi:**
    - Aggiornare una libreria su disco aggiorna automaticamente tutti i programmi che la usano.
    - Riduce lo spreco di memoria, perché più processi possono condividere la stessa libreria in memoria.
  - **Ruolo del sistema operativo:**
    - Controlla se la libreria richiesta è nello spazio di memoria di un altro processo.
    - Gestisce l'accesso protetto alla memoria condivisa.
    - Permette a più processi di usare la stessa area di memoria.
  - **Uso tipico:** librerie di sistema (**DLL** su Windows).
- 

## Overlay

- **Definizione:** tecnica che mantiene in memoria **solo** le istruzioni e i dati necessari in un determinato momento dell'esecuzione.
- **Quando serve:**
  - Quando la dimensione di un processo supera la quantità di memoria fisica allocata per esso.
- **Caratteristiche:**
  - Non richiede supporto speciale dal sistema operativo → può essere gestita direttamente dal programmatore.
  - La **struttura del programma** deve essere progettata con attenzione per permettere il caricamento e la sostituzione dinamica delle parti di codice/dati.
- **Svantaggio:**
  - Progettazione complessa, perché bisogna dividere manualmente il programma in segmenti caricabili separatamente.



## Swapping

Lo **swapping** è una tecnica di gestione della memoria in cui un processo può essere temporaneamente spostato dalla memoria centrale a un'area di memoria secondaria veloce (backing storage) per poi essere ricaricato quando deve riprendere l'esecuzione.

### Funzionamento di base

- Tipico negli algoritmi di schedulazione **round-robin**.
- L'area di memoria temporanea deve essere abbastanza capiente e fornire **accesso diretto** alle copie complete dei processi.
- Variante **roll-in/roll-out**: usata in schedulazioni a priorità → si rimuove un processo a bassa priorità per fare spazio a uno ad alta priorità.
- Di norma, un processo viene ricaricato **nella stessa posizione** di memoria, per evitare problemi di indirizzamento; con **loading dinamico**, invece, può essere ricaricato altrove perché gli indirizzi fisici sono calcolati durante l'esecuzione.

### Gestione operativa

- Il sistema mantiene una **lista dei processi pronti**, sia in RAM sia in backing storage.
- Se il processo scelto dal microscheduler non è in memoria, deve essere caricato dal disco:
  - Se manca spazio, un altro processo viene swappato fuori.
- Il cambio di contesto in questi casi è più lento; quindi, è efficiente solo se il **tempo di esecuzione** del processo è significativamente superiore al tempo di swap.
- Il tempo di swap dipende principalmente dal **tempo di trasferimento**, proporzionale alla quantità di memoria spostata.
- Per ottimizzare, è meglio swappare **solo la parte di memoria effettivamente usata** dal processo.

## Vincoli e attenzioni

- Un processo può essere swappato solo se **in stato di riposo**.
- Lo swap è problematico se c'è un'operazione di **I/O asincrona** in corso su buffer in memoria utente:
  - Rischio di corruzione dati se la memoria viene riassegnata a un altro processo.
  - Soluzioni: evitare lo swap durante I/O o eseguire I/O solo nei buffer del sistema operativo.

## Implementazione nei sistemi operativi

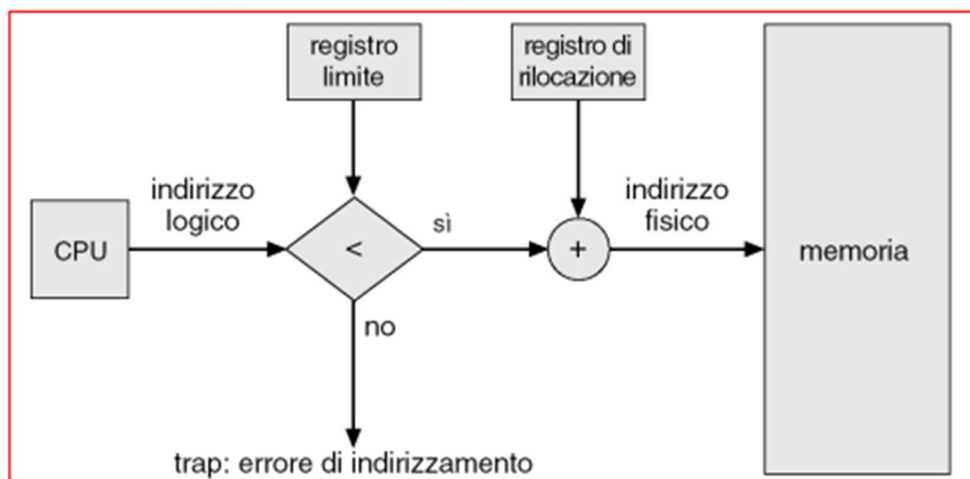
- Lo spazio di swap di solito è una **partizione dedicata** del disco, separata dal file system, per rendere l'accesso più veloce.
- Versioni modificate dello swapping sono presenti in UNIX, Linux e Windows.
- In genere è **disattivato** e si abilita automaticamente solo quando l'uso della memoria supera una certa soglia, per poi spegnersi quando il carico cala.

---

La memoria principale normalmente è divisa in due partizioni,

quella in cui risiede il sistema operativo, che di solito è collocato nella memoria bassa con il vettore di interrupt e i processi utente collocati nella memoria alta

nella partizione in cui vi sono i processi utente, può essere gestita a **partizione singola**: in cui vi sono due registri: **registro di locazione** che serve per proteggere il SO e i processi tra di loro, e contiene l'indirizzo fisico più piccolo accessibile dal processo e il **registro limite** che contiene l'intervallo degli indirizzi logici del processo. Ogni indirizzo logico deve avere un valore inferiore rispetto a quello del registro limite. Per ottenere l'indirizzo fisico se soddisfano i criteri di minoranza/uguaglianza, bisogna sommare l'indirizzo logico al registro di rilocalizzazione



Es.

PARTE 2/10

$(s, d)$   $s$  = indice ;  $d$  = offset se ho una tabella segment-base-length

offset < length  $\rightarrow$  indirizzo fis = base + offset

| segment | base | length | $(0, 610); (1, 70); (2, 260); (3, 360)$ |                                |
|---------|------|--------|-----------------------------------------|--------------------------------|
| 0       | 459  | 750    |                                         |                                |
| 1       | 3350 | 75     | $0 \rightarrow 610 < 750$ ✓             | indirizzo = $459 + 610 = 1149$ |
| 2       | 250  | 160    | $1 \rightarrow 70 < 75$ ✓               | // = $70 + 3350 = 3420$        |
| 3       | 1457 | 356    | $2 \rightarrow 260 < 160$ ✗             |                                |
|         |      |        | $3 \rightarrow 360 < 356$ ✗             |                                |

#### Partizioni multiple:

- La memoria è divisa in **più partizioni**:
  - Partizioni allocate**  $\rightarrow$  occupate dai processi.
  - Hole**  $\rightarrow$  blocchi liberi di varie dimensioni sparsi nella memoria centrale.
- Quando arriva un nuovo processo:
  - L'OS cerca un hole abbastanza grande.
  - Alloca il processo e aggiorna la lista di partizioni libere/occupate.
- Se non c'è un hole sufficientemente grande:
  - Aspetta la liberazione di memoria.
  - Oppure cerca un altro processo in coda che richieda meno spazio.

#### 4. Algoritmi di allocazione dinamica

##### 1. First-fit

- Alloca il **primo** hole grande abbastanza.
- Ricerca veloce (scansione fino al primo match).

##### 2. Best-fit

- Alloca il **più piccolo** hole sufficiente.
- Richiede la scansione completa (o lista ordinata per dimensione).

##### 3. Worst-fit

- Alloca il **più grande** hole disponibile.
- Anche qui ricerca completa o lista ordinata.

#### Prestazioni:

- First-fit**: più veloce del best-fit.
- First-fit** e **Best-fit**: simili in efficienza d'uso della memoria.



- **Worst-fit**: peggiore sia in tempo che in utilizzo della memoria.

## 1. Frammentazione Esterna

- La memoria totale libera è sufficiente, ma è divisa in blocchi piccoli e non contigui.
  - Di conseguenza, nessun blocco singolo è abbastanza grande per un processo grande.
  - Colpisce gli algoritmi **first-fit** e **best-fit**.
  - **Come ridurla**:
    - **Compattazione**: spostare i processi in memoria per creare un unico grande blocco libero.
    - La compactazione è possibile solo con **rilocalizzazione dinamica** (modifica del registro base e spostamento di codice e dati).
    - Oppure si evitano problemi di contiguità con tecniche come **paginazione** e **segmentazione**.
- 

## 2. Frammentazione Interna

- Si verifica quando la memoria assegnata a un processo è più grande di quella effettivamente richiesta.
- Lo spazio non usato all'interno della partizione è sprecato.
- Succede spesso in sistemi con partizioni o pagine di dimensioni fisse.

### Paginazione

La **paginazione** è una tecnica di gestione della memoria che consente di allocare i processi in memoria fisica in **blocchi non contigui**, semplificando l'uso dello spazio disponibile.

#### Principio di funzionamento

- La memoria fisica è divisa in **frame** (blocchi di dimensione fissa, tipicamente una potenza di 2 tra 512 B e 16 MB).
- La memoria logica (vista dal processo) è divisa in **pagine** di uguale dimensione ai frame.
- Per eseguire un processo di  $n$  pagine, il sistema trova  $n$  frame liberi e vi carica il contenuto.

#### Gestione degli indirizzi

- La dimensione della pagina è definita **dall'hardware**.
- Un indirizzo logico è diviso in:
  - **Numero di pagina** → identifica quale pagina logica vogliamo.
  - **Displacement (offset)** → indica la posizione all'interno della pagina.
-

- Se l'indirizzo logico è lungo  $m$  bit e la memoria fisica richiede  $n$  bit per l'indirizzamento, allora:
  - $m - n$  bit servono per il numero di pagina.
  - $n$  bit servono per il displacement.

Es.

Si calcoli, motivando la risposta, qual è il numero minimo di bit (numero di pagina e spostamento all'interno della pagina) per indirizzare una memoria virtuale paginata di 32 GB suddivisa in frame di 8 kB.

frame da 8kB =  $8 \cdot 1024 \text{ byte} \Rightarrow 8192 \text{ byte} \rightarrow \text{indirizzamento} \rightarrow \log_2(8192) = 13 \text{ bit} = d$

oppure  $1 \text{ kB} = 2^{10} \rightarrow 8 = 2^3 \rightarrow 2^3 \cdot 2^{10} = 2^{13}$  spostamento  $n$

memoria 32 GB  $\Rightarrow 1 \text{ GB} = 2^{30} \rightarrow 32 = 2^5 \rightarrow 32 \text{ GB} = 2^{35} \rightarrow 35 \text{ bit}$  grandezza indirizzo logico  $m$

|          |          |
|----------|----------|
| $p$      | $d$      |
| $2^{12}$ | $2^{13}$ |

$\Rightarrow \text{bit per } p \rightarrow 35 - 13 = 22 \text{ bit}$

$2^{35} =$

per calcolare la dimensione della PHT =  $\text{entry} \cdot \text{indirizzamento } p$

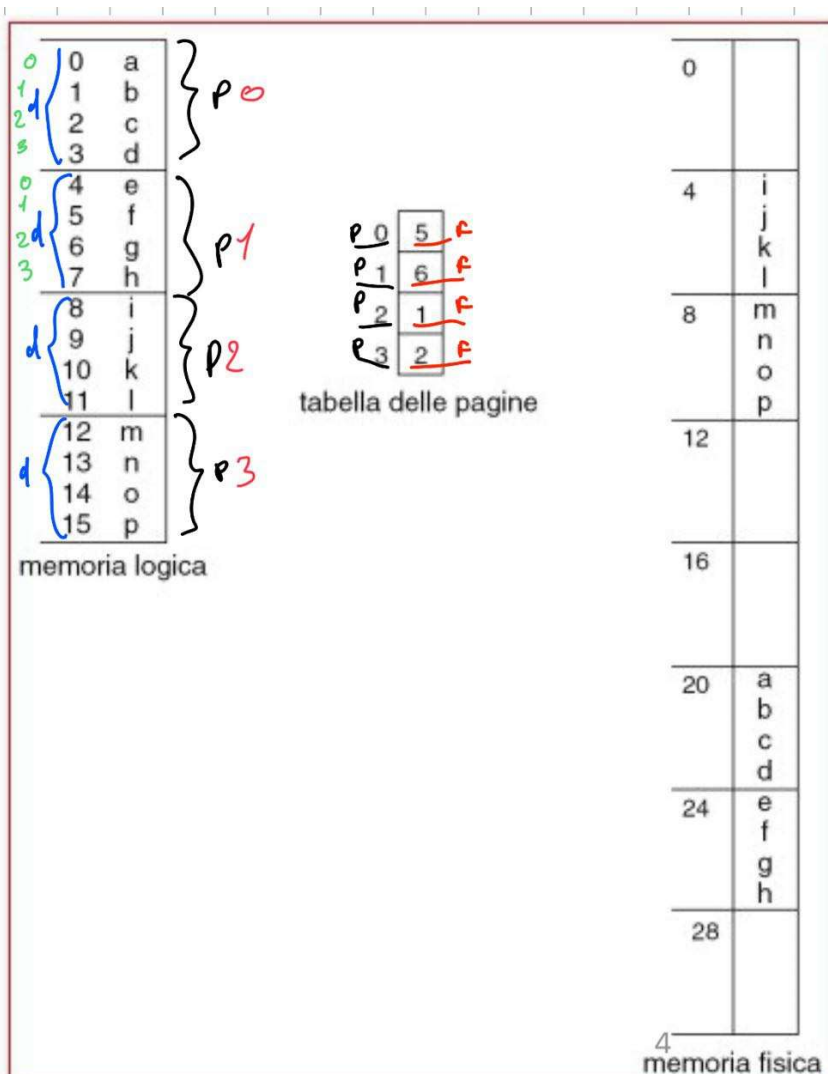
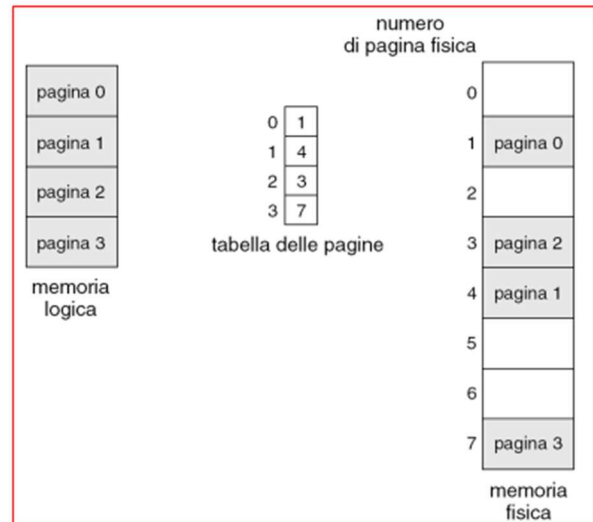
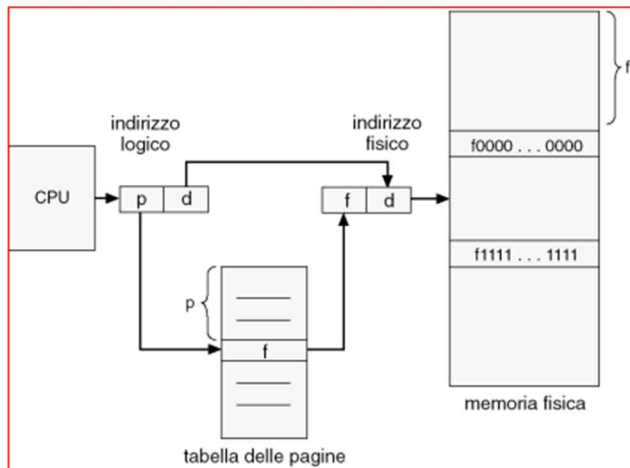
$\Rightarrow \text{PHT} = 2^{22} \cdot 2 = 2^{24} = 2^{20} \cdot 2^4 = 16 \text{ MB}$

### Traduzione degli indirizzi

- La conversione da indirizzo logico a fisico avviene tramite una **tabella delle pagine**:
  - Ogni voce della tabella indica in quale frame fisico si trova la pagina logica corrispondente.

Ogni indirizzo generato dalla CPU è diviso in due parti:

- Numero di pagina ( $p$ ) – usato come indice nella tabella delle pagine che contiene l'indirizzo di base di ogni frame nella memoria fisica.
- Spiazzamento nella pagina ( $d$ ) – combinato con l'indirizzo di base per calcolare l'indirizzo di memoria fisica che viene mandato all'unità di memoria centrale.



Indirizzo logico 0 = (p=0, d=0) P=0 -> F=5 -> indirizzo fisico =  $(5 * 4) + d = 20$

Indirizzo logico 3 = (0,3) P=0 -> F=5 -> indirizzo fisico =  $(5 * 4) + 3 = 23$

Indirizzo logico 13 = (3,1) P=3 -> F= 2 -> indirizzo fisico =  $(4 * 2) + 1 = 9$

## Frame liberi

La **paginazione** suddivide la memoria fisica in **frame** e usa un **registro di rilocazione** per ciascuno.

La **page table** funge da insieme di questi registri, indicando per ogni pagina logica in quale frame fisico si trova.

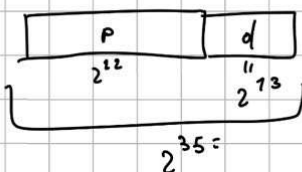
- Un processo che richiede  $n$  pagine **più un solo byte** deve comunque occupare  **$n+1$  frame**, causando nel peggiore dei casi **un frame quasi vuoto** → frammentazione interna.
- In media, la frammentazione interna equivale a **mezza pagina per processo**.
- **Pagine piccole** → meno spreco di spazio, ma:
  - aumentano le dimensioni della page table;
  - rendono gli accessi disco meno efficienti.
- **Dimensioni tipiche** delle pagine: **4 KB – 8 KB**.
- Un' **entry** contiene tutte le informazioni necessarie per tradurre un indirizzo logico in un indirizzo fisico.
- Ogni entry della page table è spesso di **4 byte**:
  - Con frame da 8 KB → fino a  **$2^{45}$  byte (32 TB)** di memoria fisica indirizzabile.

Si calcoli, motivando la risposta, qual è il numero minimo di bit (numero di pagina e scostamento all'interno della pagina) per indirizzare una memoria virtuale paginata di 32 GB suddivisa in frame di 8 kB.

frame da 8kB =  $8 \cdot 1024 \text{ byte} \Rightarrow 8192 \text{ byte} \rightarrow \text{indirizzamento} \rightarrow \log_2(8192) = 13 \text{ bit} = d$

oppure  $1 \text{ kB} = 2^{10}$  e  $8 = 2^3 \Rightarrow 2^3 \cdot 2^{10} = 2^{13}$  **scostamento**

memoria 32 GB  $\Rightarrow 1 \text{ GB} = 2^{30}$  e  $32 = 2^5 \Rightarrow 32 \text{ GB} = 2^{35}$   $\Rightarrow 35 \text{ bit}$  **grandezza indirizzo logico**



$\Rightarrow$  bit per  $p \Rightarrow 35 - 13 = 22 \text{ bit}$

per calcolare la

**dimensione della PHT = entry · indirizzamento  $p$**

$$\Rightarrow \text{PHT} = 2^{22} \cdot 2^2 = 2^{24} = 2^{10} \cdot 2^4 = 16 \text{ MB}$$

- L'utente vede un'unica memoria continua, ma in realtà le pagine sono sparse nella Memoria fisica insieme a quelle di altri processi.
- L'hardware di traduzione degli indirizzi riconcilia la visione logica con la disposizione reale.
- L'OS mantiene una **tabella dei frame** con:
  - numero totale di frame;
  - frame allocati;

- frame liberi.

## Page Table

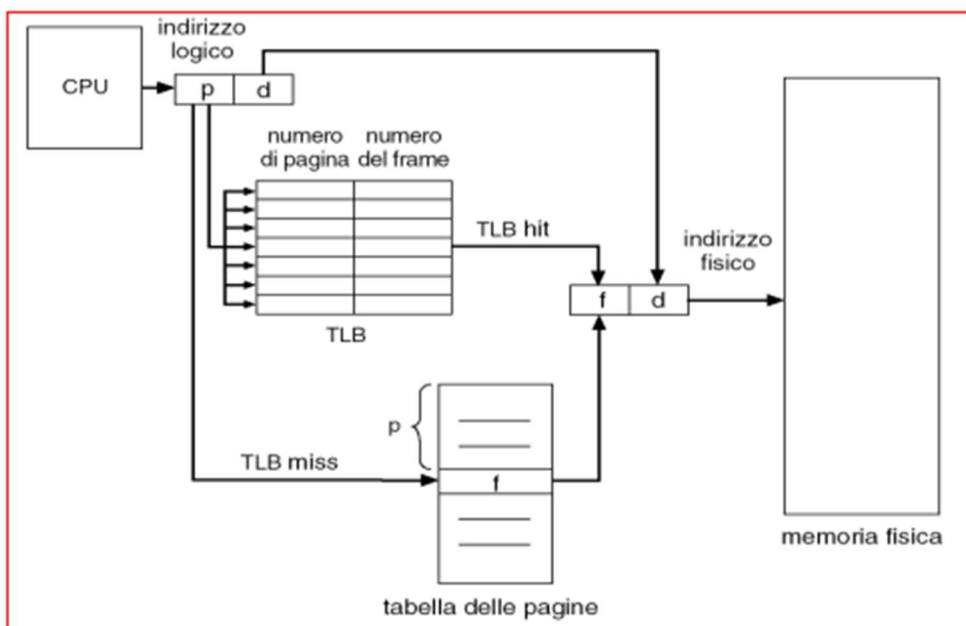
- Ogni processo ha **la propria page table**, con il puntatore memorizzato nel **PCB**.
- La tabella può stare:
  - in registri dedicati (se piccola);
  - in memoria centrale (più comune).
- Il **PTBR** (*Page Table Base Register*) punta alla page table del processo.
- Cambiare processo richiede solo aggiornare il PTBR → **context switch veloce**.
- Problema: ogni accesso a dati o istruzioni richiede **due accessi in memoria**:

1. Uno per leggere la page table.

2. Uno per i dati/istruzioni reali.

→ Risultato: **memoria più lenta di 2×**.

- **Soluzione:** usare un **Translation Lookaside Buffer (TLB)**:
  - Piccola cache ad alta velocità per tradurre rapidamente pagine logiche in frame fisici.
  - Dimensioni tipiche: **64 – 1024 voci**.
  - Funziona con **memoria associativa** → ricerca parallela:
    - Se il numero di pagina ( $p$ ) è nel TLB → si ottiene subito il frame.
    - Se non è nel TLB → si consulta la page table in memoria centrale.



## TLB (Translation Lookaside Buffer)

- **Scopo:** velocizzare la traduzione degli indirizzi logici in fisici riducendo il numero di accessi alla memoria.
- **Vantaggio:** con un buon tasso di hit, può richiedere **meno del 10% del tempo** rispetto alla memoria non mappata.
- **Politiche di sostituzione** (quando il TLB è pieno):
  - **LRU** (Least Recently Used).
  - **Random**.
- Alcune voci del TLB sono **wired down** → non possono essere rimosse (ad esempio codice del kernel).

## ESERCIZI TEMPO DI EAT

### Calcolo del Tempo di Accesso Effettivo (EAT)

- **Parametri:**
  - $\epsilon$ : tempo per l'accesso associativo (lookup nel TLB).
  - $\beta$ : tempo di ciclo della memoria.
  - $\alpha$ : **hit ratio** (probabilità che la pagina richiesta sia nel TLB).
- **Formula generale:**

$$EAT = (\beta + \epsilon) \cdot \alpha + (2\beta + \epsilon) \cdot (1 - \alpha)$$

che si può semplificare in:

$$EAT = \epsilon(2 - \alpha) + \beta$$

Es.

$$T_{\text{test}} = \beta(2 - \alpha) + \epsilon$$

} de Hit ratio = 100%  $\rightarrow T_{\text{test}} = \beta + \epsilon \rightarrow \alpha = 1$   
 } de Hit ratio = 0%  $\rightarrow T_{\text{test}} = 2\beta + \epsilon \rightarrow \alpha = 0$

$\epsilon = 20 \text{ ns}$  ,  $\beta = 100 \text{ ns}$  ,  $\alpha = 80\% \rightarrow T_{\text{test}} = 100 \cdot (2 - 0,8) + 20 = 140 \text{ ns}$

---

Es 2  $\beta = 100 \text{ ns} \rightarrow$  eccesso alla TLB  $\rightarrow \epsilon = 3 \text{ ns} \rightarrow \alpha = 94\%$ .  
 1.  $T_{\text{test}}$   
 2.  $\alpha$  con un degrado del 5%.

$T_{\text{test}} = \beta(2 - \alpha) + \epsilon \Rightarrow 100(2 - 0,94) + 3 = 109 \text{ ns}$

$T_{\text{test}}' = T_{\text{test}} \cdot (1 - 0,05) = 114,45 \text{ ns} \rightarrow T' = \beta(2 - \alpha') + \epsilon \rightarrow \alpha' = 2 - \frac{T' - \epsilon}{\beta} = 88,55\%$

$\rightarrow$  DEGRADO

### Protezione della memoria centrale in ambiente paginato

In un sistema con paginazione, la protezione si ottiene **associando bit di protezione** a ogni pagina nella PMT.

- **Bit di protezione:** indicano se la pagina è:
  - **Sola lettura** (read-only)  $\rightarrow$  scrivere genera una *trap* hardware (interruzione).
  - **Lettura/Scrittura** (read-write).
- **Bit valid/invalid:**
  - **Valid:** la pagina fa parte dello spazio di indirizzi del processo  $\rightarrow$  accesso consentito.
  - **Invalid:** la pagina non appartiene al processo  $\rightarrow$  accesso negato.

Questo è utile perché spesso un processo **non usa tutto lo spazio di indirizzamento** disponibile.

### Protezione della memoria centrale in ambiente paginato

In un sistema con paginazione, la protezione si ottiene **associando bit di protezione** a ogni pagina nella PMT.

- **Bit di protezione:** indicano se la pagina è:
  - **Sola lettura** (read-only)  $\rightarrow$  scrivere genera una *trap* hardware (interruzione).
  - **Lettura/Scrittura** (read-write).
- **Bit valid/invalid:**
  - **Valid:** la pagina fa parte dello spazio di indirizzi del processo  $\rightarrow$  accesso consentito.
  - **Invalid:** la pagina non appartiene al processo  $\rightarrow$  accesso negato.

Questo è utile perché spesso un processo **non usa tutto lo spazio di indirizzamento** disponibile.

---

### PTLR – Page Table Length Register

- Indica quante voci (pagine) contiene realmente la PMT di un processo.
  - Ad ogni accesso alla memoria, il sistema controlla che il numero di pagina richiesto sia **minore del PTLR** → se no, l'accesso è illegale.
- 

### Esempio

- Spazio di indirizzamento: **14 bit** → indirizzi 0–16383. Ogni indirizzo è per un byte
- Programma che usa solo indirizzi **0–10468**. 10KB
- Dimensione pagina: **2 KB (2048 byte)** → servono **6 pagine** (0–5).
  - $6 \times 2048 = 12288$  byte totali allocati (di cui parte inutilizzata → frammentazione interna).
- Le pagine **6** e **7** sono **invalid**.
- L'ultima pagina (pagina 5) va da indirizzo **10240** a **12287**, ma il programma in realtà si ferma a **10468** → lo spazio tra 10469 e 12287 è **sprecato**.

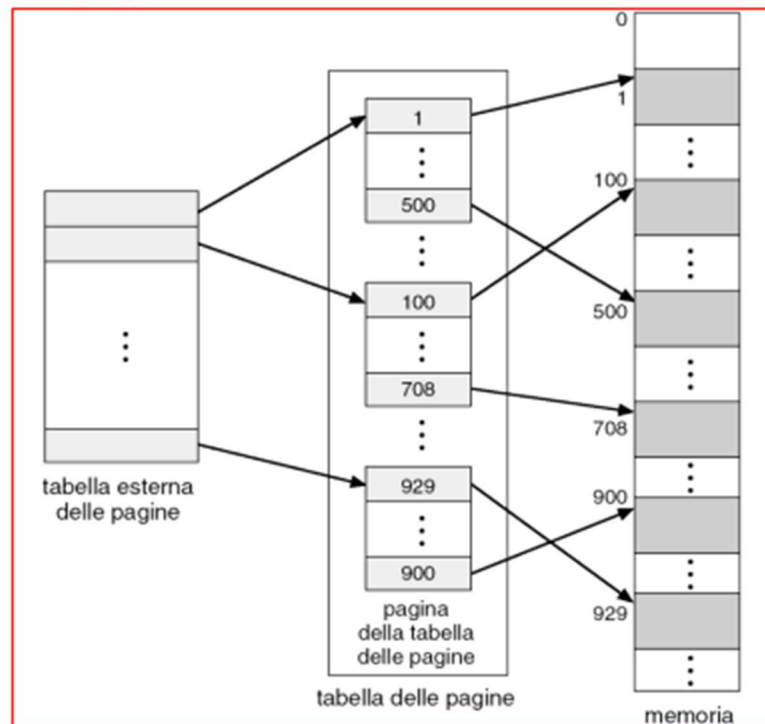
Es2

### TECNICHE EFFICIENTI PER LA STRUTTURAZIONE DELLA TABELLE DELLE PAGINE

#### 1. Paginazione gerarchica

- a. Suddivide lo spazio degli indirizzi logici in più tabelle di pagine
- b. Paginazione a due livelli:
  - i. La PMT è paginata e servirà una external PMT che contiene il riferimento alle pagine della PMT





ii.

## 2. Tabelle delle pagine con hashing

- Utile per trattare spazi di indirizzamento più grandi di 32 bit Il numero di pagina virtuale viene passato a una funzione di hashing per ottenere un indice in una tabella ridotta. Ogni voce contiene:
  - Numero di pagina virtuale.
  - Numero del frame fisico corrispondente.
  - Puntatore al prossimo elemento (per gestire collisioni in liste collegate).
 Si cerca nella lista finché non si trova la corrispondenza; il frame trovato, sommato all'offset, dà l'indirizzo fisico.  
 Variante: **clustered page table**, in cui ogni voce si riferisce a più pagine contigue.

## 3. Tabella delle pagine invertita

- Un'unica PMT per l'intero sistema, con un elemento per ogni pagina fisica (non per ogni pagina logica di ogni processo).  
 Ogni voce contiene:
  - Indirizzo virtuale della pagina logica memorizzata.
  - Info sul processo proprietario.
 Vantaggio: riduce l'uso di memoria per la tabella.  
 Svantaggio: la ricerca è più lenta; perciò, si usa una hash table per velocizzarla.

## 4. Pagine condivise

- La paginazione consente di condividere **codice comune** (es. compilatori, librerie) tra processi tramite una copia di sola lettura, posizionata nello stesso punto dello spazio logico di tutti.
- Codice privato e dati:** ogni processo ha la propria copia, posizionabile ovunque nello spazio logico.
- Condivisione realizzata mappando più indirizzi virtuali a un unico indirizzo fisico.
- Nei sistemi con **tabella delle pagine invertita** la condivisione non è possibile, perché c'è una sola pagina virtuale per pagina fisica.

## 5. Segmentazione

La segmentazione è uno **schema di gestione della memoria centrale** che riflette il punto di vista dell'utente.

- Lo spazio di indirizzamento logico è diviso in **segmenti**, ciascuno dei quali è un'unità logica ben definita (es. programma principale, funzioni, metodi, oggetti, variabili locali, stack, array).
- Ogni segmento ha **lunghezza variabile** e rappresenta una parte coerente del programma o dei dati.

### Architettura della segmentazione

- Un indirizzo logico è composto da due parti:
  1. **Numero del segmento** (identifica quale segmento si vuole usare).
  2. **Spiazzamento** (offset, indica la posizione all'interno di quel segmento).
- Gli indirizzi logici vengono tradotti in indirizzi fisici tramite la **Tabella dei Segmenti**.
  - Ogni voce della tabella contiene:
    - **Base**: indirizzo fisico iniziale del segmento in memoria centrale.
    - **Limite**: lunghezza del segmento (serve per controllare che lo spiazzamento sia valido).

### Registri usati

- **STBR** (*Segment Table Base Register*): contiene l'indirizzo fisico in cui si trova la tabella dei segmenti del processo corrente.
- **STLR** (*Segment Table Length Register*): indica quanti segmenti il programma sta effettivamente usando.
  - Un numero di segmento  $s$  è considerato valido se  $s < \text{STLR}$ .

### Vantaggi principali

- Rappresenta la memoria in modo logico e comprensibile per l'utente/programmatore.
- Permette protezione e condivisione a livello di segmento.

## 6. Segmentazione paginata

---

### Frammentazione nella Segmentazione

- Poiché i **segmenti** sono di **lunghezza variabile**, l'allocazione della memoria è un **problema di allocazione dinamica**.
- Metodi comuni di allocazione:
  - **First fit**: si assegna il primo blocco abbastanza grande.
  - **Best fit**: si assegna il blocco più piccolo che può contenere il segmento.

- **Problema:** si verifica **frammentazione esterna**, cioè la memoria libera si trova in blocchi sparsi e troppo piccoli per essere usati.
  - Soluzioni:
    - Aspettare la liberazione di blocchi.
    - **Compattazione:** spostare i segmenti per unire lo spazio libero (operazione costosa).
  - **Segmentazione = algoritmo di rilocalizzazione dinamica:** in teoria si potrebbe mettere ogni byte in un segmento separato, eliminando la frammentazione esterna.
    - Ma servirebbe un registro base per ogni byte → **raddoppio dell'uso di memoria**.
  - Se i segmenti sono **mediamente piccoli**, la frammentazione esterna sarà bassa.
- 

### Segmentazione paginata

Sistema **MULTICS**: combina **segmentazione** e **paginazione** per:

- Eliminare la frammentazione esterna.
- Ridurre i tempi di ricerca.

#### Funzionamento:

- Nella **segmentazione con paginazione**, la tabella dei segmenti **non contiene l'indirizzo fisico del segmento**, ma **l'indirizzo della tabella delle pagine** associata a quel segmento.
- Lo spazio di indirizzamento logico di un processo è diviso in:
  1. **Segmenti riservati** → informazioni nella **Local Descriptor Table (LDT)**.
  2. **Segmenti condivisi** → informazioni nella **Global Descriptor Table (GDT)**.

#### Descrittore di segmento:

- Contiene informazioni dettagliate sul segmento, tra cui **BASE** e **LIMITE**.

#### Formato indirizzo logico:

- Coppia <selettore, spiazzamento>:
  - **Selettore:** 16 bit → <s, g, p>
    - s (13 bit): numero di segmento.
    - g (1 bit): indica se il segmento è nella GDT o nella LDT.
    - p (2 bit): protezione.
  - **Spiazzamento:** 32 bit, posizione della word nel segmento.

#### Traduzione:

1. Controllo del **LIMITE**: se valido, si somma lo spiazzamento al valore **BASE** → si ottiene **l'indirizzo lineare**.
2. L'indirizzo lineare è tradotto con **paginazione a due livelli**:

- Puntatore all'indice di pagina.
  - Puntatore alla **Page Map Table (PMT)**.
  - Spiazzamento nella pagina.
- 

## CAPITOLO 10

### MEMORIA VIRTUALE

Comporta la separazione della memoria logica dalla memoria fisica. Poiché solo una parte del programma necessita di essere in memoria per l'esecuzione, lo spazio di indirizzamento logico può essere più grande di quello fisico, e dunque più programmi possono essere in esecuzione siccome usano poca memoria fisica, così facendo gli spazi di indirizzamento sono condivisi da numerosi processi

#### Richiesta di Paginazione (Demand Paging)

##### Concetto base

- **Memoria virtuale:** le pagine vengono caricate in memoria **solo quando servono** (non tutte in anticipo).
  - Se un processo non usa mai certe pagine → non vengono mai caricate → risparmio di risorse.
- 

##### Vantaggi

- **Meno I/O:** si leggono meno pagine dal disco.
  - **Meno RAM necessaria:** solo le pagine realmente usate occupano memoria.
  - **Tempi di risposta migliori:** il processo può iniziare anche se non tutte le pagine sono in RAM.
  - **Supporto a più utenti:** più processi contemporanei grazie al risparmio di memoria.
- 

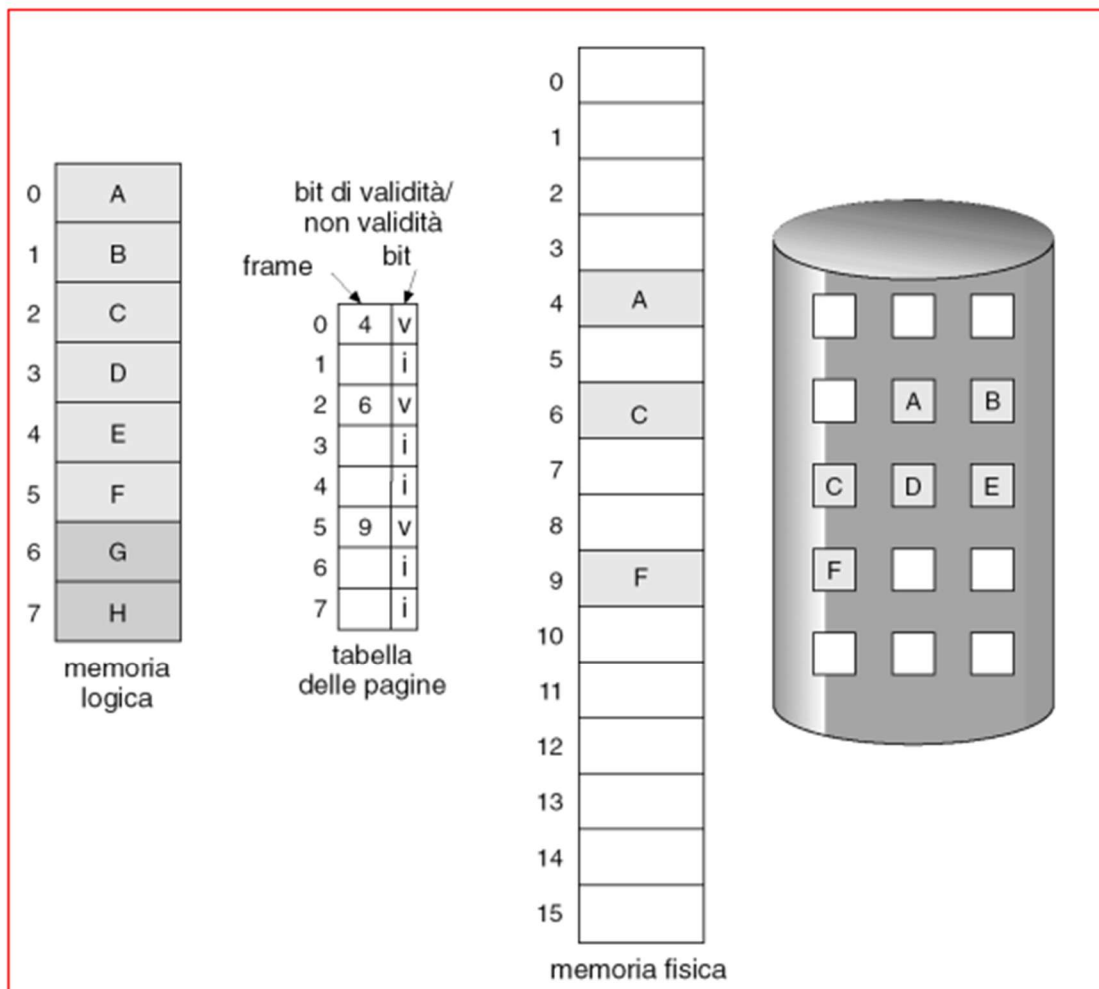
##### Come funziona

1. **Accesso a una pagina** → si controlla se è in memoria.
2. **Caso 1:** riferimento **non valido** (indirizzo sbagliato) → il processo viene terminato (errore grave).
3. **Caso 2:** pagina **non in memoria** (page fault) → la si carica dalla memoria secondaria (disco) nella RAM.

Inoltre si usano i Bit valido\_ non valido che si aggiungono ad ogni riga della tabella e dicono:

1. 1 se è valido
2. 0 se non è valido

Di default è impostato su 0 per ogni elemento



Se il processo accede ad una pagina non valida, si attiva una trap di mancanza di pagina,

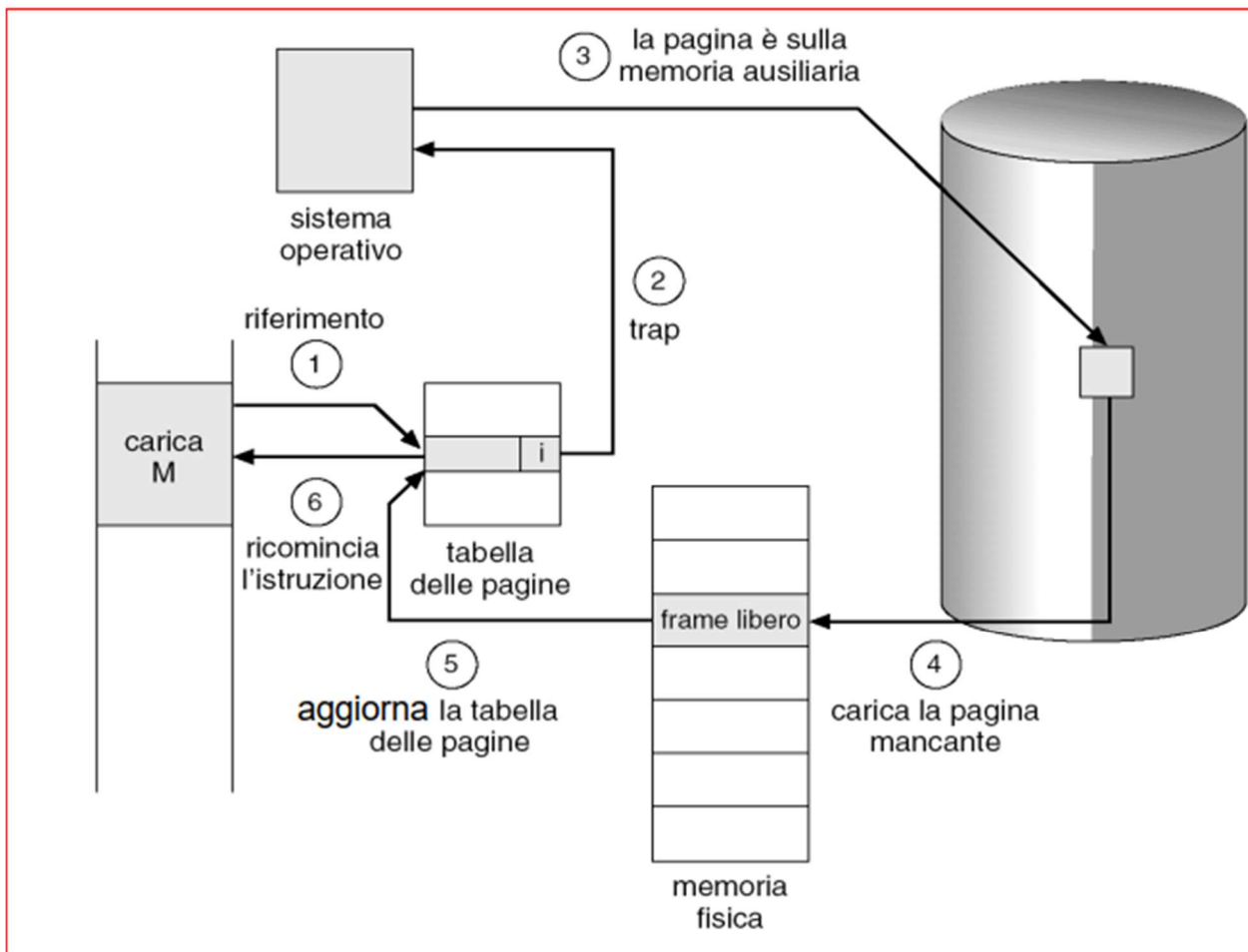
### Definizione

- Avviene quando un processo tenta di accedere a una pagina **marcata come non valida** nella tabella delle pagine.
- Genera una **trap** (interruzione gestita dal SO).

### Gestione del Page Fault

1. **SO controlla un'altra tabella** per capire il motivo:
  - **Indirizzo non valido** → processo terminato (errore di memoria).
  - **Pagina non in memoria ma valida** → procedere al caricamento.
2. **Trovare un frame libero** in RAM.
3. **Caricare la pagina** dal disco al frame.
4. **Aggiornare la tabella delle pagine:**
  - Bit di validità → 1 (pagina ora in memoria).
5. **Riprendere l'istruzione interrotta:**

- Possibile sia un'operazione complessa come **block move** o **auto increment/decrement location**, che il SO deve far ripartire correttamente.



**Se non ci sono frame liberi → Sostituzione di Pagina**

**Concetto**

- Quando la RAM è piena e arriva un **page fault**, il SO deve **scegliere una pagina in memoria da rimuovere** (che non sia attualmente in uso) per fare spazio alla nuova.
- La pagina rimossa può essere ricaricata più volte in futuro se servirà di nuovo.

**Passaggi generali**

1. **Identificare una pagina "non in uso"** (secondo un criterio deciso dall'algoritmo).
2. **Liberare il frame:**
  - Se la pagina è stata modificata → scriverla su disco.
  - Se è "pulita" (unchanged) → semplicemente rimuoverla.
3. **Caricare la nuova pagina** nel frame appena liberato.
4. **Aggiornare la tabella delle pagine** (bit validità e info di frame).
5. **Riprendere l'esecuzione** del processo.

---

### Requisiti dell'algoritmo di sostituzione

- **Prestazioni:** minimizzare il numero di **page fault futuri**.
- Esempi di algoritmi (verranno studiati più avanti): FIFO, LRU, Optimal, Clock, ecc.

Oltre a questi benefici la memoria virtuale permette altri benefici:

- **Copia durante la scrittura**

#### Concetto

- Quando un processo padre crea un processo figlio (fork), inizialmente condividono le stesse pagine in RAM.
- Le pagine condivise sono marcate come sola lettura.

#### Funzionamento

1. Se uno dei due scrive su una pagina condivisa → si genera un page fault.
2. Il SO crea una copia privata della pagina per il processo che scrive.
3. Così si copiano solo le pagine effettivamente modificate, non tutte.

#### Vantaggi

- Risparmio di memoria.
- Miglior tempo di creazione del processo.
- Pagine libere prese da un pool inizializzato a zero (sicurezza e pulizia dati).
- **File mappati in memoria (Memory-Mapped Files)**

#### Concetto

- Trattare l'I/O su file come accesso alla memoria:
  - Un blocco del file su disco viene mappato su una pagina di memoria fisica.
  - Si accede al file leggendo/scrivendo quella pagina in RAM.

#### Funzionamento

1. Primo accesso → page fault → il blocco del file viene letto dal disco in RAM.
2. Accessi successivi → avvengono direttamente in memoria (molto più veloci).
3. Alcuni OS richiedono chiamate di sistema speciali (mmap(), read(), write()), altri gestiscono la mappatura trasparentemente.

## Condivisione

- Più processi possono mappare lo stesso file contemporaneamente → condivisione dati in RAM.

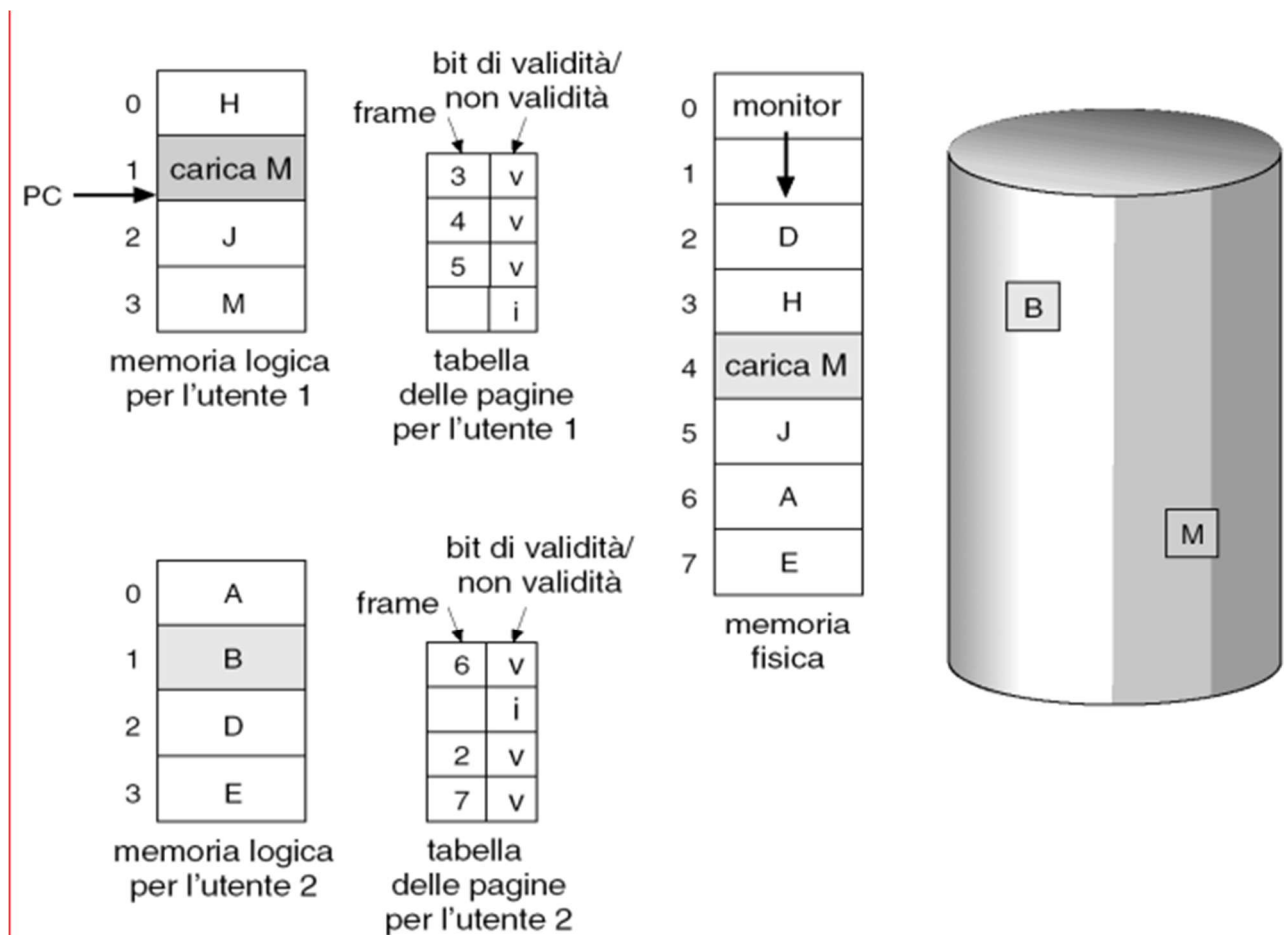
## SOSTITUZIONE DELLA PAGINA

Per evitare le sovra-allocazione della memoria se avviene una page fault, bisogna implementare negli algoritmi di mancanza di pagina, la sostituzione di pagina

usare un bit di modifica

## Ottimizzazione

- **Modify bit (dirty bit):**
  - Indica se la pagina è stata **modificata** in RAM.
  - Solo le pagine modificate vengono **riscritte su disco** → riduzione del tempo e del carico I/O.



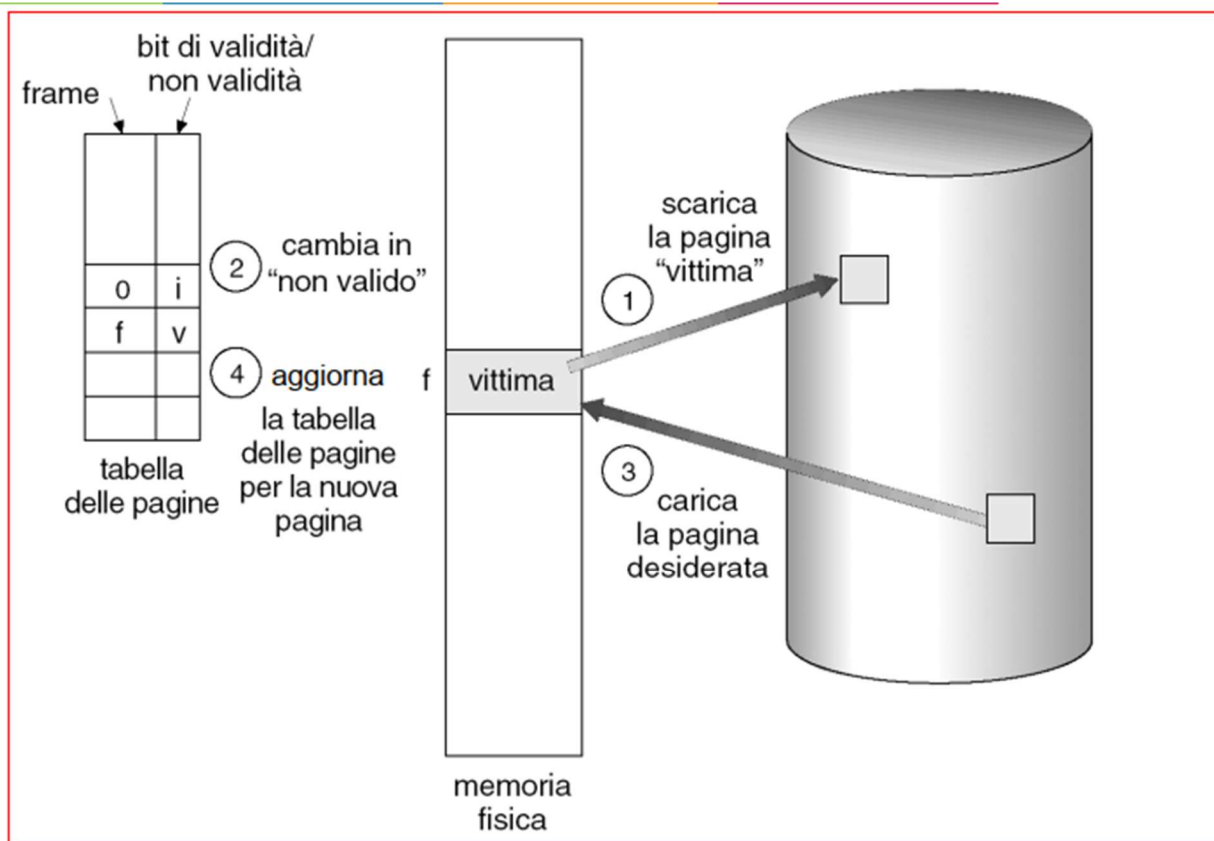
Questa modalità di sostituzione permette una grande memoria virtuale anche se in presenza di poca memoria fisica

L'algoritmo alla base è il seguente

7. TROVARE LA POSIZIONE DESIDERATA SUL DISCO
8. TROVARE UN FRAME LIBERO
  - a. Se non è presente usare l'algoritmo di sostituzione
9. LEGGERE LA PAGINA DESIDERATA NEL FRAME LIBERO



## 10. RIPRENDERE IL PROCESSO



Obiettivo è quello di avere il più basso tasso di mancanza di pagine (page fault)

In tutti i nostri esempi la stringa di riferimento è:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.

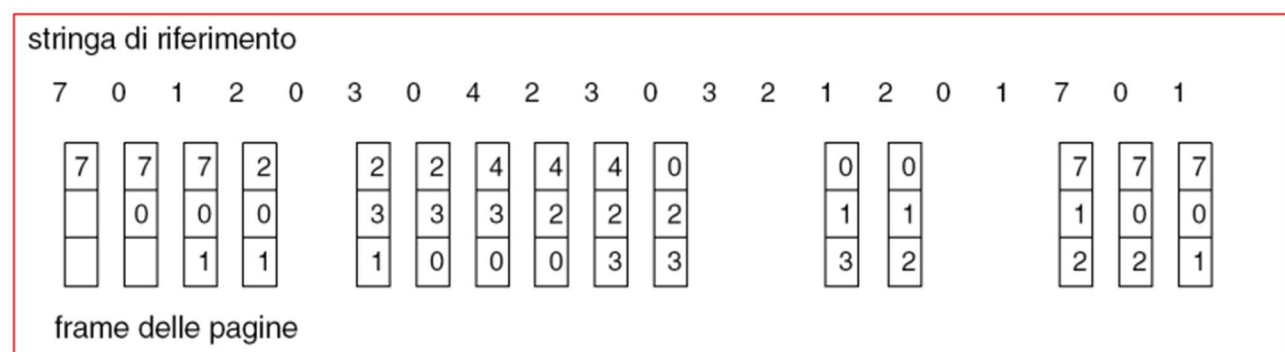
### **ALGORITMO FIFO**

Rimuove la pagina più vecchia ( la prima entrata in memoria) usando code circolari, questo può portare l'anomalia di belady ( più frame = più page fault)

| Passo | Riferimento | Stato dei frame (da più vecchio → più nuovo) | Page Fault? | Sostituzione |
|-------|-------------|----------------------------------------------|-------------|--------------|
| 1     | 1           | 1                                            | ✓           | —            |
| 2     | 2           | 1, 2                                         | ✓           | —            |
| 3     | 3           | 1, 2, 3                                      | ✓           | —            |
| 4     | 4           | 4, 2, 3                                      | ✓           | 1 → out      |
| 5     | 1           | 4, 1, 3                                      | ✓           | 2 → out      |
| 6     | 2           | 4, 1, 2                                      | ✓           | 3 → out      |
| 7     | 5           | 5, 1, 2                                      | ✓           | 4 → out      |

| Passo | Riferimento | Stato dei frame (da più vecchio → più nuovo) | Page Fault? | Sostituzione |
|-------|-------------|----------------------------------------------|-------------|--------------|
| 8     | 1           | 5, 1, 2                                      | ×           | —            |
| 9     | 2           | 5, 1, 2                                      | ×           | —            |
| 10    | 3           | 3, 1, 2                                      | ✓           | 5 → out      |
| 11    | 4           | 4, 1, 2                                      | ✓           | 1 → out      |
| 12    | 5           | 5, 4, 2                                      | ✓           | 2 → out      |

## 10 PAGE FAULT



## ALGORITMO OTTIMALE

Si conosce già in partenza la sequenza che deve entrare; infatti, si basa su questa

Regola: quando serve spazio, si sostituisce la **pagina il cui prossimo uso è più lontano nel futuro** (o che **non verrà più usata**).

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.

| Passo | Riferimento | Frame (1-4)  | Page fault? | Sostituita | Note sulla scelta |
|-------|-------------|--------------|-------------|------------|-------------------|
| 1     | 1           | [1, -, -, -] | ✓           | —          | riempimento       |
| 2     | 2           | [1, 2, -, -] | ✓           | —          | riempimento       |
| 3     | 3           | [1, 2, 3, -] | ✓           | —          | riempimento       |
| 4     | 4           | [1, 2, 3, 4] | ✓           | —          | riempimento       |
| 5     | 1           | [1, 2, 3, 4] | ×           | —          | hit               |
| 6     | 2           | [1, 2, 3, 4] | ×           | —          | hit               |

| Passo | Riferimento | Frame (1-4)  | Page fault? | Sostituita | Note sulla scelta                                          |
|-------|-------------|--------------|-------------|------------|------------------------------------------------------------|
| 7     | 5           | [1, 2, 3, 5] | ✓           | 4          | tra {1,2,3,4} il più lontano è 4 (prossimo uso a passo 11) |
| 8     | 1           | [1, 2, 3, 5] | ✗           | —          | hit                                                        |
| 9     | 2           | [1, 2, 3, 5] | ✗           | —          | hit                                                        |
| 10    | 3           | [1, 2, 3, 5] | ✗           | —          | hit                                                        |
| 11    | 4           | [4, 2, 3, 5] | ✓           | 1          | tra {1,2,3,5} la 1 non verrà più usata → via lei           |
| 12    | 5           | [4, 2, 3, 5] | ✗           | —          | hit                                                        |

## 6 PAGE FAULT

### ALGORITMO LRU – LEAST RECENTLY USED

- Evict (sostituire) la pagina **usata meno di recente**.
- Intuizione: se non è stata usata da tanto, probabilmente non servirà a breve.

### Come si implementa (concetto)

- Tenere il “tempo di ultimo accesso” per ogni pagina e rimuovere quella con timestamp più vecchio.
- Implementazioni pratiche: lista con aggiornamento al front (stack), contatori, o hardware/bit di riferimento + algoritmo *Clock* come approssimazione.

### Passo Riferimento Frame (ordine fisso 1→3) Fault? Sostituita Motivazione sostituzione

|    |   |           |   |   |                  |
|----|---|-----------|---|---|------------------|
| 1  | 1 | [1, -, -] | ✓ | — | frame vuoto      |
| 2  | 2 | [1, 2, -] | ✓ | — | frame vuoto      |
| 3  | 3 | [1, 2, 3] | ✓ | — | frame vuoto      |
| 4  | 4 | [4, 2, 3] | ✓ | 1 | 1 = meno recente |
| 5  | 1 | [4, 1, 3] | ✓ | 2 | 2 = meno recente |
| 6  | 2 | [4, 1, 2] | ✓ | 3 | 3 = meno recente |
| 7  | 5 | [5, 1, 2] | ✓ | 4 | 4 = meno recente |
| 8  | 1 | [5, 1, 2] | ✗ | — | già presente     |
| 9  | 2 | [5, 1, 2] | ✗ | — | già presente     |
| 10 | 3 | [3, 1, 2] | ✓ | 5 | 5 = meno recente |
| 11 | 4 | [3, 4, 2] | ✓ | 1 | 1 = meno recente |

### Passo Riferimento Frame (ordine fisso 1→3) Fault? Sostituita Motivazione sostituzione

12    5            [3, 4, 5]                2            2 = meno recente

Totale page fault: 10

Totale hit: 2 (passi 8 e 9)

### LRU CON BIT

Implementare un comportamento **simile a LRU** senza dover registrare il “tempo” esatto di ogni accesso, che sarebbe troppo costoso.

---

### ✂ Il Bit di Riferimento (Reference Bit)

#### Definizione

- Ogni pagina in memoria ha associato **un bit** che indica se **è stata usata di recente**, inizialmente = 0.
- Questo bit è mantenuto dall’hardware di gestione della memoria (MMU) o dal sistema operativo, e può essere azzerato periodicamente.

---

#### Regole di funzionamento

1. **All’accesso di una pagina** (lettura o scrittura):
  - Il **bit di riferimento** di quella pagina viene impostato a 1.
2. **Quando serve scegliere una pagina da rimpiazzare:**
  - Si scorre l’elenco delle pagine in memoria.
  - Si cercano quelle con bit 0 (non usate di recente).
  - La prima trovata viene sostituita.
3. **Se tutte hanno bit = 1:**
  - Si azzerano tutti i bit a 0 e si ripete la scansione.
4. **Clock / Second Chance** (versione ottimizzata):
  - Le pagine sono organizzate come in un anello (clock).
  - Un “puntatore” indica la prossima candidata alla sostituzione.
  - Se la pagina trovata ha bit = 1 → gli si dà una “seconda chance” (bit = 0 e puntatore avanza).
  - Se la pagina ha bit = 0 → viene sostituita.

---

#### Vantaggi

-  Molto meno costoso di LRU “puro” (non serve tracciare l’ordine preciso di accesso).
-  Buona approssimazione di LRU.

-  Semplice da implementare a livello hardware/software.

### ⚠ Svantaggi

-  Non è LRU perfetto: può sostituire pagine che in realtà sarebbero state usate a breve.
-  Efficienza dipende da quanto spesso si azzerano i bit e da come si gestisce il puntatore.

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Prima una rapida ricorrenza delle **regole** (riassunto veloce):

- Ogni frame ha un **bit di riferimento** (R), impostato a 1 ad ogni accesso (read o write).
- C'è un **puntatore** (hand) che indica la prossima candidata alla sostituzione.
- Quando serve rimpiazzare:
  - Se il frame puntato ha R = 0 → lo rimpiazziamo (victim).
  - Se R = 1 → imposti R = 0 (gli dai una *seconda chance*) e avanzi il puntatore; ripeti finché trovi un 0.
- All'inserimento in frame vuoto si mette la pagina e si imposta R = 1; il puntatore avanza al frame successivo.

| Passo | Riferimento | Puntatore (prima) | Frame (dopo accesso)  | Fault?                                                                                    | Azione / Note                                                                                                       |
|-------|-------------|-------------------|-----------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1     | 1           | 1                 | [(1,1), (-,-), (-,-)] |        | caricato in frame1 (vuoto). puntatore→2                                                                             |
| 2     | 2           | 2                 | [(1,1), (2,1), (-,-)] |        | caricato in frame2. puntatore→3                                                                                     |
| 3     | 3           | 3                 | [(1,1), (2,1), (3,1)] |        | caricato in frame3. puntatore→1                                                                                     |
| 4     | 4           | 1                 | [(4,1), (2,0), (3,0)] |        | hand=1: R=1→0,avanza; hand=2: R=1→0,avanza; hand=3: R=1→0,avanza; poi hand=1 R=0 → sostituisci 1 con 4; puntatore→2 |
| 5     | 1           | 2                 | [(4,1), (1,1), (3,0)] |        | frame2 (R=0) scelto → replace 2 con 1; puntatore→3                                                                  |
| 6     | 2           | 3                 | [(4,1), (1,1), (2,1)] |        | frame3 (R=0) scelto → replace 3 con 2; puntatore→1                                                                  |
| 7     | 5           | 1                 | [(5,1), (1,0), (2,0)] |        | hand cicla: f1 R=1→0; f2 R=1→0; f3 R=1→0; torna f1 R=0 → replace 4 con 5; puntatore→2                               |
| 8     | 1           | 2                 | [(5,1), (1,1), (2,0)] |  (HIT) | 1 già in frame2 → set R=1                                                                                           |

| Passo | Riferimento | Puntatore (prima) | Frame (dopo accesso)  | Fault?     | Azione / Note                                                                                               |
|-------|-------------|-------------------|-----------------------|------------|-------------------------------------------------------------------------------------------------------------|
| 9     | 2           | 2                 | [(5,1), (1,1), (2,1)] | ✗<br>(HIT) | 2 già in frame3 → set R=1                                                                                   |
| 10    | 3           | 2                 | [(5,0), (3,1), (2,0)] | ✓          | hand=2: R=1→0,avanza; hand=3: R=1→0,avanza; hand=1: R=1→0,avanza; hand=2 R=0 → replace 1 con 3; puntatore→3 |
| 11    | 4           | 3                 | [(5,0), (3,1), (4,1)] | ✓          | hand=3 R=0 → replace 2 con 4; puntatore→1                                                                   |
| 12    | 5           | 1                 | [(5,1), (3,1), (4,1)] | ✗<br>(HIT) | 5 già in frame1 (R=0 → set R=1)                                                                             |

## Algoritmo di conteggio – concetti

### 1 LFU (Least Frequently Used)

- Tiene un **conteggio di accessi per ogni pagina**.
- Quando serve sostituire una pagina, si sceglie **quella con il conteggio più basso**.
- Idea: le pagine usate raramente probabilmente non serviranno a breve.

#### Regole

- All'accesso a una pagina già in memoria → incremento del contatore.
- Quando serve rimpiazzare → scegli pagina con contatore più basso.
- In caso di parità → si può usare FIFO come tie-breaker.

### 2 MFU (Most Frequently Used)

- Paradossalmente sostituisce **la pagina con il conteggio più alto**.
- Idea: la pagina con molti accessi probabilmente **ha già terminato la sua utilità immediata**.
- La pagina nuova (conteggio basso) ha più probabilità di essere utilizzata a breve.

## ✂ Esempio pratico (3 frame)

**Sequenza:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

**Inizializzazione:** tutti i frame vuoti, contatore iniziale = 0.

## LFU – 3 frame

| Passo Riferimento |   | Frame (pagina, conteggio) | Fault? Azione / Note                                                       |
|-------------------|---|---------------------------|----------------------------------------------------------------------------|
| 1                 | 1 | [(1,1), -, -]             | ✓ frame vuoto                                                              |
| 2                 | 2 | [(1,1),(2,1), -]          | ✓ frame vuoto                                                              |
| 3                 | 3 | [(1,1),(2,1),(3,1)]       | ✓ frame vuoto                                                              |
| 4                 | 4 | [(4,1),(2,1),(3,1)]       | ✓ sostituisco LFU → tutti hanno 1 → tie-break: sostituisco più vecchio (1) |
| 5                 | 1 | [(4,1),(1,1),(3,1)]       | ✓ LFU → min=1 → sostituisco 2 (più vecchio)                                |
| 6                 | 2 | [(2,1),(1,1),(3,1)]       | ✓ LFU → min=1 → sostituisco 4                                              |
| 7                 | 5 | [(5,1),(1,1),(2,1)]       | ✓ LFU → min=1 → sostituisco 3                                              |
| 8                 | 1 | [(5,1),(1,2),(2,1)]       | ✗ incremento contatore 1                                                   |
| 9                 | 2 | [(5,1),(1,2),(2,2)]       | ✗ incremento contatore 2                                                   |
| 10                | 3 | [(3,1),(1,2),(2,2)]       | ✓ sostituisco LFU → 5 (conteggio 1)                                        |
| 11                | 4 | [(4,1),(1,2),(2,2)]       | ✓ sostituisco LFU → 3 (conteggio 1)                                        |
| 12                | 5 | [(5,1),(1,2),(2,2)]       | ✓ sostituisco LFU → 4 (conteggio 1)                                        |

**Page fault totali LFU: 9**

### MFU – 3 frame

- Qui sostituisco la pagina **con conteggio più alto**.

| Passo Riferimento |   | Frame (pagina, conteggio) | Fault? Azione / Note                                                                                              |
|-------------------|---|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| 1                 | 1 | [(1,1), -, -]             | ✓ frame vuoto                                                                                                     |
| 2                 | 2 | [(1,1),(2,1), -]          | ✓ frame vuoto                                                                                                     |
| 3                 | 3 | [(1,1),(2,1),(3,1)]       | ✓ frame vuoto                                                                                                     |
| 4                 | 4 | [(4,1),(2,1),(3,1)]       | ✓ sostituisco MFU → tutti 1 → tie-break vecchio → sostituisco 1                                                   |
| 5                 | 1 | [(1,1),(4,1),(3,1)]       | ✓ sostituisco MFU → tutti 1 → sostituisco 2? no è già 4, sostituisco vecchio → 4? dipende tie-break → prendiamo 4 |
| 6                 | 2 | [(2,1),(1,1),(3,1)]       | ✓ sostituisco MFU → tutti 1 → sostituisco 3                                                                       |

| Passo | Riferimento | Frame (pagina, conteggio) | Fault? | Azione / Note                                                          |
|-------|-------------|---------------------------|--------|------------------------------------------------------------------------|
| 7     | 5           | [(5,1),(2,1),(1,1)]       | ✓      | sostituisco MFU → tutti 1 → sostituisco 3 → già fuori → sostituisco 2? |
| 8     | 1           | [(5,1),(2,1),(1,2)]       | ✗      | incremento contatore 1                                                 |
| 9     | 2           | [(5,1),(2,2),(1,2)]       | ✗      | incremento contatore 2                                                 |
| 10    | 3           | [(3,1),(2,2),(1,2)]       | ✓      | sostituisco MFU → max=2 → sostituisco 1 o 2 → sostituiamo 1            |
| 11    | 4           | [(4,1),(2,2),(3,1)]       | ✓      | sostituisco MFU → max=2 → sostituisci 2                                |
| 12    | 5           | [(5,1),(4,1),(3,1)]       | ✓      | sostituisco MFU → max=1 → sostituisci 2? già fuori → sostituisci 4?    |

**Page fault totali MFU:** circa **9–10** (dipende tie-break).

#### 💡 Commento

- **LFU:** mantiene in memoria le pagine più usate → utile per carichi “caldi”.
- **MFU:** sostituisce le pagine più usate → utile quando la pagina nuova ha più probabilità di essere richiesta subito.
- LFU e MFU sono **meno prevedibili** di FIFO, LRU, OPT, perché il conteggio può cambiare molto velocemente.
- Nel nostro esempio, LFU e MFU hanno comportamenti simili a causa della sequenza di accessi breve e dei tie-break basati sull'età.

#### • 1. Allocazione dei frame

- Ogni processo richiede un numero minimo di pagine per poter funzionare correttamente. Ad esempio, sull'IBM 370 un'istruzione di tipo **SS MOVE** può attraversare più pagine, quindi occorrono frame per gestire sia la pagina di origine sia quella di destinazione. Per distribuire i frame disponibili, si usano principalmente due schemi: **allocazione fissa** e **allocazione a priorità**.

#### • 2. Allocazione fissa

- Con l'allocazione fissa, ogni processo riceve un insieme determinato di frame. Se l'allocazione è **omogenea**, tutti i processi prendono lo stesso numero di frame. Se è **proporzionale**, la quantità di frame assegnata dipende dalle dimensioni del processo: processi più grandi ottengono più frame, quelli piccoli meno.

#### • 3. Allocazione a priorità

- L'allocazione a priorità è simile all'allocazione proporzionale, ma considera la priorità dei processi anziché la dimensione. In caso di **page fault**, il sistema può sostituire un frame del processo corrente o, se necessario, uno appartenente a un processo di priorità più bassa, ottimizzando l'uso della memoria per i processi più importanti.

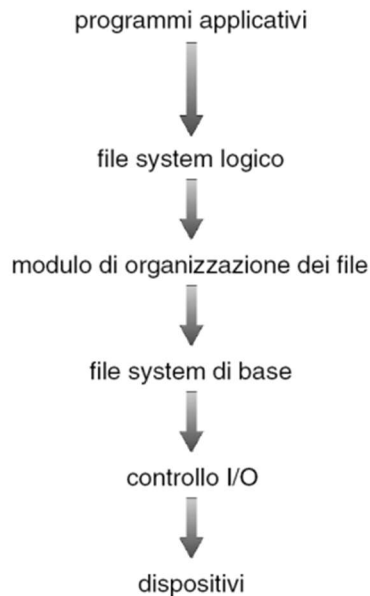
#### • 4. Sostituzione dei frame



- Quando la memoria è piena, bisogna scegliere quale frame liberare. Ci sono due strategie principali: la **sostituzione globale**, dove un processo può usare frame assegnati ad altri processi, e la **sostituzione locale**, dove un processo può sostituire solo i propri frame. La scelta influisce sulle prestazioni complessive del sistema e sulla frequenza dei page fault.
- **5. Thrashing**
- Il **thrashing** si verifica quando un processo non ha abbastanza frame, generando un numero altissimo di page fault. In questa situazione, la CPU viene utilizzata pochissimo perché il sistema spende più tempo a spostare pagine tra memoria e disco che a eseguire il processo. Il fenomeno è dovuto alla somma delle dimensioni delle località dei processi che supera la memoria fisica disponibile.
- **6. Modello di località**
- Il modello di località spiega perché la paginazione funziona: i processi tendono ad accedere a blocchi contigui di memoria (località) che possono sovrapporsi. Capire dove si trova la località attuale permette al sistema operativo di mantenere in memoria solo le pagine realmente necessarie, riducendo i page fault.
- **7. Modello Working Set**
- Il **working set** è l'insieme delle pagine che un processo ha usato recentemente, definito su una finestra temporale  $\Delta$  di riferimenti di pagina. Se la somma dei working set di tutti i processi supera il numero di frame disponibili, si verifica il thrashing. Il working set può essere approssimato tramite interrupt periodici e bit di riferimento per ogni pagina, ma non è mai perfettamente preciso.
- **8. Frequenza delle mancanze di pagina**
- Monitorare il tasso di page fault è fondamentale per ottimizzare l'allocazione della memoria. Un tasso troppo basso indica che il processo ha troppi frame, mentre un tasso troppo alto segnala che ha bisogno di più frame. L'obiettivo è mantenere un equilibrio che riduca i page fault senza sprecare memoria.
- **9. Altre considerazioni**
- Alcuni accorgimenti possono migliorare le prestazioni della memoria virtuale. La **prepaginazione** consiste nel caricare anticipatamente le pagine che saranno probabilmente usate. La **dimensione della pagina** influisce sulla frammentazione, sulla dimensione della tabella delle pagine e sulle operazioni di I/O. La **TLB (Translation Lookaside Buffer)** accelera l'accesso alla memoria; idealmente, il working set di un processo dovrebbe stare nella TLB. Usare pagine di **dimensioni multiple** permette di adattarsi alle esigenze delle applicazioni senza aumentare la frammentazione.
- **10. Struttura dei programmi**
- La disposizione dei cicli di lettura/scrittura negli array può avere un impatto enorme sulle performance. Ad esempio, un accesso riga-per-riga può generare molti page fault, mentre un accesso colonna-per-colonna riduce drasticamente gli accessi non contigui.
- **11. Domanda di segmentazione**
- Quando l'hardware non supporta pienamente la paginazione, la memoria può essere gestita a **segmenti**. Ogni segmento ha un descrittore con un bit di validità che indica se il segmento è in memoria: se è presente, il processo continua l'esecuzione; se non lo è, l'accesso fallisce.
- ---

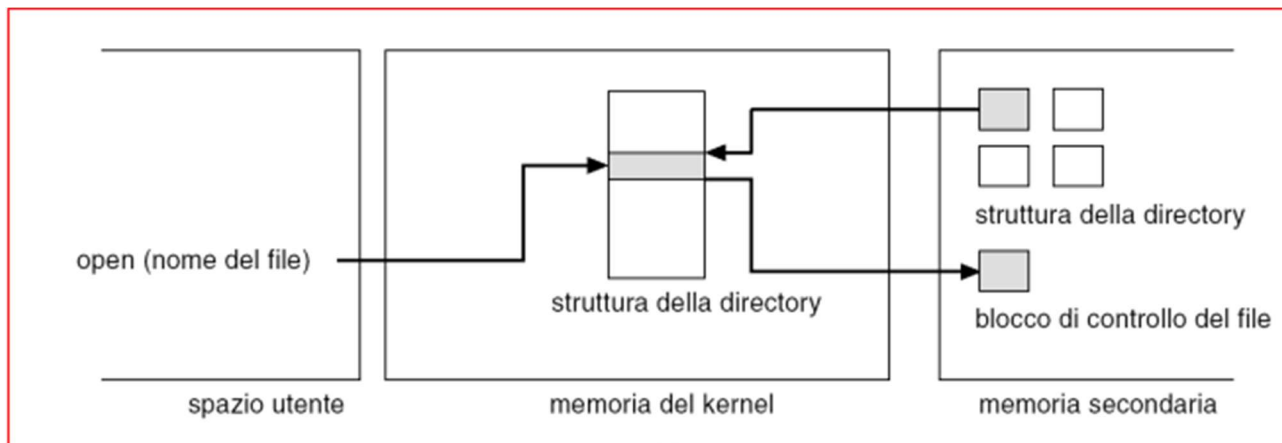
## CAPITOLO 12 FILE SYSTEM

Il file system è memorizzato nella memoria secondaria, organizzato in livelli



## Un tipico blocco di controllo di file

|                                               |
|-----------------------------------------------|
| permessi sul file                             |
| date del file (creazione, accesso, scrittura) |
| proprietario, gruppo, ACL del file            |
| dimensione del file                           |
| blocchi di dati del file                      |



Apertura di un file

### 1. Apertura di un file (parte alta del diagramma)

Quando fai un **open(nome\_del\_file)** nello spazio utente:

#### 1. Richiesta dal processo al kernel

- Il processo in *user space* invia una system call `open()` al kernel, specificando il nome del file.
- Questo nome è solo una stringa: il kernel deve cercare dove si trova e come gestirlo.

#### 2. Accesso alla struttura della directory

- Nel kernel esiste una struttura in memoria che rappresenta la directory.
- Il kernel la usa per localizzare il file nella memoria secondaria (es. disco).
- La directory contiene informazioni su dove si trova il **blocco di controllo del file** (in UNIX chiamato *inode*).

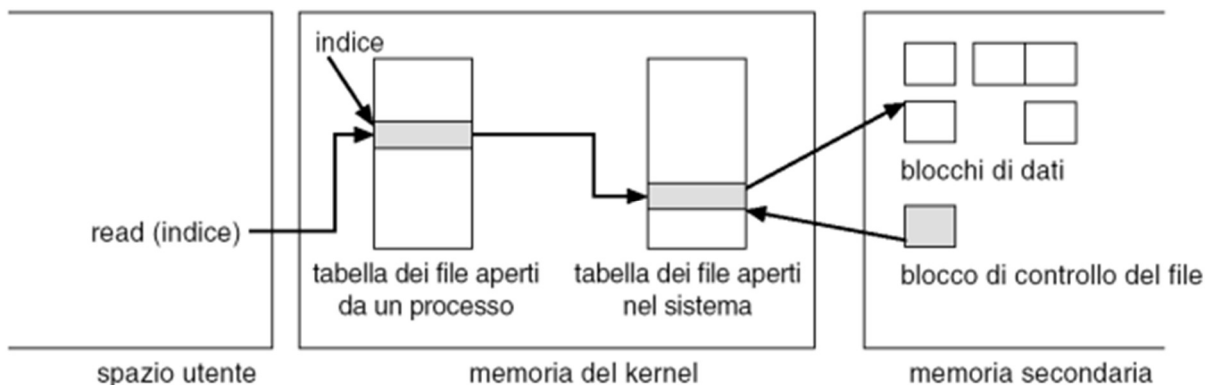
#### 3. Caricamento del blocco di controllo del file

- Il blocco di controllo (*inode*) viene letto dalla memoria secondaria (disco) in memoria del kernel.
- Contiene: dimensione del file, permessi, date, puntatori ai blocchi di dati ecc.

#### 4. Aggiornamento della tabella dei file aperti

- Il kernel crea una voce nella **tabella dei file aperti del sistema**.
- Questa voce punta al blocco di controllo del file.
- Poi crea una voce nella **tabella dei file aperti dal processo** che punta a quella del sistema.

✦ **Risultato finale:** il processo non manipola direttamente il file, ma ha un **file descriptor** (indice) che il kernel usa per trovare tutte queste informazioni.



Lettura di un file

### 2. Lettura di un file (parte bassa del diagramma)

Quando fai una **read(indice)**:

#### 1. Uso del file descriptor

- Nel processo, indice è il file descriptor ritornato da open().
- Questo indice viene usato per trovare la voce corrispondente nella **tabella dei file aperti del processo**.
- 2. **Accesso alla tabella dei file aperti nel sistema**
  - La voce nella tabella del processo punta a una voce nella **tabella dei file aperti del sistema** (memoria del kernel).
  - Qui ci sono informazioni come la posizione corrente del puntatore di lettura/scrittura.
- 3. **Accesso al blocco di controllo del file**
  - La voce del sistema punta al **blocco di controllo del file** (inode) in memoria.
  - Questo inode contiene i puntatori ai **blocchi di dati** reali del file nella memoria secondaria.
- 4. **Lettura dei dati**
  - Il kernel legge i blocchi di dati richiesti dal disco (o dalla cache se già in RAM).
  - I dati vengono copiati dallo spazio kernel allo spazio utente del processo.

✦ **Risultato finale:** il processo riceve i byte richiesti, senza mai accedere direttamente alla struttura del file nel disco — è sempre il kernel che gestisce l'accesso.

Il **VFS** è uno strato software all'interno del kernel che agisce come **intermediario** tra le chiamate ai file fatte dai programmi (tramite le API) e i veri file system che gestiscono fisicamente i dati su disco.

Funziona con un'idea "**a oggetti**":

- Ogni elemento (file, directory, dispositivo, ecc.) è trattato come un **oggetto** con metodi e attributi.
- Quando un'applicazione fa, ad esempio, una open() o read(), in realtà non sta parlando direttamente con il file system del disco, ma con il VFS, che sa quale tipo di file system c'è dietro (ext4, FAT32, NFS, ecc.) e lo chiama in modo appropriato.

Il vantaggio è che **l'API rimane la stessa**, indipendentemente dal tipo di file system reale:

- Il programma vede solo funzioni standard (open, read, write, close...)
- Il VFS traduce queste operazioni nelle chiamate specifiche del file system concreto.

Per la realizzazione delle directory si utilizzano delle tecniche che sono semplici da programmare: si usa una lista di nomi di file con puntatori ai blocchi dei dati e una tabella di hash ossia un metodo che utilizza una lista e una tabella di hash

per allocare i file all'interno del file system si possono usare diversi metodi di allocazione

- **ALLOCAZIONE CONTIGUA**
- 

**Come funziona**

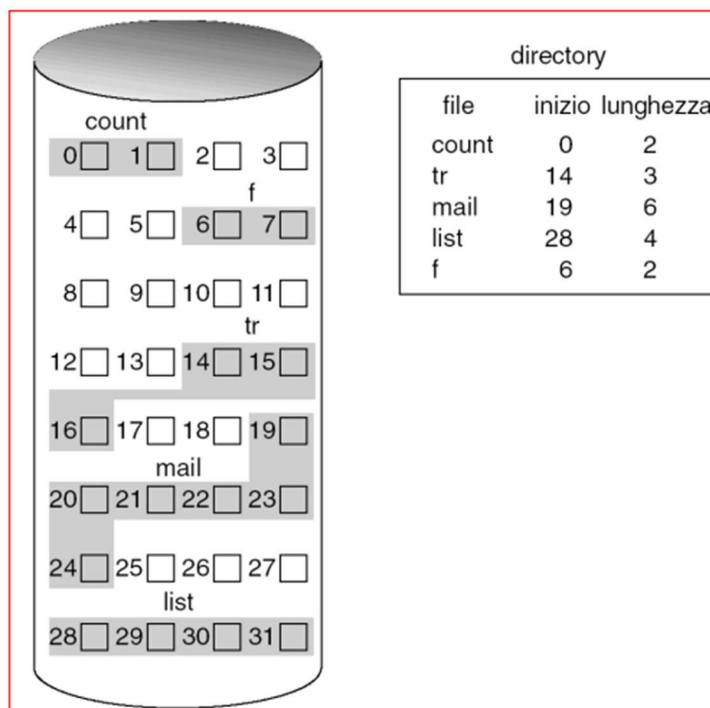
- Quando crei un file, il sistema trova un insieme di **blocchi su disco fisicamente consecutivi**.
- Per identificarlo basta sapere **dove inizia** (blocco iniziale) e **quanto è lungo** (numero di blocchi).
- Vantaggio:

- È **molto veloce** per lettura sequenziale (i blocchi sono già in ordine).
- Supporta facilmente anche l'**accesso diretto**: basta calcolare l'offset partendo dal blocco iniziale.

---

### Problemi principali

1. **Frammentazione esterna** – Col tempo, lo spazio libero si frammenta in pezzetti sparsi e diventa difficile trovare un'area contigua grande abbastanza per un nuovo file.
2. **Espansione limitata** – Se il file cresce oltre lo spazio inizialmente allocato, bisogna spostarlo altrove (costoso) o non è possibile ampliarlo.
3. **Spreco di spazio** – Un po' come l'allocazione dinamica della memoria in RAM, si rischia di lasciare "buchi" inutilizzati.



### • ALLOCAZIONE COLLEGATA

#### Come funziona

- Il file system memorizza solo l'indirizzo del **primo blocco** (e a volte anche dell'ultimo).
- Ogni blocco del file ha una parte di spazio per i dati e una piccola sezione per il puntatore al **blocco successivo**.
- I blocchi possono trovarsi **ovunque** sul disco, non devono essere contigui.

---

#### Vantaggi

- **Nessuna frammentazione esterna**: qualunque blocco libero può essere usato.
- **Facile espansione**: basta aggiungere un blocco alla fine della catena.
- **Semplice gestione dello spazio**: serve solo tenere traccia del primo blocco.

---

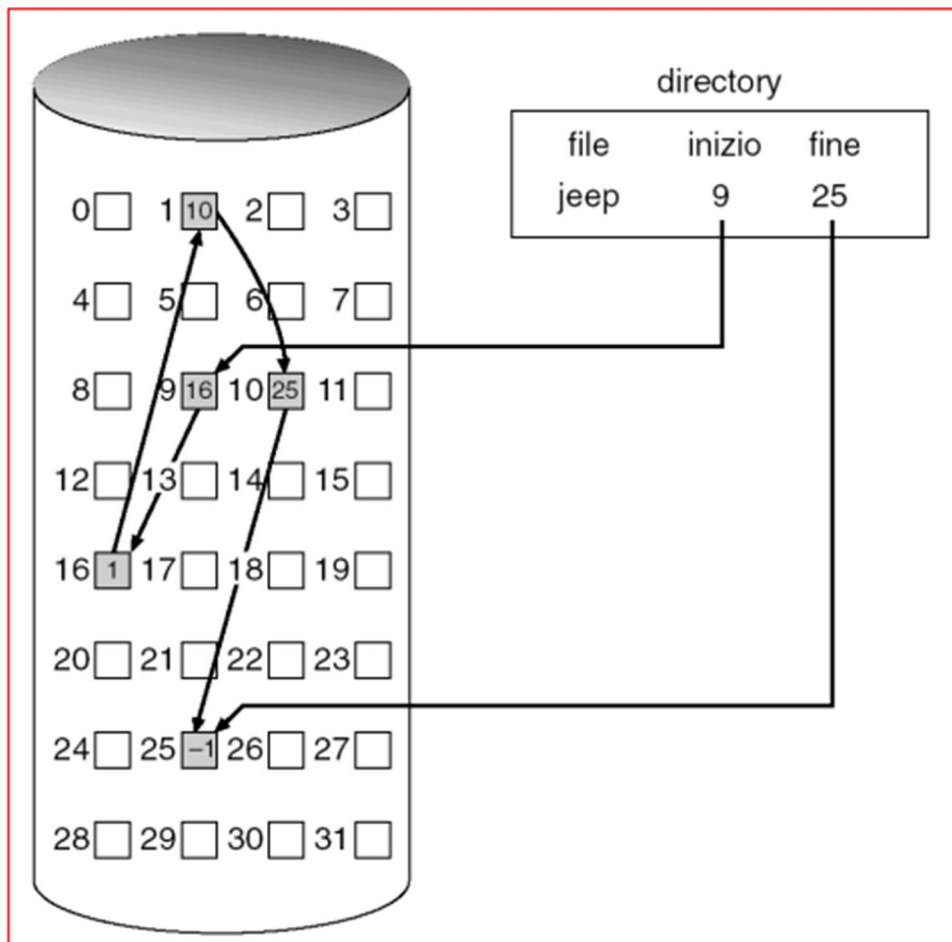
#### Svantaggi

1. **Niente accesso casuale** – Per arrivare a un blocco in mezzo al file, devi partire dal primo e seguire i puntatori uno a uno.
2. **Overhead di spazio** – Ogni blocco deve riservare un po' di spazio per il puntatore, quindi meno spazio effettivo per i dati.

3. **Rischio di perdita di dati** – Se un puntatore si corrompe, si perde tutta la parte di file dopo quel punto.

### Mappatura

- È la tabella o il meccanismo che permette di sapere dove si trova il primo blocco del file e come seguire la catena.
- Nei sistemi più moderni, la mappatura è spesso centralizzata in strutture come la **FAT (File Allocation Table)**, che elimina il bisogno di memorizzare i puntatori dentro i blocchi stessi.



### ALLOCAZIONE INDICIZZATA

#### Come funziona

- Ogni file ha un **blocco indice** dedicato che contiene **tutti i puntatori** ai blocchi di dati del file.
- I dati **non** contengono puntatori (al contrario dell'allocazione collegata).
- Il sistema, per leggere un blocco, consulta il blocco indice per sapere dove si trova sul disco.

#### Vantaggi

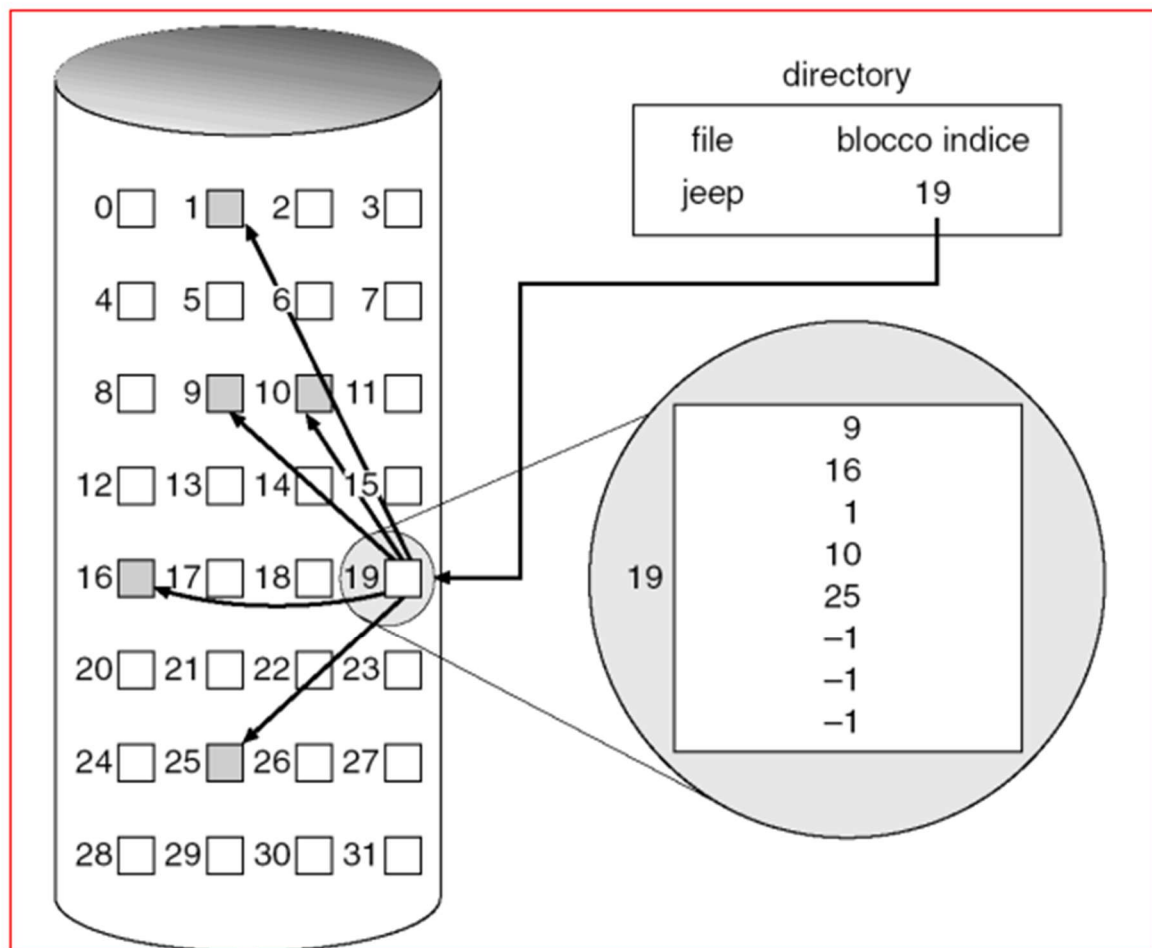
1. **Accesso casuale diretto** – Basta leggere il blocco indice e saltare subito al blocco desiderato, senza scorrere una catena di puntatori.
2. **Nessuna frammentazione esterna** – I blocchi possono stare ovunque sul disco.
3. **Gestione semplice della mappatura** – La logica “numero di blocco → indirizzo fisico” è tutta nel blocco indice.

#### Svantaggi

- **Overhead:** serve un blocco in più (il blocco indice) che occupa spazio, soprattutto se il file è piccolo.
- **Dimensione massima del file limitata** dal numero di puntatori che il blocco indice può contenere.
  - Nell'esempio che dai:
    - Blocco = **512 parole**
    - File max = **256K parole**
    - Ogni puntatore indica un blocco → serve un solo blocco indice per mappare tutti i blocchi del file.

### Mappatura logico → fisico

1. Prendo il numero logico di blocco del file (es. blocco logico 37).
2. Vado nel blocco indice alla posizione 37.
3. Leggo il puntatore al blocco fisico sul disco.
4. Leggo i dati dal blocco fisico.



Certo, parliamo dell'**allocazione indicizzata a più livelli**, che è l'evoluzione diretta di quella indicizzata semplice che hai appena visto.

### Perché serve

Nell'allocazione indicizzata semplice, la dimensione massima di un file è limitata dal numero di puntatori che può contenere **un singolo blocco indice**.

Se il blocco indice è pieno, non c'è più spazio per aggiungere puntatori → non si possono mappare altri blocchi.

L'idea della **multi-level indexing** è:

Quando un blocco indice non basta, lo facciamo puntare ad altri blocchi indice.

---

### Come funziona

- **Primo livello:** hai un *indice principale* (Primary Index Block).
  - Ogni voce di questo indice **non punta a un blocco dati**, ma a un **blocco indice di secondo livello**.
  - **Secondo livello:** ogni blocco indice di secondo livello contiene i puntatori ai blocchi dati veri e propri.
- ✚ Se non basta neanche questo, si può estendere a **più livelli** (es. 3, 4...), ma già con 2 livelli si raggiungono dimensioni enormi.

---

### Esempio pratico

Supponiamo:

- **Blocco** = 512 byte
- **Puntatore** = 4 byte →  $512 / 4 = 128$  **puntatori per blocco**

#### Allocazione a 1 livello (semplice)

- 1 blocco indice = 128 puntatori → 128 blocchi dati → file max =  $128 \times 512$  byte = **64 KB**

#### Allocazione a 2 livelli

- Primo blocco indice: 128 puntatori → ognuno punta a un **blocco indice di 2° livello**
- Ogni blocco indice di 2° livello → 128 puntatori a blocchi dati
- Capacità totale:  $128 \times 128 \times 512$  byte = **8 MB**

#### Allocazione a 3 livelli

- Primo blocco indice → puntatori a blocchi di 2° livello
- Blocchi di 2° livello → puntatori a blocchi di 3° livello
- Blocchi di 3° livello → puntatori a blocchi dati
- Capacità:  $128 \times 128 \times 128 \times 512$  byte = **1 GB** circa

---

### Vantaggi

- ✓ Permette file enormi senza spreco di spazio per file piccoli.
- ✓ Facile da implementare.

### Svantaggi

- ✗ Accesso più lento: per trovare un blocco dati a livello 3, devo leggere **tre blocchi indice** prima di arrivare ai dati.
- ✗ Maggior overhead di metadati (più blocchi indice).

## Struttura dei dischi e mappatura logica-fisica

Il disco viene visto dal sistema operativo come un **grande array monodimensionale di blocchi logici**.

Ogni blocco logico è la più piccola unità che può essere letta o scritta.

Per poter accedere fisicamente ai dati, questi blocchi logici devono essere **mappati** nelle reali coordinate del disco, che sono: **cilindro – traccia – settore**.

Il blocco logico 0 corrisponde al primo settore della prima traccia del cilindro più esterno.

I blocchi successivi vengono mappati sequenzialmente avanzando prima nei settori, poi nelle tracce e infine nei cilindri.

Questa traduzione (logico → fisico) permette al sistema di sapere **dove posizionare la testina** per poter leggere un determinato blocco.



---

## Schedulazione degli accessi al disco

Una volta mappato, il blocco deve essere effettivamente letto. Qui diventa importante **come** il sistema operativo pianifica gli accessi al disco, perché un disco ha parti meccaniche che si muovono e quindi introduce ritardi.

Le prestazioni dipendono da due tempi principali:

| Componente                          | Descrizione                                                                  |
|-------------------------------------|------------------------------------------------------------------------------|
| <b>Tempo di ricerca</b> (seek time) | tempo necessario per spostare il braccio della testina sul cilindro corretto |
| <b>Latenza di rotazione</b>         | tempo necessario perché il settore desiderato “passi sotto” la testina       |

La somma di questi due rappresenta il **tempo di accesso**.

Per migliorare le prestazioni bisogna **minimizzare il tempo di ricerca**, che a sua volta dipende essenzialmente dalla **distanza** che il braccio deve percorrere.

La **larghezza di banda del disco**, invece, indica quanta informazione viene trasferita in rapporto al tempo totale impiegato e si calcola così:

$$\text{bandwidth} = \frac{\text{byte trasferiti}}{\text{tempo totale (richiesta} \rightarrow \text{completamento)}}$$

In altre parole:

meno tempo spendo a muovere la testina e aspettare che il disco giri → più byte riesco a trasferire → maggiore è la larghezza di banda.

Poiché il tempo di accesso al disco dipende in gran parte dalla **posizione delle richieste** sul disco e dalla **distanza** che la testina deve percorrere tra una richiesta e l'altra, il sistema operativo utilizza **algoritmi di schedulazione del disco** per decidere *in quale ordine* servirle.

L'obiettivo di questi algoritmi è proprio quello di **ridurre la distanza di movimento del braccio**, e quindi **minimizzare il tempo di ricerca** e **massimizzare la larghezza di banda**.

A questo punto si possono introdurre i principali algoritmi:

---

Tra gli algoritmi più utilizzati troviamo:

- **FCFS (First-Come First-Served)** → serve le richieste in ordine di arrivo, semplice ma poco efficiente se le richieste sono sparse.
- **SSTF (Shortest Seek Time First)** → seleziona la richiesta più vicina alla posizione attuale della testina.
- **SCAN** → la testina si muove in una direzione servendo tutte le richieste che incontra, poi inverte direzione (anche detto “ascensore”).
- **LOOK** → come SCAN, ma inverte direzione solo quando non ci sono più richieste in quella direzione.

- **C-SCAN / C-LOOK** → versioni circolari che trattano il disco come una struttura ciclica, migliorando l'equità nelle richieste..

### FCFS

- Le richieste sono gestite in ordine di arrivo

Coda richiesta:  $[98, 183, 37, 122, 14, 124, 65, 67]$

Posizione testina = 53

$53 \rightarrow 98 \rightarrow 183 \rightarrow 37 \rightarrow 122 \rightarrow 14 \rightarrow 124 \rightarrow 65 \rightarrow 67$

$$98 - 53 = 45$$

$$124 - 65 = 59$$

$$183 - 98 = 85$$

$$67 - 65 = 2$$

$$183 - 37 = 146$$

$$122 - 37 = 85$$

Spostamenti totali = 640

$$122 - 14 = 108$$

$$124 - 14 = 110$$

6

### SSTF

- Seleziona ad ogni passo la richiesta più vicina alla posizione della testina

$[~~98~~, 183, ~~37~~, ~~122~~, ~~14~~, ~~124~~, ~~65~~, ~~67~~]$

Posizione testina = 53

$53 \rightarrow 65 \rightarrow 67 \rightarrow 37 \rightarrow 14 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$

$$65 - 53 = 12$$

$$98 - 14 = 84$$

$$67 - 65 = 2$$

$$122 - 98 = 24$$

$$67 - 37 = 30$$

$$124 - 122 = 2$$

$$37 - 14 = 23$$

$$183 - 124 = 59$$

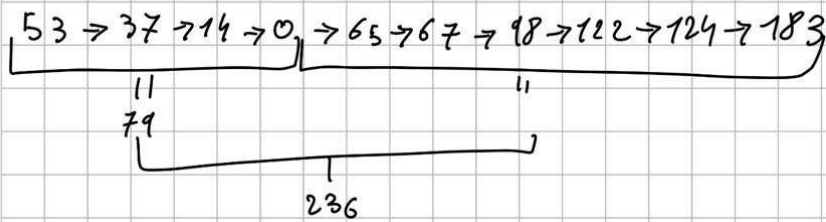
$\rightarrow \Sigma = 236$  spostamenti

## SCAN

- muove la testina in una direzione prefinita, prendendo tutte le richieste che incontra fino alla fine, poi torna indietro e soddisfa

Coda richieste: [~~98~~, ~~183~~, ~~37~~, ~~122~~, ~~14~~, ~~124~~, ~~65~~, ~~67~~]

Posizione testina = 53      Direzione: ←



## LOOK

- simile allo scan, solo che all'ultima richiesta, non va all'estremo e soddisfa quelle che incontra

53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

222 spostamenti

### C-SCAN

• Come lo scan, solo che una volta arrivati all'ultima richiesta  
testina, arriva fino a un capo e ritorna all'altro senza soddisfare

53 → 65 → 67 → 98 → 112 → 124 → 183 → 199 → 0 → 11 → 37

982 spostamenti

### C-LOOK

Come C-SCAN, però ora invece di arrivare ad un estremo  
passo alla richiesta successiva più vicina all'altro estremo

53 → 65 → 67 → 98 → 112 → 124 → 183 → 11 → 37

es lezione

Si consideri un hard disk con richiesta in corso di servizio al cilindro 41,  
ultima richiesta precedentemente servita al cilindro 35 e con la seguente  
coda di richieste:

23, 16, 54, 47, 101, 33, 78, 95

Indicare sequenza di servizio e spostamento totale della testina per una  
schedulazione con algoritmo dell'ascensore unidirezionale (C-SCAN)  
sapendo che il disco consta di 120 cilindri (da 0 a 119). Inoltre, si stabilisca il  
tempo di seek totale risultante sapendo che il suo valore unitario è di 0,11  
msec.

→ direzione  
→

41 → 47 → 54 → 78 → 95 → 101 → 119 → 0 → 16 → 23 → 33

6 + 7 + 24 + 17 + 6 + 18 + 11 + 16 + 7 + 10 = 230 distanza ricoperta

tempo seek Tot = 230 · 0,11 = 25,3 msec